

Goal-Guided, Inference-Accelerated Algorithmic Chemistry on METTA + MORK

Architecture, Design, and Implementation Reference

. Software architecture, attention economy, hardware targets, benchmark suite, and full
implementation reference for algorithmic chemistry on METTA/MM2/MORK

Ben Goertzel and various LLMs

May 6, 2026

Abstract

This document presents a design for algorithmic chemistry on top of the Hyperon stack, together with hardware targets, an evaluation plan, and the implementation reference needed to actually build the system. It specifies a two-tier weighted rewrite-rule chemistry whose inner loop runs as fast, phase-local MM2 subroutines over indexed atoms in MORK, and whose outer loop is a slower structural-evolution layer using MOSES, gCoDD, CENF, AIRIS, and PLN. Sitting between the inner and outer loops is an attention economy based on ECAN (Economic Attention Networks) with an information-geometric flavor: ECAN maintains Short Term Importance (STI) and Long Term Importance (LTI) currencies on atoms, spreads attention along Hebbian and InverseHebbian association links, charges rent and pays wages from stimulus, and decides which rules are cold, warm, or hot at each moment. ACTPC-CHEM learns which rules predict and act well; ECAN decides which atoms get enough attention and memory to matter computationally. The two distributions are coupled but not identified.

The execution model is deliberately backend-portable. A single MM2 reaction chamber is treated as single-threaded: within one chamber, MM2 executes a small, deterministic, context-local hot loop whose ordered trace is used for credit assignment, ACS/RAF detection, replay, and debugging. Parallelism is expressed one level up, as a graph of coarse-grained workers. The long-term target is to express these workers in METTA-IL using Rholang-style `par`; until that runtime is ready, the same worker graph can be implemented as a PETTA/SWI-Prolog wrapper layer. The portability boundary is a Worker IR consisting of MORK facts for workers, chambers, snapshots, event logs, par-groups, reducers, budgets, and outputs. PETTA executes this IR now; METTA-IL should execute the same IR later.

The document recommends concrete hardware and population-sizing regimes (cold reservoir of millions of rules, warm active subset of tens of thousands, hot MM2 working set of hundreds-to-thousands per cognitive tick), with an emphasis on CPU- and RAM-heavy bare-metal servers rather than GPU-centric configurations. It proposes a staged benchmark suite whose central object is not a task score but a *useful active autocatalytic set*: a recurrent network of rules, intermediate artifacts, lemmas, or program fragments whose autocatalytic organization is causally responsible for useful and transferable external behavior.

The body of the document presents concepts, architecture, design principles, sizing recommendations, and evaluation philosophy. The appendices contain the detailed material needed to actually build the system: MORK atom schemas, per-phase pseudo-MM2 code, inner-loop pseudocode, kernel implementations, the ECAN kernel suite, the worked virtual-bug example, the rule compilation pipeline, the benchmark harness and ACS detection pseudocode, the configuration files, and the mathematical summary. Most readers will only need the body; implementers and coding LLMs will need the appendices.

The document is written for Hyperon-team insiders with mixed expertise: it provides bridge sections so that those who know MORK/MM2 internals well can quickly orient on ACTPC-CHEM and autocatalysis, those who know algorithmic chemistry and origin-of-life autocatalytic-set theory can quickly orient on MORK/MM2 operational semantics, and those unfamiliar with ECAN can quickly orient on its economic and information-geometric formulations.

Contents

1	How to read this document	3
2	Executive summary	3
2.1	What we are building	3
2.2	Architecture in three layers	3
2.3	Core design claims	5
2.4	Hardware and sizing in one glance	6
2.5	What success should look like	7
3	Orientation bridge: ACTPC-CHEM and autocatalytic sets, for MORK insiders	8
3.1	What is ACTPC-CHEM?	8
3.2	Why “chemistry” and not just “learner”?	8
3.3	Active autocatalytic sets and RAFs	9
3.4	Useful active autocatalytic sets	9
3.5	Why this is hard, and where ACTPC-CHEM fits	10
4	Orientation bridge: MORK and MM2, for chemistry/autocatalysis insiders	10
4.1	What MORK is	10
4.2	What MM2 is	10
4.3	What MOSES-on-MORK contributes	11
4.4	What AIRIS and PLN contribute	11
5	Orientation bridge: ECAN attention economics	12
5.1	What ECAN solves	12
5.2	STI, LTI, and the Attentional Focus	12
5.3	The economic metaphor	13
5.4	Hebbian and InverseHebbian links	13
5.5	Diffusion and attractor behavior	13
5.6	Information-geometric ECAN	13
5.7	Why ECAN fits algorithmic chemistry	14
5.8	What ECAN is not	14
6	Glossary	14
7	High-level architecture	16
7.1	Subsystems	16
7.2	Separation of responsibilities	17
7.3	Portable worker orchestration: PETTA now, METTA-IL later	18

8	Data model	19
8.1	General atom design rules	19
8.2	Identifier conventions	19
9	Rule representation	19
9.1	Three-level representation	19
9.2	Why not match directly on source rules?	20
10	Inner loop: MM2 phase architecture	20
10.1	Phase overview	20
11	Grounded kernel interfaces	21
11.1	Why grounded kernels are required	21
11.2	Kernel API table	21
12	Inner loop pseudocode	22
12.1	Top-level tick	22
13	Wasserstein natural-gradient module	23
13.1	Purpose	23
13.2	Rule-space graph	23
13.3	When to apply OT	23
14	Outer loop: structural evolution with MOSES, gCoDD, and CENF	23
14.1	Outer-loop role	23
14.2	Structural job types	24
14.3	Rule mutation operators	24
15	AIRIS and PLN integration	25
15.1	Design principle	25
16	Attention economy: ECAN integration	25
16.1	The two distributions, and what they do	25
16.2	Working memory and long-term memory under ECAN	25
16.3	Atom and link schemas	26
16.4	Stimulus sources	26
16.5	Core ECAN dynamics	27
16.6	Family-level structures and anti-precedence kernels	28
16.7	Sparse Hebbian diffusion	29
16.8	Pool migration	29
16.9	Hot-pool quotas	29
16.10	Information-geometric ECAN	30
16.11	Coupling ECAN and ACTPC-CHEM	30
16.12	Division of labor: MM2, METTA, and grounded kernels	31
16.13	Grounded ECAN kernels	32
16.14	Worker-level ECAN	32
17	MORK-specific performance strategy	33
17.1	Hot-path query rules	33

18	Hardware targets and population sizing	33
18.1	Why a CPU-focused server is preferred	33
18.2	Recommended machine classes	34
18.2.1	Starter engineering server	34
18.2.2	Default serious single-server target	35
18.2.3	Aggressive single-node target	35
18.3	Cold, warm, and hot populations	35
18.4	Inner MM2-loop sizing	36
18.4.1	Recommended hot-loop caps	36
18.4.2	MM2 tick skeleton with bounding	37
18.4.3	Why not all rules hot?	38
18.5	Persistent rule-soup memory sizing	38
18.6	MOSES/gCoDD/CENF outer-loop sizing	38
18.6.1	Default MOSES parameters	38
18.6.2	Outer-loop rhythm	39
18.7	Trace and checkpoint requirements	39
18.8	Role of GPUs	40
18.9	Bare metal versus cloud VM	40
18.10	NUMA and process layout	41
19	Evaluation: useful autocatalytic generativity	42
19.1	Benchmark philosophy	42
19.2	Pareto objective, not a single fitness function	42
19.2.1	Autocatalysis–utility coupling	43
19.3	Core diagnostics	43
19.3.1	ACS / RAF metrics	43
19.3.2	Usefulness metrics	43
19.3.3	Generativity metrics	43
19.3.4	Heredity metrics	43
19.3.5	Anti-rigidity metrics	44
19.3.6	Adaptive-continuity and anti-precedence metrics	44
19.3.7	Attention-economy metrics	45
19.3.8	Near-ACS defect score	45
19.3.9	Rare-tail enrichment	45
19.3.10	Sclerosis index	46
19.4	Benchmark series	46
19.4.1	Benchmark 0: ACS ignition and habit-trap calibration	46
19.4.2	Benchmark 1: productive grammar ecology	47
19.4.3	Benchmark 2: autocatalytic abstraction discovery	47
19.4.4	Benchmark 3: lemma / proof autocatalysis	48
19.4.5	Benchmark 4: program-fragment repair and synthesis ecology	49
19.4.6	Benchmark 5: blackboard protocol alchemy	49
19.4.7	Benchmark 6: serial transfer and daughter-soup inheritance	50
19.4.8	Benchmark 7: adaptive continuity under implementation change	50
19.5	Geoteleomic guidance, precedence, and value-relative anti-precedence	51
19.5.1	From broad anti-resonance to value-relative anti-precedence	51
19.5.2	Pattern families	52
19.5.3	Excess canalization	52

19.5.4	What “value” means here	53
19.5.5	Functional protection	53
19.5.6	Structured release	53
19.5.7	The recommended effective rule selection	54
19.5.8	Anti-precedence on rule-space transport	54
19.5.9	The simple-baseline form	54
19.6	Experimental matrix	54
19.7	Minimum practical suite and recommended first benchmarks	55
19.8	Anti-cheat and anti-rigidity rules	56
20	Implementation and scaling plan	57
20.1	Minimal initial implementation	57
20.2	PeTTA wrapper runtime as the first worker backend	57
20.3	Scaling sequence	59
21	Failure modes and mitigations	60
22	Conclusion	63
A	Atom schemas	64
A.1	General atom design rules	64
A.2	Identifier conventions	65
A.3	Persistent fact schemas	65
A.3.1	Context and tick facts	65
A.3.2	Portable worker IR facts	65
A.3.3	Observation and feature facts	67
A.3.4	Rule metadata	67
A.3.5	Compiled rule facts	67
A.3.6	Rule source facts	68
A.3.7	Candidate, chosen, and action facts	68
A.3.8	Prediction and outcome facts	68
A.3.9	Trace and credit facts	68
A.3.10	Rule-space graph facts	69
A.3.11	Pattern-family facts (anti-precedence)	69
A.3.12	Meta-guard and abstraction facts	70
A.3.13	gCoDD and MOSES facts	70
B	Rule representation	70
B.1	Three-level representation	70
B.2	Example source rule	71
B.3	Example compiled representation	71
B.4	Why not match directly on source rules?	71
C	Inner loop: MM2 phase architecture	71
C.1	Phase overview	71
C.2	Phase atoms	72
C.3	Pseudo-MM2 notation	72
C.4	Chamber contract: MM2 is local and single-threaded	72
C.5	Phase 00: initialize tick	73

C.6	Phase 10: ingest observations	73
C.7	Phase 20: derive features	73
C.8	Phase 30: activate signatures	74
C.9	Phase 40: match candidates	74
C.10	Phase 50: score candidates	74
C.11	Phase 60: sample or select	75
C.12	Phase 70: fire action and trace	75
C.13	Phase 80: environment step	75
C.14	Phase 100: compute error	76
C.15	Phase 110: assign credit	76
C.16	Phase 120: update rule weights	76
C.17	Phase 130: trigger structural evolution	77
C.18	Phase 140: cleanup	77
D	Grounded kernel interfaces	77
D.1	Why grounded kernels are required	77
D.2	Kernel API table	78
D.3	Host-side data structures	79
E	Inner loop pseudocode	79
E.1	Top-level tick	79
E.2	PETTA worker orchestration around single-threaded chambers	80
E.3	Backend translation principle	81
E.4	Scoring candidates	81
E.5	Softmax sampling	83
E.6	Prediction distribution	84
E.7	Epistemic reward	84
E.8	Credit assignment	85
E.9	Weight update: simple online version	85
E.10	Delayed rewards	86
F	Wasserstein natural-gradient module	86
F.1	Purpose	86
F.2	Rule-space graph	86
F.3	Measure-dependent Laplacian	87
F.4	OT step pseudocode	87
F.5	When to apply OT	88
F.6	OT host request	88
G	Outer loop: structural evolution with MOSES, gCoDD, and CENF	88
G.1	Outer-loop role	88
G.2	Structural job types	89
G.3	Rule mutation operators	89
G.4	Structural loop pseudocode	89
G.5	CENF meta-guard creation	90
G.6	Compiling meta-guards	91
G.7	MOSES factor-graph EDA	91
G.8	Sampling candidate rules from a deme	91

G.9	Inserting trial rules	92
H	AIRIS and PLN integration	92
H.1	Design principle	92
H.2	AIRIS causal rule proposal	92
H.3	PLN abstraction proposal	93
H.4	Proposal merger	93
I	Attention economy: ECAN integration	94
I.1	The two distributions, and what they do	94
I.2	Working memory and long-term memory under ECAN	94
I.3	Atom and link schemas	95
I.3.1	Attention facts	95
I.3.2	Worker-level attention facts	95
I.3.3	Anti-precedence facts (family-level)	95
I.3.4	Association links	96
I.3.5	Stimulus events	96
I.3.6	ACS macro-atoms	96
I.4	Stimulus sources	97
I.5	Core ECAN dynamics	97
I.6	Sparse Hebbian diffusion	98
I.7	Pool migration	99
I.8	Hot-pool quotas	99
I.9	ECAN-aware retrieval rules	99
I.10	Pseudocode: ECAN-lite update	100
I.11	Pseudocode: worker-level ECAN	101
I.12	Information-geometric ECAN	102
I.13	Coupling ECAN and ACTPC-CHEM	103
I.14	Division of labor: MM2, METTA, and grounded kernels	103
I.15	Grounded ECAN kernels	104
I.16	Memory retrieval examples	105
J	MORK-specific performance strategy	105
J.1	Hot-path query rules	105
J.2	Good and bad schemas	106
J.3	Candidate caps	106
J.4	Garbage collection	107
J.5	Sharding	107
J.6	PETTA / METTA-IL snapshot-log-reducer concurrency	107
K	Hardware targets and population sizing	108
K.1	Why a CPU-focused server is preferred	108
K.2	Recommended machine classes	108
K.2.1	Starter engineering server	108
K.2.2	Default serious single-server target	109
K.2.3	Aggressive single-node target	109
K.3	Cold, warm, and hot populations	110
K.4	Inner MM2-loop sizing	110

K.4.1	Recommended hot-loop caps	110
K.4.2	MM2 tick skeleton with bounding	111
K.4.3	Why not all rules hot?	112
K.5	Persistent rule-soup memory sizing	112
K.6	MOSES/gCoDD/CENF outer-loop sizing	113
K.6.1	Default MOSES parameters	113
K.6.2	Outer-loop rhythm	113
K.7	Trace and checkpoint requirements	114
K.8	Role of GPUs	114
K.9	Bare metal versus cloud VM	115
K.10	NUMA and process layout	115
K.10.1	Worker placement	116
K.11	Autotuning policy	116
K.12	Operational acceptance criteria	118
L	Evaluation: useful autocatalytic generativity	118
L.1	Benchmark philosophy	118
L.2	Pareto objective, not a single fitness function	118
L.2.1	Autocatalysis–utility coupling	119
L.3	Core diagnostics	119
L.3.1	ACS / RAF metrics	119
L.3.2	Usefulness metrics	119
L.3.3	Generativity metrics	120
L.3.4	Heredity metrics	120
L.3.5	Anti-rigidity metrics	120
L.3.6	Attention-economy metrics	120
L.3.7	Near-ACS defect score	121
L.3.8	Rare-tail enrichment	121
L.3.9	Sclerosis index	121
L.4	Benchmark series	121
L.4.1	Benchmark 0: ACS ignition and habit-trap calibration	121
L.4.2	Benchmark 1: productive grammar ecology	122
L.4.3	Benchmark 2: autocatalytic abstraction discovery	123
L.4.4	Benchmark 3: lemma / proof autocatalysis	124
L.4.5	Benchmark 4: program-fragment repair and synthesis ecology	124
L.4.6	Benchmark 5: blackboard protocol alchemy	125
L.4.7	Benchmark 6: serial transfer and daughter-soup inheritance	125
L.5	Geoteleomic guidance, precedence, and value-relative anti-precedence	126
L.5.1	Three-layer control	126
L.5.2	Family-level quantities	126
L.5.3	Functional-protection coefficient	127
L.5.4	Effective rule-selection probability	127
L.5.5	Anti-precedence on rule-space transport	127
L.5.6	Trace updates	127
L.6	Experimental matrix	127
L.7	Benchmark harness pseudocode	128
L.8	ACS detection pseudocode	130
L.9	Minimum practical suite and recommended first benchmarks	130

L.10	Anti-cheat and anti-rigidity rules	131
M	Implementation and scaling plan	132
M.1	Repository layout	132
M.2	Minimal initial implementation	133
M.3	Implementation order for an LLM coder	133
M.4	Scaling sequence	134
M.5	Mapping integration milestones to scaling stages	136
M.6	Promotion from trial to active	137
N	Worked example: virtual bug food and poison	137
N.1	Initial facts	137
N.2	Specialized learned rules	137
N.3	Feature-combination abstraction	138
N.4	Delayed conditional poison	138
N.5	End-to-end example tick	139
N.6	Acceptance criteria for the virtual bug	139
O	Compilation pipeline	140
O.1	Compile source rule to GCoDD	140
O.2	Compile GCoDD to hot-path atoms	140
O.3	Choose primary trigger	141
P	Logging, observability, and testing	141
P.1	Event log facts	141
P.2	Debug snapshots	141
P.3	Invariants to check	142
P.4	Unit tests for grounded kernels	143
Q	Guidance for coding LLMs	143
R	Pseudo-MM2 skeleton	145
S	Rust-like kernel skeleton	147
T	Mathematical summary	151
T.1	Predicted and observed distributions	151
T.2	Prediction error	151
T.3	Reward	151
T.4	Simple logit update	151
T.5	Wasserstein-style update	151
T.6	Resonance / anti-resonance modifier (legacy baseline)	151
T.7	Value-relative anti-precedence (recommended)	152
T.8	Coupling score	152
T.9	ECAN attention distribution	152
T.10	ECAN economic update	152
T.11	Mirror-descent ECAN step	153
T.12	Wasserstein ECAN step	153
T.13	Joint ACTPC-CHEM/ECAN regularizer	153

T.14 Combined selection score	153
U Recommended initial configuration files	153
U.1 Starter config	153
U.2 Default serious config	156
U.3 Minimal experiment configuration	160
V References and further reading	161

1 How to read this document

The body of this document covers four things in sequence: a software design for algorithmic chemistry on METTA/MM2/MORK; an ECAN-based attention economy that mediates between the chemistry and the rest of the cognitive system; concrete hardware and population-sizing recommendations for the combined system; and a benchmark suite for evaluating whether the resulting system actually exhibits the kind of useful autocatalytic generativity the design is aimed at.

The appendices contain reference material: detailed MORK atom schemas, per-phase pseudo-MM2 code, inner-loop pseudocode, MOSES/AIRIS/PLN pseudocode, ECAN kernel implementations, MORK performance examples, autotuning policy, benchmark harness and ACS detection pseudocode, the worked virtual-bug example, the compilation pipeline, logging and observability specifications, coding-LLM guidance, the pseudo-MM2 skeleton, the Rust kernel skeleton, the mathematical summary, and the recommended initial configuration files.

Most readers should focus on the body. The appendices are intended primarily for implementers and coding LLMs who need concrete schemas and pseudocode, and can be skipped on a first read.

The intended audience is Hyperon-team insiders. Some readers know MORK/MM2 operational semantics, prefix-sensitive PathMap indexing, and MOSES-on-MORK ideas in detail, but may not have followed the ACTPC-CHEM and autocatalytic-set thread. Others have followed the ACTPC-CHEM and origin-of-life autocatalysis thread but have not internalized exactly how MM2 `exec` atoms are scheduled or how MORK’s indexes punish unselective patterns. Still others know one or both of those but have never used ECAN in earnest. Sections 3, 4, and 5 are deliberately written as orientation bridges so that readers in any of these groups can read the rest of the document on equal footing. Readers who already know all three threads can safely skip them.

A reader pressed for time can read in this order: the executive summary (§2), whichever orientation bridges are needed (§3–§5), the high-level architecture (§7), the population-sizing summary (§2.4), and the benchmark philosophy (§19.1). The appendices can then be consulted as reference when implementation details are needed.

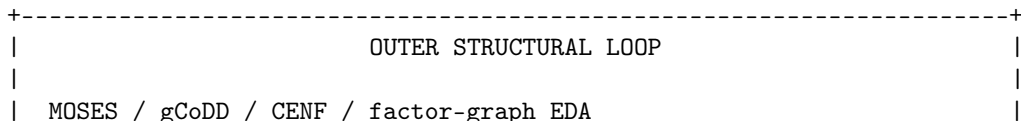
2 Executive summary

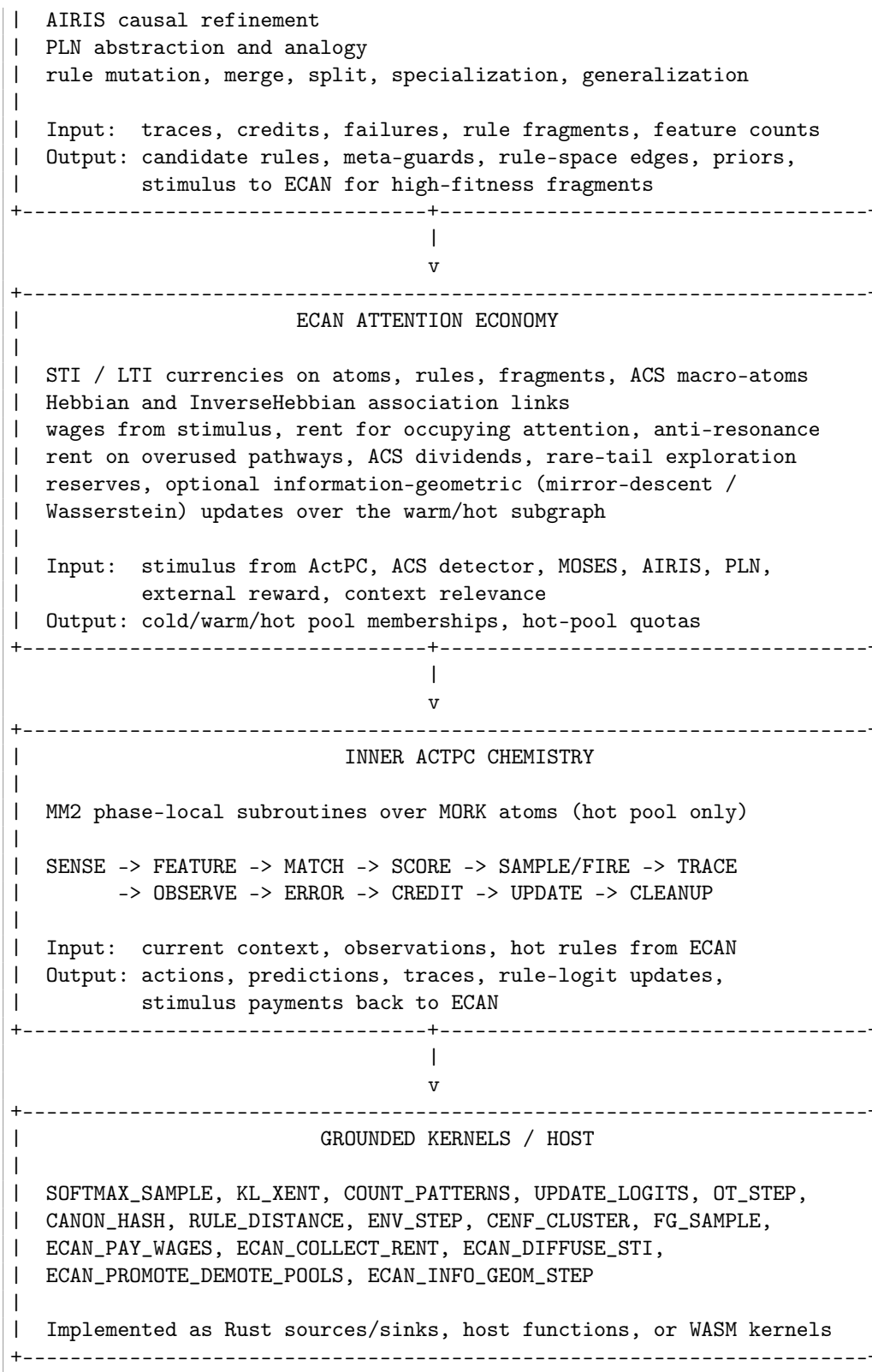
2.1 What we are building

We are building a practical ACTPC-CHEM: a stochastic soup of metagraph rewrite rules whose probabilities and structures are updated by prediction error, reward, causal surprise, and symbolic abstraction. The data, the models, and the rewrite rules all live as metagraph structures in MORK. Rule weights move quickly under a discrete active predictive-coding update. Rule *structures* move more slowly under MOSES/gCoDD/CENF-style structural evolution, with AIRIS and PLN contributing causal and abstract proposals. The eventual goal is for the rule population to behave as an active, autocatalytic, generative ecology rather than as a static collection of policy rules.

2.2 Architecture in three layers

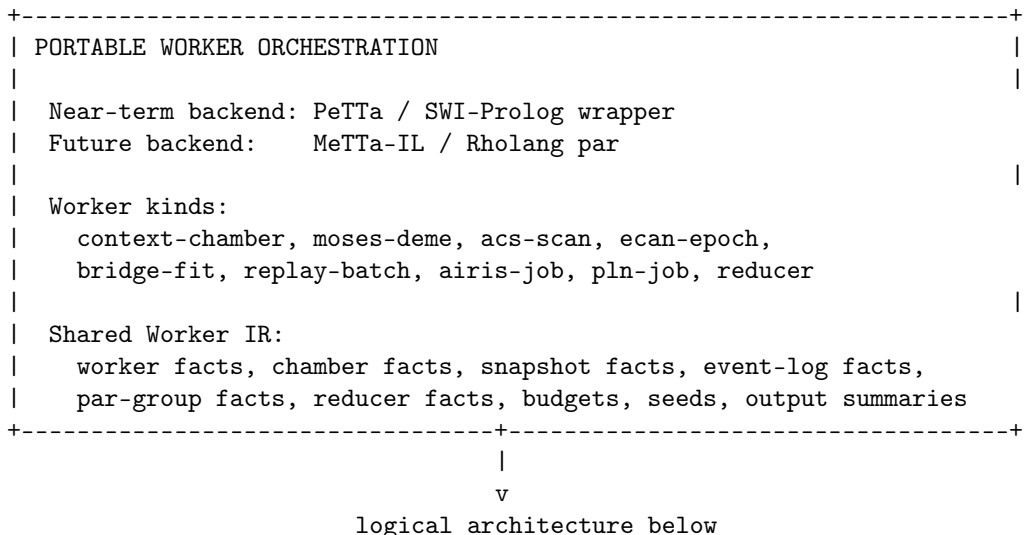
The recommended architecture is organized as three interacting layers: an outer structural-evolution loop, an ECAN-based attention economy, and an inner ACTPC-CHEM chemistry, all sitting on top of a grounded-kernel host.





Portable worker orchestration. The logical stack above is embedded in a backend-neutral worker-orchestration layer. The recommended near-term backend is PETTA, using Prolog predi-

cates, thread pools, message queues, or worker processes to run coarse-grained jobs. The intended future backend is METTA-IL with Rholang-style `par`. Both backends consume the same Worker IR and emit the same event-log and reducer schemas.



The orchestration layer does not perform arbitrary concurrent mutation inside one hot pool. A context-chamber worker receives a context snapshot, a context-local hot pool, a seed, and a bounded budget; it then runs one single-threaded MM2 phase loop and emits an ordered event log. Reducers merge event logs, stimulus deltas, candidate-rule proposals, ACS reports, bridge potentials, and attention updates. This gives immediate parallelism without requiring the MM2 hot loop itself to be multithreaded.

The inner loop is conservative and fast: it matches only the current context, only the active signatures, and only the rules that ECAN has promoted to the hot pool for this context. It does not scan the whole rule soup. The outer loop is more creative and expensive, but runs less often and uses accumulated traces to propose new rule molecules. The ECAN layer mediates between them: it gathers stimulus from every other component, decides which atoms (rules, fragments, traces, meta-guards, ACS macro-atoms) deserve scarce attention and memory at any moment, and migrates atoms among the cold, warm, and hot pools accordingly.

The most important separation is between two distributions:

$$\begin{aligned}
 p_t(R) &= \text{the ACTPC-CHEM probability that rule } R \text{ is useful as model/action,} \\
 a_t(A) &= \text{the ECAN attention mass assigned to atom } A.
 \end{aligned}$$

These distributions are coupled via stimulus and selection logits, but they are not identified. A rule may have high p_t but low STI if it is not context-relevant; a rule may have low p_t but high STI if it is novel, diagnostic, relevant to a rare-tail near-ACS motif, or needed for exploration. Keeping these distributions separate is what prevents premature rigidification of the chemistry.

2.3 Core design claims

1. Represent each rule as a bundle of small indexed atoms, not as one giant S-expression.
2. Store both a human-readable rule source and a compiled hot-path representation. Match against the compiled representation only.

3. Treat MM2 `exec` atoms as phase-local agenda items or transactions, not as persistent functions.
4. Put stochastic selection, numeric learning, KL/cross-entropy, Wasserstein steps, hashing, and environment stepping in grounded kernels.
5. Use MOSES, GCoDD, CENF, and friends as structural proposal machinery, not as the online action selector.
6. Use AIRIS and PLN as rule-proposal services. They add molecules to the soup; ACTPC-CHEM decides which molecules survive.
7. Use ECAN as the attention economy that decides which atoms are cold, warm, or hot at any moment. ECAN maintains STI/LTI currencies and Hebbian/InverseHebbian links; every other component pays stimulus to atoms it finds useful. The hot pool is what MM2 sees.
8. Maintain two coupled but distinct distributions: ACTPC-CHEM’s probability $p_t(R)$ over rules, and ECAN’s attention $a_t(A)$ over atoms. Couple them via stimulus and a soft KL regularizer; do not merge them.
9. Implement ECAN-lite first (sparse STI/LTI, wages, rent, top- k Hebbian diffusion, pool migration). Add information-geometric ECAN (mirror descent, Wasserstein on the association graph) later, only over the bounded warm/hot subgraph.
10. Treat a single MM2 chamber as intentionally single-threaded. One context-local chamber executes one ordered phase loop over one hot pool. The ordered trace is part of the scientific object: it makes credit assignment, ACS/RAF detection, serial replay, and ablation analysis well-defined.
11. Express parallelism above MM2 as coarse-grained workers. The intended future expression is METTA-IL with Rholang-style `par`; the near-term prototype can use PETTA/SWI-Prolog as the worker scheduler. Both backends consume the same Worker IR.
12. Keep the Worker IR backend-neutral. Worker facts, chamber facts, snapshot facts, event-log facts, `par`-group facts, reducer facts, budgets, seeds, and output summaries should be MORK-level data, not PETTA-private or METTA-IL-private structures.
13. Synchronize workers by snapshots, append-only event logs, reducers, and explicit proposal records. Do not allow many parallel branches, whether PETTA threads or METTA-IL `par` branches, to mutate one shared hot pool concurrently.
14. Let ECAN allocate resources at two levels: atom-level attention (`pool-hot` membership inside a chamber) and worker-level attention (which context chambers, demes, scans, bridge fits, and replay batches receive runtime budget).

2.4 Hardware and sizing in one glance

The recommended first serious rented machine is a CPU- and RAM-focused bare-metal server, not a GPU box:

Preferred:

```
96-core AMD EPYC-class bare-metal server
768 GB ECC DDR5 RAM
2 x 4 TB local NVMe
```

10 Gbps network
no GPU required for the symbolic chemistry itself

Acceptable starter:

64-core CPU
512 GB ECC RAM
2-4 TB local NVMe

Aggressive later:

128-192 cores
1.0-1.5 TB RAM
4-8 TB NVMe
optional GPU for neural/scoring/simulation side tasks

The guiding population-size heuristic is:

Millions cold, tens of thousands warm, hundreds-to-thousands hot.

The persistent rule soup may eventually contain hundreds of thousands or millions of rule records. Only a much smaller *active* (warm) subset should be eligible for retrieval at a given time, and only a small context-local *hot* subset should participate in any one MM2 cognitive tick. ECAN is what migrates atoms among these three pools: cold reservoir, warm active soup, and hot MM2 reaction chamber. A serious default starting configuration is:

<code>persistent_rule_target</code>	=	150_000	-	500_000	
<code>warm_rule_target</code>	=	15_000	-	50_000	
<code>hot_rules_per_context</code>	=	1_024	-	2_048	
<code>candidate_cap_per_tick</code>	=	2_048	-	4_096	
<code>fire_cap_per_tick</code>	=	16	-	32	
<code>MOSES_demes</code>	=	32	-	64	
<code>population_per_deme</code>	=	256	-	512	
<code>recent_trace_cap</code>	=	500_000	-	2_000_000	
<code>ECAN_max_hebb_edges_per_atom</code>	=	16	-	64	
<code>ECAN_info_geom_subgraph_cap</code>	=	512	-	4_096	(when enabled)

GPUs are useful for optional perception, learned rule scoring, embedding retrieval, batch environment rollouts, and neural predictive-coding modules, but are not the central scaling bottleneck for the symbolic chemistry itself. The symbolic substrate is dominated by indexed lookup, candidate generation, local graph rewrites, trace management, and working-memory churn. These are CPU- and RAM-bound rather than dense-linear-algebra-bound.

2.5 What success should look like

A simple “does it solve the task?” benchmark is not appropriate for this system. A fixed circuit, a memorized solution, or a narrowly specialized policy can solve a narrow benchmark while lacking the kind of self-maintaining generativity that algorithmic chemistry is supposed to bring. The proposed benchmark suite therefore measures the chemistry along multiple Pareto axes:

A = active autocatalysis,
 U = external utility,
 C = autocatalysis–utility coupling,
 G = generativity / novelty / diversity,
 H = heredity / serial-transfer stability,
 R = resilience / anti-rigidity,
 K = cost / complexity.

The success criterion the suite is designed to certify is not “the soup solves the task,” but rather:

The soup evolves active autocatalytic subnetworks whose ablation damages useful generative behavior, whose serial transfer preserves function, whose rare fringe contributes new useful structures, and whose overused habits are released before they congeal into sterile repetition.

3 Orientation bridge: ACTPC-CHEM and autocatalytic sets, for MORK insiders

This section is for readers who are fluent in MORK/MM2 but have not followed the ACTPC-CHEM and autocatalytic-set threads. Readers who have can skip it.

3.1 What is ACTPC-CHEM?

ACTPC-CHEM treats data, models, and rewrite rules as metagraph structures. The rule population is itself a stochastic generative model. A set of rules R_t applied to an input state G_t^{in} induces a predicted distribution $p_t(m)$ over patterns or outcomes. The environment yields an observed distribution $q_t(m)$. Learning is driven by a discrepancy such as

$$e_t = D_{KL}(q_t \parallel p_t) = \sum_m q_t(m) (\log q_t(m) - \log p_t(m)). \quad (1)$$

Rule weights, and more slowly rule *structures*, are updated to reduce prediction error, increase goal-directed reward, and encourage useful exploration. In a more ambitious version, the distribution over rules is updated using a discrete optimal-transport or Wasserstein natural-gradient step on a rule-space graph, so that probability mass moves along structurally or behaviorally similar rules rather than independently per rule.

The mental picture is a chemistry rather than a neural network. Rewrite rules are molecules. Their application is a reaction. The reaction propensity is a function of rule weights, context, novelty, cost, and goals, just as a chemical reaction rate is a function of concentrations, temperature, and catalysts. The role of an optimizer in this picture is closer to thermodynamic and selection pressure than to gradient descent on parameters of a fixed model.

3.2 Why “chemistry” and not just “learner”?

The point of insisting on the chemistry framing is that we want emergent, self-maintaining, generative dynamics, not merely a high-scoring policy. We want subgraphs of rules that produce, transform, and consume each other’s outputs. We want rules to behave as catalysts for the production or applicability of other rules. We want long-tailed populations in which a few common motifs dominate but a productive rare fringe keeps creating useful novelty.

3.3 Active autocatalytic sets and RAFs

The terminology comes from origin-of-life work on autocatalytic sets and RAFs (reflexively autocatalytic and food-generated sets). The mapping for algorithmic chemistry is:

Chemical term	Algorithmic-chemistry analogue
Species	Rule IDs, rule fragments, templates, meta-guards, intermediate concepts, generated artifacts, proof lemmas, program fragments, task-state motifs.
Food set	Primitive constructors, base observations, task inputs, basic operators, type symbols, initial seed rules, reward / prediction-error atoms.
Reaction	One actual rule application, structural mutation, abstraction, repair, lemma derivation, or program-fragment generation event.
Catalysis	Species X catalyzes reaction r if the presence or activity of X increases the propensity, matchability, score, sampling probability, or success rate of r without being consumed as the main input.
Active reaction graph	The graph of reactions and catalysis built from actual logged events over a sliding window, not merely from theoretical reachability.
ACS / RAF	A food-generated, reflexively autocatalytic, active and persistent subnetwork of rules and artifacts.

The operational emphasis is on *actual active graphs*. It is not enough that a rule could in principle catalyze another rule; the reaction and catalysis edges must be observed in the running soup over sliding windows. This is what we instrument the system to detect.

3.4 Useful active autocatalytic sets

A candidate subnetwork S is a *useful active autocatalytic set* if:

- (1) **Food-generated.** All required reactants of S are reachable from the food set F using reactions in S .
- (2) **Reflexively autocatalytic.** Every important reaction in S is catalyzed by some species produced by, retained by, or reachable inside S .
- (3) **Active.** The reactions in S fire repeatedly above an activity threshold over the observation window.
- (4) **Persistent.** S persists across multiple windows, task samples, or transfer cycles.
- (5) **Useful.** S causally contributes to external work, not merely to self-maintenance.
- (6) **Generative.** S produces a family of valid useful variants, not only one rigid output.

The utility-coupling clause (5) is the most important and most easily forgotten. Without it, a run can cheat by evolving a useless autocatalytic loop alongside a separate brittle task-solving circuit and scoring well on a naive autocatalysis-plus-utility scalar.

3.5 Why this is hard, and where ACTPC-CHEM fits

Pure ACTPC-CHEM provides the local mechanism: rules with weights, prediction error, credit, and structural evolution. By itself it does not guarantee that the rules organize into autocatalytic ecologies rather than into isolated task-specific circuits. The benchmark suite in §19 is designed to detect, reward, and stress-test the autocatalytic structure, and to reveal when the system has degenerated into rigid circuits even when its task scores look fine.

4 Orientation bridge: MORK and MM2, for chemistry/autocatalysis insiders

This section is for readers who are fluent in ACTPC-CHEM, RAF theory, and the autocatalytic-generativity benchmarks but who have not internalized the operational details of MORK/MM2. Readers who have can skip it.

4.1 What MORK is

MORK is the indexed, persistent atom store on which the rest of the system runs. For our purposes it has three properties that matter most:

- **Atoms are small typed expressions.** Facts, rules, traces, candidates, and metadata all live as atoms. “Atom” here means an atomic expression in the metagraph sense, not an atom in the chemical sense; a “molecule” in the chemistry framing of ACTPC-CHEM is therefore typically a *bundle of MORK atoms*, not a single MORK atom.
- **Indexes are prefix-sensitive.** The PathMap index walks atoms from the front. The first symbol, then the next, then the next, drive lookup. A relation name and grounded keys at the front of an atom give fast, selective access. Variables or unknown values at the front of an atom give slow scans.
- **Sources and sinks bridge to host code.** MORK can call external host functions (Rust kernels, WASM modules, custom sinks) for operations that do not belong in pure graph rewriting: stochastic sampling, numeric KL or cross-entropy, hashing, environment stepping, bulk garbage collection, and so on.

The implication for our design is that *schemas matter*. Atoms must be designed so that their first arguments are relations and grounded keys. Hot- path queries should be selective in their first few symbols. Whole-soup scans should be impossible by construction.

4.2 What MM2 is

MM2 is the operational rewrite layer that runs on top of MORK. It is a restricted dialect of METTA oriented toward fast local rewrites and phase-local execution. Its central object is the **exec** atom:

`(exec LOCATION_OF_COMPUTATION PATTERNS TEMPLATES)`

Operationally:

1. An **exec** atom is selected.
2. Its conjunctive pattern is matched against MORK.

3. Its template is written.
4. The **exec** is consumed.

This is important. MM2 **exec** atoms are agenda items or transactions, not persistent functions. We will exploit this by treating **execs** as phase-scoped, context-scoped, and tick-scoped: each “cognitive tick” enqueues a small batch of **execs** that drive the inner loop forward by one step, and they consume themselves as they fire.

The MM2 dialect intentionally encourages small expressions, short symbols, and limited variable mentions. The order of conjunctive patterns inside an **exec** matters at runtime: the most selective ground-prefixed atoms should appear first.

Single-threaded chamber semantics. For this architecture, MM2 should be treated as a single-threaded local reaction interpreter. A chamber receives a context snapshot, a context-local hot pool, a seed, and a runtime budget; it then executes the tick phases in a deterministic order and emits an ordered event log.

This does not make the whole system single-threaded. Parallelism is lifted one level up. In the near-term implementation, PETTA/SWI-Prolog can dispatch many context chambers, MOSES demes, ACS scans, ECAN epochs, replay batches, and bridge-fitting jobs. In the future implementation, the same worker graph should be expressed in METTA-IL using Rholang-style **par**. The important invariant is that no backend should race multiple rewrites against one shared mutable hot pool.

4.3 What MOSES-on-MORK contributes

MOSES is the structural-evolution layer. The MOSES-on-MORK project contributes a few specific components that we will rely on in the outer loop:

- gCoDD: a canonical DAG representation of programs and rule bodies, with grounded predicate calls as atomic leaves. Used for hashing, deduplication, and structural mutation.
- ENF and CENF: normal forms that deduplicate equivalent programs and optionally cluster correlated guards into meta-guards.
- Factor-graph EDA: a generative model over knobs, guards, meta-guards, and rule fragments, used to sample promising new programs.
- Quantale-inspired variation: crossover and mutation represented as local algebraic operations over program fragments.

The inner chemistry supplies traces, credits, and high-value rule fragments to MOSES; MOSES proposes better rules, abstractions, and meta-guards; those are compiled back into the MM2 hot path as new bundles of small indexed atoms.

4.4 What AIRIS and PLN contribute

AIRIS contributes causal-rule proposals after surprising transitions: if something unexpected just happened, AIRIS abduces what set of conditions might have caused it and emits one or more candidate causal rules.

PLN contributes probabilistic logical abstraction and analogy: given a collection of positive and negative examples plus background knowledge, it proposes inductive generalizations, analogical mappings, and abductive explanations, with confidence-valued truth.

In our framing, both AIRIS and PLN are rule-*proposal* services. They enter rules into the same weighted soup as everything else, with `trial` status and a moderate prior. Whether those proposals survive is decided by ACTPC-CHEM’s prediction-error, reward, and credit dynamics, not by AIRIS or PLN directly.

5 Orientation bridge: ECAN attention economics

This section is for readers who are fluent in ACTPC-CHEM, MORK/MM2, or both, but have not used ECAN (Economic Attention Networks) before. Readers who already know ECAN can skip it.

5.1 What ECAN solves

A large cognitive system has far more possible internal operations than it can execute. The persistent metagraph may contain millions of atoms: rules, rule fragments, traces, meta-guards, concepts, contexts, and ACS motifs. The system cannot match, score, or update all of them every tick. It needs a principled mechanism that answers, over and over again:

- Which atoms should be in RAM at all?
- Which atoms should occupy the small hot MM2 reaction chamber for this context, this tick?
- When one atom becomes useful, which other atoms should be pulled into focus with it?
- Which atoms have lost their value and should be cooled, archived, or forgotten?

These are not cache-management questions; they are cognitive resource-allocation questions. ECAN is designed for them.

5.2 STI, LTI, and the Attentional Focus

ECAN assigns each atom A two attention values:

$$\text{STI}(A) = \text{Short Term Importance}, \quad \text{LTI}(A) = \text{Long Term Importance}.$$

STI measures immediate urgency: high STI means the atom should be eligible for selection by a scheduler, included in the current Attentional Focus, and indexed into a hot working set. LTI measures retention value: high LTI means the atom should be kept in memory even when it is not currently useful; low LTI means the atom is a candidate for forgetting, compression, or demotion to a cold archive.

The two are separate currencies. An atom may have high LTI and low STI (important but not currently focal) or high STI and low LTI (currently salient but not yet proven durable). The Attentional Focus is the set of atoms with the highest STI; in our setting, the context-local hot pool is an operational version of the AF restricted to a given context.

5.3 The economic metaphor

The economic language is not cosmetic. STI and LTI are treated as conserved or homeostatically bounded currencies. Atoms earn *wages* from *stimulus* paid by cognitive processes that found them useful; they pay *rent* for occupying attention; they spread funds to associated atoms via diffusion; they may be *taxed* if total attention drifts outside its budget; they may receive *welfare* if they are temporarily poor but structurally important; and they may be *forgotten* if their LTI falls below a retention floor. In our setting, ACTPC-CHEM, AIRIS, PLN, MOSES, the ACS detector, and the external reward channel all pay stimulus to atoms they find useful, and ECAN aggregates these payments into the STI/LTI dynamics.

5.4 Hebbian and InverseHebbian links

ECAN maintains a sparse association graph over atoms. A Hebbian link $H(A, B)$ encodes the tendency that if A is in focus, B should also be in focus:

$$H(A, B) \approx P(B \in \text{AF} \mid A \in \text{AF}).$$

An InverseHebbian link $I(A, B)$ encodes the opposite: if A is in focus, B should not be. Hebbian links support associative retrieval and attractor formation; InverseHebbian links support competition, inhibition, mutually exclusive contexts, and anti-resonance-like release from overused pathways. Weights are updated from coactivation observed during recent windows. The graph should be kept sparse: a typical cap is on the order of 16–64 Hebbian and 8–32 InverseHebbian edges per atom.

5.5 Diffusion and attractor behavior

When STI is spread along Hebbian links and inhibition along InverseHebbian links, ECAN behaves like a Hopfield-style attractor network with conserved currencies. A partial cue activates a few members of a previously useful cluster; spreading pulls the rest of the cluster into focus; coactivation strengthens the cluster’s associative links; and after enough such cycles a macro-atom can be created to represent the whole cluster as a single symbolic object. This last step, *map formation*, is exactly how ACS macro-atoms appear in our design: an attractor in attention space becomes an explicit manipulable handle.

5.6 Information-geometric ECAN

A natural further step views the STI vector, suitably normalized, as a probability distribution over atoms:

$$a_t(A) = \frac{\text{STI}(A)}{\sum_B \text{STI}(B)}.$$

ECAN’s job is then to move this distribution toward a desired distribution defined by a utility field u_t supplied by ACTPC-CHEM, the ACS detector, and other modules. In information-geometric ECAN, this movement is performed along Fisher–Rao or related natural-gradient directions rather than via heuristic Hebbian diffusion alone.

A simple and useful first step is KL-regularized mirror descent:

$$a_{t+1} = \arg \max_{a \in \Delta} \left[\langle u_t, a \rangle - \lambda \text{KL}(a \parallel a_t) \right],$$

which has the closed-form update

$$a_{t+1}(A) \propto a_t(A) \exp(u_t(A)/\lambda).$$

A more advanced step adds Wasserstein transport cost over the Hebbian association graph, so attention prefers to flow along cheap association paths rather than teleport across the simplex (§16.10).

The original ECAN information-geometric work (Ikle and Goertzel, AGI 2011) applied adaptive natural-gradient learning to small ECAN-like associative-memory experiments and reported improved focusing, while leaving scalable implementation as an open problem. The lesson for our setting is to run sparse classical ECAN continuously and to apply information-geometric ECAN only periodically over a bounded warm/hot subgraph.

5.7 Why ECAN fits algorithmic chemistry

ECAN fits the chemistry framing well. Algorithmic chemistry already treats rules as molecules and rule applications as reactions, and useful emergent organization shows up as autocatalytic subnetworks. ECAN adds:

- *a circulatory system* — STI moves attentional resources to where reactions are productive;
- *a memory policy* — LTI decides which molecules persist over long time scales;
- *associative retrieval* — Hebbian links pull whole ACS-like clusters into focus from a small cue;
- *an anti-rigidity mechanism* — overuse rent on hot pathways releases the system from sterile habits, and InverseHebbian competition prevents single-cluster monopolies.

Slogan: ACTPC-CHEM decides which rules predict, explain, and act well. ECAN decides which atoms get enough attention and memory to matter computationally.

5.8 What ECAN is not

ECAN is not a replacement for logical inference, program learning, or ACTPC-CHEM rule-probability learning. It is not a universal optimizer, a dense neural attention layer, or a mechanism that should scan the entire MORK every tick. It is the resource-allocation and associative-memory substrate that lets the rest of the system focus on the right parts of the metagraph at the right times.

6 Glossary

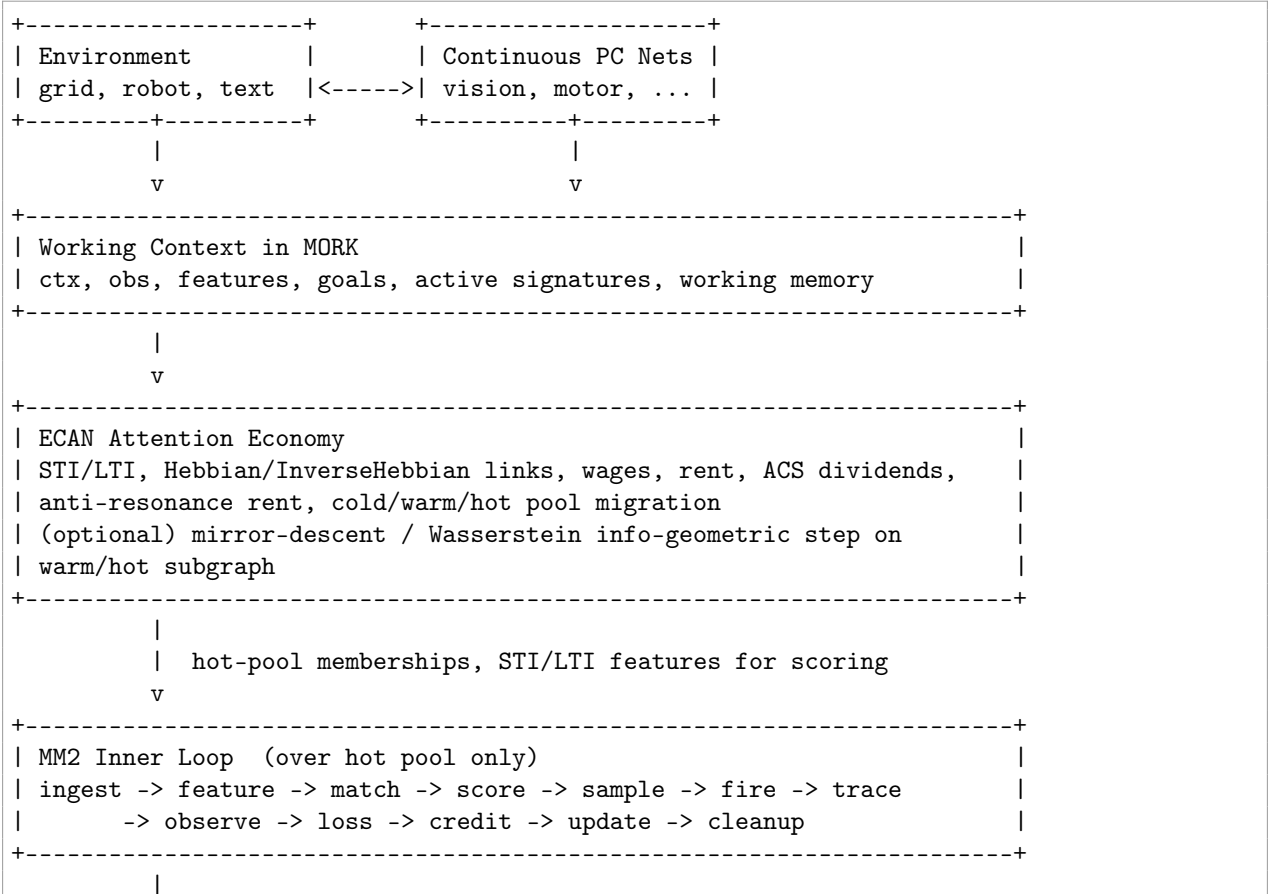
Term	Meaning
Chemistry	The evolving population of rewrite rules, traces, candidates, rewards, and abstractions.
Molecule	A rule, rule fragment, meta-rule, meta-guard, or reusable subprogram. Typically a bundle of multiple small MORK atoms.
Reaction	A rule application or a rule-modifying rule application.
Catalysis	Species X catalyzes reaction r if X ’s presence or activity increases the propensity, matchability, score, sampling probability, or success rate of r without being consumed as a main input.

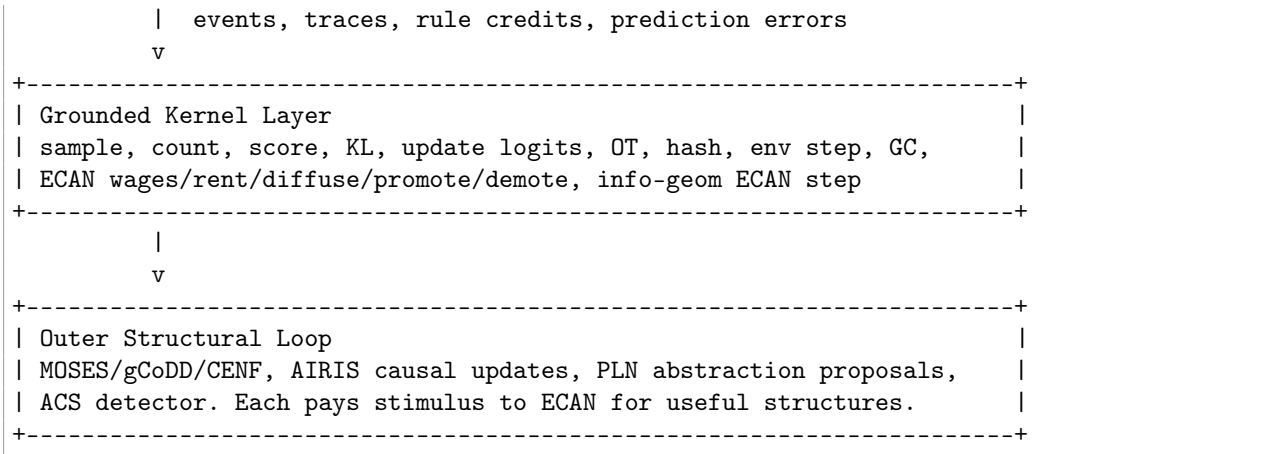
Term	Meaning
ACS / RAF	Active, food-generated, reflexively autocatalytic subnetwork of rules and artifacts, observed in actual logged events.
Context	The current local working state for one tick, episode, agent, or problem instance.
Tick	One pass through the inner loop for one context.
Signature	A compact trigger key describing the kind of observation or pattern a rule can match.
Compiled rule	Hot-path atoms used by MM2 matching, derived from a richer source rule.
Rule source	Human-readable METTA or GCoDD representation used for inspection and evolution.
Trace	A record that rule R was applied in context C with bindings B and later outcome O .
Credit	A scalar or vector assigned to a rule or trace from reward, prediction error, and novelty.
Meta-guard	A learned conjunction or cluster of low-level guards represented as a single guard.
Deme	A MOSES subpopulation exploring a region of rule/program space.
Cold soup	The full persistent rule reservoir. May be very large.
Warm soup	The active subset eligible for retrieval and use.
Hot set	The context-local bounded subset admitted into one MM2 tick.
Worker IR	Backend-neutral MORK-level representation of coarse-grained executable workers, including worker kind, backend, budget, snapshot, seed, output log, and reducer information. PETTA executes it now; METTA-IL should execute it later.
PETTA wrapper	Near-term orchestration backend implemented using PETTA/SWI-Prolog. It reads Worker IR facts, dispatches workers using Prolog predicates, thread pools, or worker processes, and writes the same event logs and deltas expected by the future METTA-IL backend.
METTA-IL backend	Future orchestration backend in which the same Worker IR is executed using METTA-IL and Rholang-style par .
Context chamber	A worker that owns or is assigned one context snapshot, one context-local hot pool, one MM2 phase loop, one seed, and one runtime budget. The chamber executes MM2 single-threadedly and emits an ordered event log.
Par group	A set of workers scheduled as one logical concurrent epoch. Under PETTA this is realized as a Prolog worker batch, thread-pool batch, or worker-process batch; under METTA-IL it is realized as a par expression.
Reducer	A worker or grounded service that merges outputs from parallel workers: event logs, stimulus deltas, rule proposals, ACS reports, bridge potentials, or attention updates.
Worker-level ECAN	Extension of ECAN in which workers, worker templates, and worker families also receive STI/LTI, rent, wages, and runtime budgets. This lets ECAN allocate processor time across chambers and services, not only across atoms inside one chamber.

Term	Meaning
ECAN	Economic Attention Networks: an attention economy over atoms that maintains STI/LTI currencies and Hebbian/InverseHebbian links and mediates pool migration.
STI	Short Term Importance: an atom's immediate-attention currency. Drives hot-pool participation and scheduler eligibility.
LTI	Long Term Importance: an atom's retention currency. Drives forgetting, archival, and persistence in the cold pool.
AF	ECAN Attentional Focus: the highest-STI atoms. The hot pool is an operational, context-local version of the AF.
Hebbian link / InverseHebbian link	Sparse association edges encoding that two atoms tend, or tend not, to be coactive in the AF.
Stimulus / wage / rent	The economic primitives by which atoms gain attention from useful events, and pay it back for occupying focus.
Overuse rent / anti-resonance rent	A penalty term on atoms with high recent-use traces, used to break sticky high-STI habits.
ACS macro-atom	A reified handle for a recurring useful active ACS, treated as an atom in its own right and itself subject to STI/LTI.

7 High-level architecture

7.1 Subsystems





The dataflow has two important loops besides the inner MM2 cycle. The first is between ECAN and MM2: ECAN sets which atoms are hot for this context, MM2 fires reactions among them, and the events fired (with their predictions, errors, rewards, and credits) become stimulus that ECAN uses to update STI/LTI and association links for the next tick. The second is between the outer loop and ECAN: MOSES, AIRIS, PLN, and the ACS detector all pay stimulus to atoms they find useful, and ECAN decides whether the corresponding structures are worth promoting, retaining, or eventually forgetting.

7.2 Separation of responsibilities

Component	Responsibility
MORK	Persistent indexed storage of atoms, traces, rules, contexts, attention facts, association links, pool memberships, and models.
MM2	Fast local pattern-triggered rewrites and phase scheduling, over the hot pool only.
METTA	Human-readable semantics, high-level rule language, integration layer.
PETTA wrapper	Near-term worker-orchestration backend. Reads Worker IR facts from MORK or a MORK-backed view, dispatches coarse-grained workers using Prolog predicates, thread pools, message queues, or worker processes, calls MM2 chambers and grounded kernels, and writes append-only event logs and reducer inputs. It is not part of the hot MM2 loop.
METTA-IL / Rholang	Future worker-orchestration backend. Consumes the same Worker IR and expresses concurrent worker batches using Rholang-style par . It should replace the PETTA wrapper without changing chamber semantics, event logs, reducers, or ECAN worker-level accounting.
Reducers	Synchronization boundary for worker outputs: event logs, stimulus deltas, candidate-rule proposals, ACS reports, bridge potentials, and attention updates. Workers should not race on one shared hot pool.
Grounded kernels	Numeric and stochastic operations not suited to pure graph rewriting, including ECAN wages, rent, diffusion, and pool migration.
ACTPC inner loop	Online rule selection, prediction, trace recording, credit, weight update, and stimulus payment to ECAN.

Component	Responsibility
ECAN	Attention economy over atoms: STI/LTI dynamics, Hebbian/InverseHebbian associations, cold/warm/hot pool management, anti-resonance regulation, optional info-geometric updates.
ACS detector	Detection of active useful autocatalytic subnetworks; pays ECAN dividends to ACS members, catalysts, and rare-tail precursors.
MOSES outer loop	Structural invention, rule recombination, abstraction, canonicalization. Pays ECAN stimulus to high-fitness fragments.
AIRIS	Causal rule proposal and refinement after surprising transitions. Pays ECAN stimulus to causal rules that explain discrepancies.
PLN	Probabilistic abstraction, analogy, and logical hypothesis generation. Pays ECAN stimulus to abstractions that improve heldout prediction.

7.3 Portable worker orchestration: PETTA now, METTA-IL later

The system should be implemented as a graph of coarse-grained workers. The stable abstraction is not backend syntax but the worker contract. A worker consumes a snapshot or work item, performs a bounded computation, and emits an event log, delta, proposal, or summary.

Near-term, this worker graph can be implemented in PETTA/SWI-Prolog:

```
run_epoch(Epoch) :-
    build_worker_batch(Epoch, Workers),
    schedule_workers_petta(Workers, ParGroup),
    run_par_group_petta(ParGroup),
    reduce_par_group_outputs(ParGroup).
```

Future METTA-IL should implement the same logical pattern:

```
(par
  (run-worker W_ctx_001)
  (run-worker W_ctx_002)
  (run-worker W_deme_001)
  (run-worker W_acs_001)
  (run-worker W_ecan_001))

(reduce-par-group PG17)
```

The PETTA form and the METTA-IL form should be generated from the same Worker IR facts:

```
(worker W17)
(worker-kind W17 context-chamber)
(worker-backend W17 petta)          ; petta now, metta-il later
(worker-context W17 C42)
(worker-snapshot W17 SNAP17)
(worker-hot-pool W17 HP42)
(worker-output-log W17 LOG99)
(worker-budget W17 cpu-ms 50)
(worker-seed W17 12345)
```

The scheduler backend is replaceable because semantics are attached to the Worker IR, not to PETTA predicate names or METTA-IL surface syntax. The most important slogan is therefore: MM2 supplies ordered local chemistry; PETTA supplies the first parallel wrapper; METTA-IL supplies the eventual native parallel wrapper; the Worker IR is the portability boundary.

8 Data model

8.1 General atom design rules

All hot-path atoms should follow these rules:

1. The first symbol is a relation name, such as `obs`, `rule`, `trigger`, or `cand`.
2. The second and third positions should usually be grounded keys, such as context id, signature id, rule id, tick id, or phase id.
3. Variables should appear toward the suffix of an expression.
4. Avoid deep nested expressions on hot paths. Use multiple shallow facts.
5. Keep source rules and compiled rules separate.
6. Store enough metadata to garbage collect transient atoms deterministically.

The chemistry framing makes it tempting to write a single big atom per “molecule.” Resist this. The hot path needs many small atoms whose front prefixes are selective; one giant nested atom defeats prefix indexing.

8.2 Identifier conventions

C<n>	context id
T<n>	tick id
R<n>	rule id
A<n>	action id
E<n>	trace/event id
O<n>	object id
MG<n>	meta-guard id
G<n>	guard id
N<n>	gCoDD node id
D<n>	MOSES deme id
S<n>	signature id
P<n>	pattern id

IDs should be short symbols or interned hashes. A host-side allocator should map long human names to short stable symbols.

The complete persistent fact schemas — including context and tick facts, observation and feature facts, rule metadata and compiled rule facts, candidate, chosen, action, prediction, outcome, trace, credit, rule-space graph, meta-guard, gCoDD, and MOSES facts, plus the ECAN-specific schemas — are in the appendices.

9 Rule representation

9.1 Three-level representation

Every nontrivial rule should exist in three representations:

1. **Source rule:** readable METTA or METTA-like expression.

2. **Canonical program:** gCoDD or another canonical DAG used by MOSES, hashing, deduplication, and structural evolution.
3. **Compiled hot-path atoms:** small indexed facts used by MM2.

```

Human-readable source
    |
    | compile_rule_source_to_gcodd
    v
Canonical gCoDD / CENF representation
    |
    | compile_gcodd_to_mm2_atoms
    v
Compiled trigger/guard/template facts in MORK

```

9.2 Why not match directly on source rules?

Source rules can be large, nested, and variable-rich. That is useful for humans and structural evolution, but bad for MORK hot-path performance. The compiled representation makes the first few bytes of each query selective:

```

(obs C42 see-food ...)
(trigger see-food R17)
(rule-status R17 active)

```

This avoids the common failure mode of scanning every rule for every observation.

10 Inner loop: MM2 phase architecture

10.1 Phase overview

A single tick consists of these phases:

```

00 init-tick
10 sense
20 derive-features
30 activate-signatures
40 match-candidates
50 score-candidates
60 sample-or-select
70 fire-action-or-prediction
80 environment-step
90 observe-outcome
100 compute-error
110 assign-credit
120 update-rule-weights
130 structural-trigger
140 cleanup

```

Not every phase must be pure MM2. Some phases are MM2 rewrites; others are sink or source calls. The important point is that phases are explicit, local, and interruptible.

The phase loop above is the inside of one context chamber. It is deliberately single-threaded: a chamber should not use PETTA threads or METTA-IL `par` to run multiple MM2 phase rewrites

against the same mutable hot pool. Instead, PETTA or METTA-IL may run many chambers, demes, scans, replay batches, bridge fits, ECAN epochs, and reducers concurrently. In other words, MM2 supplies ordered local chemistry; the worker runtime supplies the parallel chamber ecology.

Concrete pseudo-MM2 code for each of the fifteen phases, including the `exec` atoms that drive phase transitions and the host-kernel request atoms that bridge to the grounded layer, is in the appendix volume.

11 Grounded kernel interfaces

11.1 Why grounded kernels are required

Pure graph rewriting is not the right substrate for all operations. The following belong in host-side kernels:

- stochastic sampling and random-number generation;
- numerical comparison, softmax, KL, cross-entropy, entropy;
- vector and sparse-matrix operations;
- Wasserstein natural-gradient or optimal-transport solves;
- canonical hashing and hash-consing;
- factor-graph inference and sampling;
- environment stepping and sensor ingestion;
- bulk garbage collection and compaction.

11.2 Kernel API table

Kernel	Input atoms	Output atoms
SENSE	sense-request	obs, raw-feature, done
SCORE_CANDIDATES	score-input, rule metadata	cand-score, cand-logit
SOFTMAX_SAMPLE	cand-score, temperature	chosen
FIRE_TEMPLATE	chosen, template facts	action, trace, trace-bind
ENV_STEP	action	outcome, reward, next observations
COUNT_PATTERNS	pred, outcome, traces	histograms or distribution atoms
KL_XENT	predicted and observed distributions	error, reward epistemic
ASSIGN_CREDIT	traces, rewards, errors	credit, credit-total
UPDATE_LOGITS	credits, logits, costs	updated rule-logit facts
CANON_HASH	rule source or GCODD graph	rule-src-hash, gcodd-hash
RULE_DISTANCE	rule pairs or fragments	rule-neighbor, rule-distance
OT_STEP	rule graph, logits, gradients	updated logits or transport plan
CENF_CLUSTER	guard counts, fitness	meta-guard facts
FG_SAMPLE	factor graph model	sampled knobs/rule fragments
ECAN_GATHER_STIMULUS	events, traces, credits, ACS report, external reward	per-atom stimulus totals

Kernel	Input atoms	Output atoms
ECAN_PAY_WAGES	stimulus, current STI/LTI	updated <i>sti</i> , <i>lti</i>
ECAN_COLLECT_RENT	STI, costs, overuse traces	updated <i>sti</i> , <i>lti</i>
ECAN_DIFFUSE_STI	STI, Hebbian/InverseHebbian links, frontier	updated <i>sti</i> along bounded frontier
ECAN_UPDATE_HEBBIAN	coactivity over a recent window	<i>hebb</i> , <i>inv-hebb</i> weight updates
ECAN_PROMOTE_DEMOTE	STI, LTI, context, ACS membership, quotas	<i>pool</i> , <i>pool-hot</i> migrations
ECAN_INFO_GEOM_STEP	STI distribution, utility, association graph, ground cost	updated <i>sti</i> (mirror-descent or Wasserstein)
ECAN_FORGET	LTI, protected-set membership	archived or deleted atoms

12 Inner loop pseudocode

12.1 Top-level tick

```

procedure ACTPC_TICK(space, ctx):
  tick <- allocate_tick(ctx)
  add(space, (tick ctx tick))

  SENSE(space, ctx, tick)
  DERIVE_FEATURES_MM2(space, ctx, tick)
  ACTIVATE_SIGNATURES_MM2(space, ctx, tick)

  # ECAN ensures active signatures have warm and hot support.
  ECAN_RETRIEVE_AND_PROMOTE(space, ctx, active_signatures(ctx))

  MATCH_CANDIDATES_MM2(space, ctx, tick)    # over hot pool only
  SCORE_CANDIDATES(space, ctx, tick)        # uses STI/LTI/ACSBonus
  chosen <- SOFTMAX_SAMPLE(space, ctx, tick)

  FIRE_CHOSEN(space, ctx, tick, chosen)
  ENV_STEP(space, ctx, tick)

  COMPUTE_PREDICTION_ERROR(space, ctx, tick)
  ASSIGN_CREDIT(space, ctx, tick)
  UPDATE_RULE_WEIGHTS(space, ctx, tick)

  # Aggregate stimulus and update the attention economy.
  stimulus <- ECAN_GATHER_STIMULUS(space, ctx, tick)
  ECAN_UPDATE(space, ctx, stimulus, recent_window(ctx))

  if SHOULD_RUN_STRUCTURAL_LOOP(space, ctx, tick):
    enqueue_structural_job(ctx, tick)

  if tick % info_geom_period == 0:
    ECAN_INFO_GEOM_STEP(space, ctx)          # mirror-descent / Wasserstein

  CLEANUP_TRANSIENTS(space, ctx, tick)
end procedure

```

13 Wasserstein natural-gradient module

13.1 Purpose

The simple logit update moves each rule independently. The Wasserstein module moves probability mass along a graph of similar rules. This gives smoother updates: a successful specialized rule can lend probability mass to nearby variants, and a failed rule can transfer mass to close alternatives rather than forcing global random exploration.

13.2 Rule-space graph

The graph has nodes R_i and edges between rules that are structurally or behaviorally close.

```
(rule-neighbor R17 R22 0.08)    ; small number means close
(rule-neighbor R17 R35 0.21)
```

Possible edge costs include source edit distance, gCoDD graph edit distance, guard-set Jaccard distance, template/action distance, behavioral distance on a replay buffer, and abstraction/family distance. A practical ground metric is a weighted mixture:

$$\omega_{ij} = w_s d_{source}(i, j) + w_g d_{gcodd}(i, j) + w_b d_{behavior}(i, j) + w_c d_{cost}(i, j). \quad (2)$$

13.3 When to apply OT

Do not apply OT globally every tick. Use it in restricted scopes:

- active candidate set for one context;
- one rule family;
- one MOSES deme;
- rules sharing a signature;
- top- k rules by recent credit or surprise.

The measure-dependent Laplacian, OT-step pseudocode, and host-request schema are in the appendices.

14 Outer loop: structural evolution with MOSES, gCoDD, and CENF

14.1 Outer-loop role

The outer loop proposes new molecules for the chemistry. It should not be responsible for every online decision. It receives accumulated evidence and returns candidate rules or abstractions.

```
Inner loop output:
  traces, credits, errors, high-value rules, failed contexts

Outer loop processing:
  extract fragments
  canonicalize as gCoDD
  mine guard correlations
  build/update factor graph
```

```

sample candidates
normalize with ENF/CENF
compile candidates to MM2 facts

```

Inner loop input:

```

new trial rules, meta-guards, priors, rule-neighbor edges

```

14.2 Structural job types

Job	Meaning
high-credit-rule	Clone, specialize, or abstract a successful rule.
high-error-context	Propose new predictors or causal rules for an unexplained outcome.
unstable-rule	Split a rule into more specific variants.
redundant-family	Merge or retire rules with similar behavior.
feature-correlation	Create a meta-guard from co-occurring guards.
deme-refresh	Rebuild a MOSES factor graph and sample new rules.

14.3 Rule mutation operators

```

SPECIALIZE_CONDITION(rule):
    add one guard that was present in successful traces

GENERALIZE_CONDITION(rule):
    remove one guard that appears unnecessary

SWAP_TEMPLATE(rule):
    replace action or predicted outcome with a nearby template

ABSTRACT_GUARDS(rule):
    replace several guards by a meta-guard

INLINE_METAGUARD(rule):
    replace a meta-guard by its member guards

SPLIT_RULE(rule):
    create variants for high-variance trace clusters

MERGE_RULES(rule_a, rule_b):
    create a generalized rule if behavior is compatible

ADD_DELAY_CONDITION(rule):
    add elapsed-time or event-window guard

ADD_CONTEXT_CONDITION(rule):
    add activity, temperature, goal, agent, or memory condition

```

Detailed pseudocode for the structural loop, CENF meta-guard clustering, factor-graph EDA, candidate-rule sampling, and trial-rule insertion is in the appendices.

15 AIRIS and PLN integration

15.1 Design principle

AIRIS and PLN should not bypass the chemistry. They propose rules, abstractions, and constraints. Those proposals enter the same weighted rule soup as other molecules and are tested by prediction error and reward.

AIRIS-style refinement is triggered by a surprising transition: when prediction error exceeds a threshold, AIRIS abduces what set of conditions might have caused the surprising outcome and emits one or more candidate causal rules. PLN-style inference proposes a generalized rule from observed cases, analogies, and background knowledge, with confidence-valued truth. In both cases the proposed rules enter the soup with `trial` status and a moderate prior; whether they survive is decided by the credit dynamics, not by the proposer. Both AIRIS and PLN also pay ECAN stimulus to proposals that subsequently prove useful, which couples the proposal machinery to the attention economy without giving it veto power.

Detailed proposal pseudocode for both is in the appendices.

16 Attention economy: ECAN integration

This section specifies the attention-economy layer that sits between the ACTPC-CHEM inner loop and the rest of the cognitive system. Its job is to maintain the cold/warm/hot pool partition of the rule soup, decide moment-by-moment which atoms participate in any given MM2 tick, and consolidate the credit signals coming from every other component into durable retention decisions.

16.1 The two distributions, and what they do

The system maintains two coupled but distinct probability-like distributions over symbolic content:

- $p_t(R)$ — ACTPC-CHEM’s belief that rule R is useful as a model or action under the current context. This is what the inner-loop logits represent and what the Wasserstein update modifies.
- $a_t(A)$ — ECAN’s attention mass on atom A . This is what pool-migration decisions, hot-pool admission, and forgetting are based on. STI is the unnormalized representative of a_t ; LTI is a slower, more durable companion.

These distributions are coupled, not identified. A rule can have high $p_t(R)$ but low STI if the current context does not need it; an atom can have high STI but low $p_t(R)$ if it is novel, diagnostic, or part of a rare-tail near-ACS motif. The coupling is mediated by stimulus flow (ACTPC-CHEM pays ECAN stimulus when a rule reduces prediction error or earns reward) and a soft KL regularizer in the joint loss (§16.10). The separation is what prevents the system from collapsing into a brittle high-utility circuit.

16.2 Working memory and long-term memory under ECAN

Working memory and long-term memory are not separate modules in this design; they are operational consequences of the attention economy.

- *Working memory* = the hot pool plus context-local scratch atoms. Lifetime is one tick or a small number of ticks. STI for working-memory atoms is paid by the active context and decays rapidly when the context moves on.

- *Long-term memory* = atoms with sufficiently high LTI to survive forgetting. Includes durable rules, meta-guards, ACS macro-atoms, and abstractions accumulated by PLN.

In schema terms, working memory atoms are still represented as before (`wm C42 entity 09`, etc.), but their persistence is now under the control of ECAN’s STI dynamics rather than ad-hoc TTL fields.

16.3 Atom and link schemas

ECAN adds the following kinds of relation-first atoms to those of §8:

- *Attention facts*: per-atom `sti` and `lti` values, global `pool` membership, context-local `pool-hot` membership, `overuse-trace`, and `context-relevance`.
- *Association links*: sparse `hebb` and `inv-hebb` edges with weights, plus auxiliary `coactive`, `co-catalytic`, and `context-assoc` counters used by the kernels that maintain them.
- *Stimulus events*: explicit `stimulus` logs during development, summarized into running totals afterwards.
- *ACS macro-atoms*: reified handles for recurrent useful ACSs, themselves subject to STI/LTI dynamics. This is the algorithmic-chemistry version of ECAN map formation: an attentional attractor becomes a manipulable symbolic object that can be reasoned about, planned with, and evolved.

(`pool R warm`) expresses global pool membership; the context-local (`pool-hot C R`) expresses that *R* is in the hot pool for context *C*. The hot loop should query by the context-local form, since several contexts can share the same warm soup but disagree on what is hot for them. Concrete schema examples and the full set of ECAN fact relations are in the appendices.

16.4 Stimulus sources

Every other component pays ECAN stimulus on atoms it found useful. The aggregate is what drives wages.

Source	What it stimulates
ACTPC-CHEM	Rules that reduce prediction error, earn reward, or contribute to learning progress. Stimulus is proportional to per-rule credit.
ACS detector	Members, catalysts, products, and rare-tail precursors of active useful ACSs. Modulated by ablation-coupling score so that decorative ACSs do not collect stimulus.
MOSES	High-fitness program/rule fragments, surviving GCODD roots, CENF meta-guards that compress useful structure, fragments that transfer across task families.
AIRIS	Causal rules that explain surprising transitions and reduce future prediction error.
PLN	Abstractions and analogies that improve heldout prediction or compress rule count.
External reward	Atoms causally implicated in reward delivery, attributed via eligibility traces.
Context relevance	Atoms whose signatures match active observations or goals receive a small contextual stimulus.

A simple ACTPC-CHEM-side stimulus on a fired rule r is

$$\text{stim}_{\text{ACTPC-CHEM}}(r, t) = c_1 \Delta e_t^+(r) + c_2 R_t^+(r) + c_3 \text{Novelty}_t(r) - c_4 \text{cost}(r),$$

where $\Delta e_t^+(r)$ is the positive prediction-error reduction attributed to r . ACS-side stimulus on a member atom A of an active ACS S is

$$\text{stim}_{\text{ACS}}(A, t) = d_1 \text{Member}(A, S) + d_2 \text{Catalyst}(A, S) + d_3 \text{Coupling}(S) + d_4 \text{Transfer}(S),$$

with the coupling factor being the same ablation-based $U(\text{soup}) - U(\text{soup} \setminus S)$ ratio that already appears in the benchmark suite. This makes the attention economy and the benchmark suite point in the same direction: useful ACSs win, both in scoring and in attention.

16.5 Core ECAN dynamics

The core dynamics for STI and LTI are the standard ECAN economic update:

$$\begin{aligned} \text{Wage}_{\text{STI}}(A) &= w_s s_A, \\ \text{Wage}_{\text{LTI}}(A) &= w_l \text{Durable}(s_A), \\ \text{Rent}(A) &= \text{Rent}_{\text{base}}(A) + \text{Rent}_{\text{cost}}(A) + \text{Rent}_{\text{excess-can}}(A), \end{aligned}$$

where s_A is the aggregated stimulus on atom A and $\text{Durable}(s_A)$ extracts the portion of stimulus associated with repeated, transferable, or ablation-supported usefulness. The three rent components are:

- *Base rent*: a flat charge for occupying hot or warm pool slots; prevents permanent monopolies.
- *Cost rent*: scales with the runtime/structural cost of the atom.
- *Excess-canalization rent* (value-relative anti-precedence, §19.5): the recommended replacement for the older broad-overuse rent. It is computed at the level of the atom’s *pattern family* $g = \pi(A)$ and gated by a functional-protection coefficient:

$$\text{Rent}_{\text{excess-can}}(A) = r_O (1 - c_A) O_{\pi(A)}(t)^p$$

where $O_g(t) = [S_g(t) - S_g^*(t) - \theta_O]_+$ is the family’s excess canalization (realized share minus value-justified share), S_g^* is derived from the family value V_g via softmax, and $c_A \in [0, 1]$ is the functional protection coefficient. Atoms with high c_A (validators, repair rules, abstraction tools, shared meta-guards) pay reduced excess-canalization rent even when their family is over-dominant, because their value lies in shared function rather than implementation identity.

For ablation purposes the older broad-overuse rent $\text{Rent}_{\text{overuse}}(A) = r_h \text{cost}(A) [h_A - \theta_h]_+^2$ is retained as a baseline (config flag `anti_precedence.mode = broad`); the recommended setting is `anti_precedence.mode = family_protected`.

The recent-use trace updates as

$$h_A(t+1) = \lambda_h h_A(t) + \text{fired}(A, t) + \zeta \text{coactive}(A, t),$$

and is now used both for identity precedence (the positive ηh_r term in candidate scoring) and for computing family shares S_g .

After wages, rent, and diffusion, STI is renormalized to the global budget; LTI is updated more slowly and homeostatically:

$$\begin{aligned} \text{STI}(A) &\leftarrow B_{\text{STI}} \cdot \frac{\max(0, \text{STI}(A))}{\sum_B \max(0, \text{STI}(B)) + \varepsilon}, \\ \text{LTI}(A) &\leftarrow (1 - \lambda_L) \text{LTI}(A) + \text{Wage}_{\text{LTI}}(A) - \text{Rent}_{\text{LTI}}(A). \end{aligned}$$

16.6 Family-level structures and anti-precedence kernels

Anti-precedence as defined in §19.5 operates at the level of *pattern families* $g \in \mathcal{G}$, not individual atoms. ECAN maintains, for each family:

- leaky usage trace $U_g(t)$ and realized share $S_g(t)$;
- value $V_g(t)$ derived from a weighted blend of the diagnostics already computed by the benchmark suite (ACS–utility coupling, generativity, transfer, resilience, cost, lock-in);
- value-justified share $S_g^*(t) \propto \exp(\beta V_g(t))$;
- excess canalization $O_g(t) = [S_g - S_g^* - \theta_O]_+$;
- neighbor-release stimulus $R_g(t) = \sum_h K(g, h) O_h(t)$, computed over the same neighbor graph the OT module already maintains (§13).

Each rule r also carries a functional-protection coefficient c_r estimated from ablation-tested value per unit flux. High- c_r atoms (validators, repair rules, shared abstraction constructors, meta-guards that improve heldout prediction across families) are partially shielded from excess-canalization rent.

Concretely, this introduces a small set of new grounded ECAN kernels alongside the existing list (§16.13):

```
ECAN_UPDATE_FAMILY_TRACES      ; U_g(t), S_g(t)
ECAN_COMPUTE_FAMILY_VALUE      ; V_g(t)
ECAN_COMPUTE_EXCESS_CANALIZATION ; O_g(t)
ECAN_COMPUTE_FUNCTIONAL_PROTECTION ; c_r(t)
ECAN_APPLY_PROTECTED_RELEASE_RENT ; (1 - c_r) * O_pi(r)
ECAN_PAY_RELEASE_BONUSES       ; rho * R_pi(r)
ECAN_TRIGGER_VARIANT_PROPOSALS ; ask MOSES/gCoDD/CENF for variants
                                ; of high-V high-O families
```

ECAN_TRIGGER_VARIANT_PROPOSALS closes the loop with the outer loop (§14): when a family has both high V_g and high O_g , simply retiring it would lose useful function. Instead, ECAN writes structural-request atoms of the form

```
(structural-request C42 T193
  over-entrenched-family G17)
(structural-request C42 T193
  preserve-function-vary-implementation G17)
```

so that MOSES / gCoDD / CENF can mutate the implementation while preserving function, rather than randomly perturbing it.

The detailed atom and link schemas for pattern families (`pattern-family`, `family-share`, `family-value`, `family-excess-canalization`, `functional-protection`, `functional-role`, `family-release-nei`) are listed in the appendices.

16.7 Sparse Hebbian diffusion

For each atom A in the diffusion frontier (hot pool, warm pool, and a bounded one-hop neighborhood), spread at most a fraction m of its STI along its top- k Hebbian neighbors:

$$\Delta\text{STI}(B) = m \text{STI}(A) \cdot \frac{H(A, B)}{\sum_{B' \in N_H(A)} H(A, B') + \varepsilon},$$

and similarly subtract for InverseHebbian neighbors. Hebbian weights are updated from observed coactivity over a recent window:

$$H_{t+1}(A, B) = (1 - \lambda_H) H_t(A, B) + \eta_H \text{CoAct}(A, B, t),$$

where coactivity may include same-tick firing, same candidate application, same ACS membership, same MOSES individual, or same context-local AF.

Diffusion never runs over the whole MORK. The frontier is bounded by construction; this is where the prefix-friendly schemas of §8 pay off most directly.

16.8 Pool migration

Pool migration is the operational output of ECAN. The default rules:

- **Cold** → **warm**: promote when context signature matches the atom, or the atom is strongly Hebbian-associated with a hot atom, or has high LTI for the current task family, or is a near-ACS member with low defect score, or is requested by MOSES/AIRIS/PLN/ACTPC-CHEM.
- **Warm** → **hot**: under strict per-context, per-signature caps. Selected by a hot score that combines STI, context relevance, ACTPC-CHEM support, ACS bonus, novelty quota, and negative cost.
- **Hot** → **warm**: when STI falls below threshold, the active signature disappears, the atom is unused for too many ticks, or its overuse rent exceeds its stimulus.
- **Warm** → **cold**: when the atom is unused for many ticks, no longer associated with any hot atom, not part of an active or near-active ACS, and not requested by any module.
- **Cold** → **forgotten**: when LTI falls below the retention floor and the atom is not part of any archived ACS, lineage, or rare-tail archive. In experimental work, this should mean archival compaction rather than irreversible deletion.

16.9 Hot-pool quotas

The hot pool should not be filled exclusively by current high-utility atoms. To preserve the rare-tail productivity that the benchmark suite rewards, hot-pool slots are partitioned into soft buckets:

high-utility / high-STI:	60-80% of hot pool
high-novelty:	10-20%
rare-tail near-ACS:	5-15%
SB / geoteleomic guidance:	5-10% (when active)

These are soft quotas, not hard laws; the point is to make sure a few high-utility atoms cannot permanently exclude all novelty.

16.10 Information-geometric ECAN

Once the ECAN-lite economy is stable, an information-geometric step can be added on the warm/hot subgraph. View STI, suitably normalized, as a probability distribution

$$a_t(A) = \frac{\text{STI}(A)}{\sum_B \text{STI}(B)}.$$

Let $u_t(A)$ be the utility/stimulus vector. A KL-regularized mirror-descent step solves

$$a_{t+1} = \arg \max_{a \in \Delta(\mathcal{B}_t)} \left[\langle u_t, a \rangle - \lambda \text{KL}(a \parallel a_t) \right],$$

which has the closed-form update

$$a_{t+1}(A) \propto a_t(A) \exp(u_t(A)/\lambda).$$

This is the simplest information-geometric ECAN variant. It is cheap, preserves nonnegativity and normalization automatically, and gives a principled alternative to ad hoc rescaling after wages and rent.

A more advanced step adds a Wasserstein transport term over the association graph, using a ground cost ω_{ij} derived from Hebbian strength, structural similarity, co-catalysis frequency, shared ACS membership, or gCoDD/CENF similarity:

$$a_{t+1} = \arg \min_{a \in \Delta(\mathcal{B}_t)} \left[-\langle u_t, a \rangle + \lambda \text{KL}(a \parallel a_t) + \frac{1}{2h} W_\omega^2(a, a_t) + \chi \text{Overuse}(a) \right].$$

This says: move attention toward utility, but do not move too far from the current attention distribution, prefer moving along cheap association paths, and pay a penalty for stale high-traffic pathways. The transport cost can be made history-sensitive in the same way as the rule-space transport cost in §13:

$$\omega_{ij}^{eff}(t) = \omega_{ij} \exp(-\eta \bar{h}_{ij}(t) + \xi \Phi(\bar{h}_{ij}(t); \theta) - \lambda_{SB} \Delta \psi_{ij}(t)).$$

For scalability, the information-geometric step runs only on a bounded subgraph (typically 512–4096 atoms) and only periodically (every 50–500 ticks). It does not run over the whole MORK.

16.11 Coupling ECAN and ACTPC-CHEM

A useful joint regularizer couples the two distributions softly:

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{ACTPC-CHEM}}(p) + \lambda_1 \text{KL}(p \parallel a_R) + \lambda_2 \text{KL}(a_R \parallel p) + \Omega_{\text{resource}}(a),$$

where a_R is the ECAN attention distribution restricted to rule atoms. The two KL directions have different meanings:

- $\text{KL}(p \parallel a_R)$: do not starve rules that ACTPC-CHEM believes are important.
- $\text{KL}(a_R \parallel p)$: do not waste too much attention on rules unsupported by the model.

Use this coupling gently. ECAN must be allowed to diverge from ACTPC-CHEM for exploration, diagnostics, and rare-tail ACS formation.

16.12 Division of labor: MM2, MeTTA, and grounded kernels

A natural temptation is to express the entire ECAN economy in MeTTA or MM2. Resist it. The ECAN update is dominated by operations that pure graph rewriting handles poorly: numeric arithmetic on STI/LTI vectors, weighted random sampling, sparse top- k retrieval over the association graph, KL-regularized mirror descent, and periodically a Wasserstein transport step. These belong in grounded kernels, just as ACTPC-CHEM’s softmax sampling and KL/cross-entropy already do. MM2’s role with respect to ECAN is much more limited: it consumes the kernels’ output and uses it as a selectivity gate.

The clean division is:

Layer	ECAN responsibilities
MORK	Persistent indexed storage of <i>all</i> ECAN facts: (<code>sti R V</code>), (<code>lti R V</code>), (<code>pool R warm</code>), (<code>pool-hot C R</code>), (<code>hebb A B w</code>), (<code>inv-hebb A B w</code>), (<code>overuse-trace R h</code>), (<code>stimulus T R v src</code>), and ACS macro-atoms. Prefix-indexed retrieval of pool-hot atoms during the match phase, where the relation-and-context prefix gives strong selectivity.
MM2	Two things only. (1) <i>Consume</i> pool decisions: the match phase includes (<code>pool-hot \$C \$R</code>) as a conjunct, gating candidate generation to atoms ECAN has admitted. (2) <i>Emit</i> stimulus markers: when a rule fires successfully, contributes to prediction-error reduction, or is part of a winning candidate set, an <code>exec</code> writes a small (<code>stimulus-event T R kind v</code>) atom that the next <code>ECAN_GATHER_STIMULUS</code> call aggregates. MM2 does not compute STI/LTI updates, rent, diffusion, or pool migration.
MeTTA	High-level configuration and glue: stimulus-weight tables, ECAN policy parameters, retrieval helpers used outside the hot loop, and human-readable definitions of <code>durable_component</code> , hot-pool quotas, and pool-migration thresholds. MeTTA can also host small-batch retrieval rules used by structural-loop jobs (which are already off the hot path), but <i>never</i> runs the per-tick ECAN update.
Grounded kernels	Everything numeric: <code>ECAN_GATHER_STIMULUS</code> , <code>ECAN_PAY_WAGES</code> , <code>ECAN_COLLECT_RENT</code> , <code>ECAN_DIFFUSE_STI</code> , <code>ECAN_UPDATE_HEBBIAN</code> , <code>ECAN_PROMOTE_DEMOTE</code> , <code>ECAN_INFO_GEOM_STEP</code> , <code>ECAN_FORGET</code> . These read and write MORK atoms but compute in typed numeric structures host-side. The <code>ECAN_INFO_GEOM_STEP</code> kernel is the only one that uses dense linear algebra, and only on the bounded warm/hot subgraph.

Why this matters: a tempting but wrong design has MM2 `execs` walking the entire association graph each tick to update Hebbian weights, or has a MeTTA interpreter computing softmaxes over STI. Both will dominate runtime within minutes of the first nontrivial benchmark. Keeping ECAN numeric work in kernels and using MM2 only to gate retrieval and emit stimulus markers preserves the bounded-cost property of the inner loop.

There is one operationally important consequence: the (`pool-hot $C $R`) fact is what makes the MM2 hot loop *actually* hot. Without it, the match phase would have to scan (`pool $R hot`) or rule-status atoms across all contexts. With it — because it is context-prefixed and small — MORK’s PathMap index can return the per-context hot rules in essentially $O(K_{\text{hot}})$ time per active signature.

Wrapper-level parallelism. The same division of labor applies to concurrency. MM2 is not the global parallel scheduler. The global scheduler is a wrapper-level worker runtime. In the near term this can be PETTA/SWI-Prolog; in the long term it should be METTA-IL/Rholang. Both versions should run the same Worker IR.

Layer	Concurrency responsibility
MM2	Single-threaded chamber execution over one hot pool.
PETTA wrapper	Near-term scheduler for chambers, demes, scans, replay batches, ECAN epochs, and bri
METTA-IL / Rholang	Future scheduler expressing the same worker graph with <code>par</code> .
Grounded kernels	Data-parallel numeric and stochastic work called by workers.
Reducers	Synchronization boundary: merge logs and deltas, publish new global state.

If a PETTA-specific predicate cannot be translated into a Worker IR fact or a future METTA-IL worker form, it should be treated as an implementation detail rather than an architectural primitive.

16.13 Grounded ECAN kernels

The following operations live in host kernels rather than pure MM2:

ECAN_GATHER_STIMULUS	aggregate stimulus from all sources
ECAN_PAY_WAGES	apply wage_sti and wage_lti
ECAN_COLLECT_RENT	base/cost/excess-canalization rent
ECAN_UPDATE_OVERUSE_TRACES	decay and bump h_A
ECAN_UPDATE_FAMILY_TRACES	decay and bump U_g, recompute S_g
ECAN_COMPUTE_FAMILY_VALUE	blend benchmark diagnostics into V_g
ECAN_COMPUTE_EXCESS_CANALIZATION	$O_g = [S_g - S*_g - \text{theta}]_+$
ECAN_COMPUTE_FUNCTIONAL_PROTECTION	c_r per rule
ECAN_APPLY_PROTECTED_RELEASE_RENT	$(1 - c_r) * O_{pi}(r)$
ECAN_PAY_RELEASE_BONUSES	$\rho * R_{pi}(r)$ into neighbor families
ECAN_TRIGGER_VARIANT_PROPOSALS	request preserve-function-vary-impl
ECAN_SPARSE_DIFFUSE_STI	bounded top-k Hebbian/InverseHebbian spread
ECAN_UPDATE_HEBBIAN	coactivity-based H/I weight updates
ECAN_NORMALIZE_BUDGETS	STI budget renormalization, LTI homeostat
ECAN_PROMOTE_COLD_TO_WARM	retrieval-based promotion
ECAN_PROMOTE_WARM_TO_HOT	bucketed top-k under quotas
ECAN_DEMOTE_AND_FORGET	HotWarmCold demotions and archival
ECAN_INFO_GEOM_STEP	mirror-descent / Wasserstein update
ECAN_CREATE_ACS_MACRO_ATOMS	reify recurrent useful ACSs
ECAN_FIND_ATTENTIONAL_ATTRACTORS	detect attractor candidates

16.14 Worker-level ECAN

ECAN can allocate attention at two levels. Atom-level ECAN decides which rules, fragments, traces, meta-guards, and ACS macro-atoms are cold, warm, or hot. Worker-level ECAN decides which context chambers, demes, ACS scans, AIRIS/PLN proposal jobs, replay batches, and bridge-fitting jobs receive runtime budget.

This is not a separate mechanism. Workers are also atoms for ECAN purposes:

```
(worker W17)
(worker-kind W17 context-chamber)
(worker-status W17 ready)
```



```

(worker-context W17 C42)
(worker-budget W17 cpu-ms 50)
(sti W17 0.031)
(lti W17 0.44)
(worker-priority W17 0.71)
(worker-runtime-credit W17 0.18)
(worker-output-utility W17 0.22)
(worker-output-accs-yield W17 0.03)
(worker-rent W17 cpu 0.004)

```

A worker earns stimulus when it produces useful event logs, high-value candidate rules, good ACS detections, improved bridge potentials, or attention updates that improve downstream hot-pool precision. A worker pays rent for CPU time, memory, and queue occupancy. Worker-level ECAN therefore turns runtime scheduling into part of the same attention economy as atom selection.

A simple worker score is:

$$Score(W) = \alpha_s \log(STI(W) + \epsilon) + \alpha_l \log(LTI(W) + \epsilon) + \alpha_u UtilityYield(W) + \alpha_n NoveltyYield(W) - \kappa_{cpu} CpuCost(W)$$

The PETTA scheduler can use this score now, and the METTA-IL scheduler should use the same score later. This gives the system a path toward evolving not only rules, but also reaction-chamber organization.

17 MORK-specific performance strategy

17.1 Hot-path query rules

1. Query by current context first.
2. Query by active signature before rule id if signature is more selective.
3. Query by rule id before metadata if a rule has already been chosen.
4. Split disjoint operations into separate `execs`.
5. Avoid nested logical expressions on the hot path.
6. Cache derived features and meta-guards for the current tick.
7. Use trial and active rule statuses to avoid matching retired rules.
8. Use top- k candidate caps per signature.

Concrete examples of good versus bad schemas, candidate-cap mechanics, garbage collection, and sharding strategies are in the appendices.

18 Hardware targets and population sizing

18.1 Why a CPU-focused server is preferred

The core ACTPC-CHEM-on-MORK workload is not primarily a dense linear-algebra workload. The inner-loop operations are more like:

1. ingest symbolic observations into a context-local working memory;
2. activate a small set of currently relevant signatures;
3. retrieve top-k rules per active signature;
4. match compact compiled rule triggers against context facts;
5. score and sample candidate rule applications;
6. fire a small number of reactions;
7. emit actions, predictions, traces, and credit-assignment facts;
8. clean scratch atoms and compact old traces.

The performance drivers are therefore:

- CPU parallelism for many independent contexts, demes, candidate batches, and local rewrites;
- RAM capacity for the persistent rule soup, active indices, context facts, trace buffers, factor graphs, gCoDD/CENF structures, and caches;
- memory bandwidth and NUMA behavior, because indexed symbolic operations are often memory-bound;
- fast local NVMe for checkpoints, logs, trace archives, and batch evaluation data;
- stable bare-metal performance for profiling.

A GPU server is useful when the system includes neural perception, embedding-based retrieval, learned rule scorers, batch rollouts, or continuous predictive-coding modules. However, an expensive GPU with limited host RAM is a worse match than a CPU-heavy server with large RAM for the symbolic chemistry substrate.

18.2 Recommended machine classes

18.2.1 Starter engineering server

Enough to build and debug the system, validate MM2 phases, measure atom churn, and run moderate algorithmic chemistry experiments.

CPU:	32-64 physical cores
RAM:	256-512 GB ECC
Storage:	2-4 TB local NVMe
Network:	1-10 Gbps sufficient
GPU:	optional; small or absent is fine
Use case:	implementation, profiling, debugging, first learning runs

Suggested conservative population settings:

persistent rules:	30k - 150k
active rules:	2k - 15k
hot rules/context/tick:	128 - 1024
candidates/tick:	256 - 2048
firings/tick:	1 - 16
MOSES demes:	8 - 32
population/deme:	64 - 256
recent traces:	10k - 500k

18.2.2 Default serious single-server target

Recommended rental target for real experiments once the code is stable.

CPU:	64-96 physical cores, preferably AMD EPYC Genoa/Bergamo/Turin
RAM:	512-768 GB ECC DDR5
Storage:	2 x 4 TB local NVMe preferred
Network:	10 Gbps nice but not essential
GPU:	optional; useful for side modules only
Use case:	serious single-node ActPC-Chem/MORK/MOSES runs

Suggested default population settings:

persistent rules:	150k - 500k
active rules:	15k - 50k
hot rules/context/tick:	1024 - 2048
candidates/tick:	2048 - 4096
firings/tick:	16 - 32
MOSES demes:	32 - 64
population/deme:	256 - 512
recent traces:	500k - 2M

18.2.3 Aggressive single-node target

Useful once the default regime is stable and memory management has been stress-tested.

CPU:	128-192 physical cores
RAM:	1.0-1.5 TB ECC RAM
Storage:	4-8 TB local NVMe
Network:	10-25 Gbps helpful for moving checkpoints/logs
GPU:	optional; 1-4 GPUs useful if neural side workloads are significant
Use case:	scale tests, long runs, many demes, large trace buffers

Suggested aggressive population settings:

persistent rules:	500k - 2M+
active rules:	50k - 150k
hot rules/context/tick:	2048 - 8192
candidates/tick:	4096 - 16384
firings/tick:	32 - 64
MOSES demes:	64 - 128
population/deme:	512 - 1024
recent traces:	2M - 10M

Do not start here. Reach this regime only after the smaller settings have stable profiling, bounded scratch-atom churn, reliable checkpointing, and predictable garbage collection.

18.3 Cold, warm, and hot populations

The implementation should distinguish three different population sizes, and ECAN is what migrates atoms among them:

Cold / persistent soup:

Long-term population of rule records, stored, indexed, scored, canonicalized, checkpointed, and occasionally revived. Membership is set when LTI is below the warm-promotion threshold but above

the forgetting floor. Cold atoms can have high LTI and low STI.

Warm / active soup:

Rules with enough current attention, relevance, recency, or credit to be eligible for retrieval and use in the near future. ECAN promotes atoms here from cold based on context-signature matches, Hebbian neighborhoods of hot atoms, near-ACS membership, and explicit module requests.

Hot / MM2 working set:

Context-local bounded subset of eligible rules actually admitted into a cognitive tick as candidate reaction participants. ECAN promotes atoms here from warm based on STI, context relevance, ACS membership, and bucket quotas (utility / novelty / rare-tail / SB-guided).

This is the central scaling idea. The whole soup should not be chemically active inside MM2 at every point in time. A large persistent soup is the reservoir; the hot loop is a small reaction chamber; ECAN is the circulatory system that decides what enters the chamber. A reasonable ratio among the three levels is:

cold : warm : hot $\sim$ 1000 : 100 : 1

More concretely:

Regime	Cold rules	Warm rules	Hot rules/context/tick
Smoke test	1k–5k	256–1k	32–128
Small run	10k–30k	1k–3k	128–512
Default run	150k–500k	15k–50k	1024–2048
Aggressive run	500k–2M+	50k–150k	2048–8192

The Hebbian and InverseHebbian association graphs add a smaller but non-trivial memory cost on top of the rule records: at 16–64 Hebbian and 8–32 InverseHebbian edges per atom, a 500k-atom warm soup carries $\sim$ 30M to $\sim$ 50M association edges. At 64 bytes per edge record that is on the order of a few GB — comfortable on the recommended hardware, but a real budget item that should be tracked alongside trace memory.

18.4 Inner MM2-loop sizing

18.4.1 Recommended hot-loop caps

The inner loop should be explicitly capped. A default serious setup is:

```
active_signatures_per_context = 8 - 32
rules_per_signature_cap      = 32 - 64
hot_rules_per_context        = 1024 - 2048
candidate_cap_per_tick       = 2048 - 4096
fire_cap_per_tick            = 16 - 32
working_memory_fact_cap      = 8192 - 32768
scratch_atom_cap_per_tick    = 25000 - 50000
trace_ttl_ticks              = 4096 - 8192
```

ECAN-related caps

```

ecan_max_hebb_edges_per_atom    = 16 - 64
ecan_max_inv_edges_per_atom     = 8 - 32
ecan_diffusion_depth            = 1 - 2 hops
ecan_max_spread_fraction        = 0.05 - 0.20 of STI
ecan_info_geom_subgraph         = 512 - 4096    (when enabled)
ecan_info_geom_period_ticks     = 50 - 500     (when enabled)

# Hot-pool quotas (soft)
hot_quota_utility               = 0.60 - 0.80
hot_quota_novelty               = 0.10 - 0.20
hot_quota_rare_tail             = 0.05 - 0.15
hot_quota_sb_guided             = 0.05 - 0.10

```

Candidate generation should be bounded *before* it explodes. The system should not try every rule against every observation.

18.4.2 MM2 tick skeleton with bounding

```

procedure INNER_TICK(context C):

    OBS = read_current_observations(C)

    ACTIVE_SIGS = grounded_active_signature_extractor(OBS)
    ACTIVE_SIGS = top_k(ACTIVE_SIGS, K_SIGS_PER_CONTEXT)

    candidate_buffer = empty

    for sig in ACTIVE_SIGS:

        RULES_SIG = retrieve_rules_by_trigger(sig)
        RULES_SIG = top_k(
            RULES_SIG,
            K_RULES_PER_SIG,
            key = attention + logit + recent_credit - cost_penalty)

        OBS_SIG = observations_with_signature(C, sig)

        for obs in OBS_SIG:
            CANDS = match_compiled_triggers(obs, RULES_SIG)
            append CANDS to candidate_buffer

            if size(candidate_buffer) >= CANDIDATE_CAP:
                break

        if size(candidate_buffer) >= CANDIDATE_CAP:
            break

    SCORED = grounded_score_candidates(candidate_buffer)
    CHOSEN = softmax_or_topk_sample(SCORED, FIRE_CAP)

    for app in CHOSEN:
        fire_rewrite_application(app)
        write_trace(app)

```

```

update_short_term_attention(app)

delete_scratch_candidates(C)
compact_expired_traces(C)

```

18.4.3 Why not all rules hot?

If the system has R rules, O observations, and naive matching tries all rules against all observations, the hot loop tends toward an $O(RO)$ scan. This is fatal once R reaches hundreds of thousands. The scalable form converts the practical hot-loop size from the global rule count to:

```
0(active_signatures * rules_per_signature_cap * observations_per_signature)
```

which is bounded by configuration. This is why the schema design in §8 insists on relation-and-grounded-key prefixes: it is what makes the bounded form actually bounded.

18.5 Persistent rule-soup memory sizing

A compact rule record should be split into multiple small indexed atoms rather than one giant expression. A typical rule may require:

```

rule metadata atoms:      5 - 10
trigger/index atoms:      1 - 5
template/compiled atoms:  2 - 10
statistics atoms:         5 - 20
gCoDD/CENF references:    1 - 5
optional neighbor links:  0 - 32

```

A rough total is 15–50 atoms per persistent rule, before traces and neighbor expansion. Thus:

```

100k rules -> 1.5M - 5M rule-related atoms
500k rules -> 7.5M - 25M rule-related atoms
1M rules   -> 15M  - 50M rule-related atoms

```

This is plausible on a large server, but only if symbol interning is efficient, hash-consing avoids duplicate structure, traces are bounded and compressed, scratch atoms are aggressively deleted, cold rules can be frozen, checkpointed, or unloaded, and indexes are profiled and tuned.

A good rule of thumb is that the persistent rule soup should not consume more than about 40–50 percent of RAM. The rest is needed for active indices, working memory, trace buffers, demes, factor graphs, checkpoints, logs, and OS cache.

18.6 MOSES/gCoDD/CENF outer-loop sizing

The outer structural-evolution layer should run at a slower rhythm than the inner ACTPC-CHEM loop. It creates, mutates, abstracts, canonicalizes, and retires rules.

18.6.1 Default MOSES parameters

```

MOSES_demes           = 32 - 64
population_per_deme    = 256 - 512
elite_per_deme         = 32 - 64
structural_epoch_ticks = 200 - 1000
EDA_variables_per_deme = 64 - 256

```

factor_arity_cap	= 4 initially
nary_factor_cap	= 128 - 512
CENF_period_generations	= 5 - 20
migration_period_generations	= 10 - 50

A good first serious choice is:

MOSES_demes	= 32
population_per_deme	= 256
elite_per_deme	= 32
structural_epoch_ticks	= 200
EDA_variables_per_deme	= 192
factor_arity_cap	= 4

18.6.2 Outer-loop rhythm

Every inner tick:
 update rule logits, traces, local attention, and immediate credit.

Every 10-50 ticks:
 run delayed credit assignment and pattern-count updates.

Every 200-1000 ticks:
 run structural epoch:
 select high-credit traces/rules;
 extract rule fragments;
 mutate and recombine gCoDD/CENF forms;
 evaluate candidates;
 insert new rules with low/moderate priors;
 retire weak/cold active rules.

Every several structural epochs:
 run CENF/meta-guard updates;
 update factor graph structures;
 migrate exemplars among demes.

18.7 Trace and checkpoint requirements

Trace management can dominate memory unless capped. The trace system should support three levels:

Hot traces:
 Detailed, recent, RAM-resident traces for delayed credit assignment.

Warm trace summaries:
 Aggregated statistics by rule, signature, context, reward pattern, prediction-error pattern, and causal discrepancy.

Cold trace archives:
 Compressed logs on NVMe for offline analysis, reproducibility, debugging, and later batch learning.

Suggested caps:

```

Starter:
    recent_trace_cap = 100k - 250k
    trace_ttl_ticks  = 1024 - 4096

Default:
    recent_trace_cap = 500k - 2M
    trace_ttl_ticks  = 4096 - 8192

Aggressive:
    recent_trace_cap = 2M - 10M
    trace_ttl_ticks  = 8192+

```

Checkpointing should write the persistent rule table and canonical IDs; rule logits, costs, attention values, and utility statistics; active/warm rule indices; MOSES deme states and factor graphs; CENF/meta-guard definitions; recent trace summaries; random seeds and configuration; and environment/simulator state if applicable.

18.8 Role of GPUs

A GPU is optional but can help substantially if assigned the right jobs.

Good GPU tasks:

```

continuous predictive-coding perception/action modules
image/audio/sensor encoders
embedding generation for states, contexts, and rules
approximate nearest-neighbor retrieval over vectorized rules
learned rule utility scoring
batch environment rollouts
batch MOSES fitness evaluation
some factor-graph or sampling kernels
LLM-assisted rule proposal or explanation, if used locally

```

Poor GPU tasks:

```

whole-soup symbolic matching
fine-grained irregular graph rewrites
massive random atom insertion/deletion
pointer-heavy MORK index traversal
unbounded scratch-atom churn

```

The symbolic substrate should remain CPU/RAM-centric. GPU work should be batched and coarse-grained.

18.9 Bare metal versus cloud VM

Bare metal is preferred for the first serious MORK/MM2 profiling runs. Bare-metal advantages: predictable NUMA and memory-bandwidth behavior; no noisy-neighbor effects; direct local NVMe performance; easier CPU pinning and process layout; clearer profiling of MORK indexes and garbage collection; often cheaper for long-running high-RAM CPU workloads. Cloud VM advantages: easier provisioning and teardown; simpler scaling experiments; easier snapshotting and integration with managed storage; useful for short tests or when bare-metal procurement is slow. For the main research run, prefer bare metal if available. Use cloud VMs for short experiments, integration testing, and reproducibility checks.

18.10 NUMA and process layout

On a dual-socket or many-core EPYC system, NUMA matters. The implementation should avoid a single shared global hot data structure that all threads mutate randomly. Better patterns:

1. shard contexts by NUMA node;
2. shard MOSES demes by NUMA node;
3. keep context-local scratch atoms local;
4. batch cross-shard migrations;
5. use per-shard candidate buffers and trace buffers;
6. merge global statistics periodically, not every tick;
7. pin worker threads to cores or NUMA regions during profiling.

A natural layout:

```
NUMA node 0:
  contexts 0..K
  demes 0..D0
  local rule-trigger cache
  local scratch and trace buffers

NUMA node 1:
  contexts K+1..M
  demes D0+1..D
  local rule-trigger cache
  local scratch and trace buffers

Global service:
  cold rule store
  checkpoint writer
  global statistics aggregator
  rule retirement/insertion manager
```

Worker placement for PETTA and METTA-IL. For the PETTA-backed prototype, the same layout becomes a worker-placement problem. Context-chamber workers should be assigned near their context-local MORK view, scratch buffers, trigger cache, and event-log writer. MOSES deme workers should be assigned near their deme state and factor graph. ACS-scan workers should be assigned near event-log shards. ECAN reducers and bridge workers can run less frequently and may be placed globally.

A practical deployment target on a 96-core server is:

```
48-64 cores: context-chamber workers
8-16 cores:  MOSES / CENF / structural workers
8-16 cores:  ACS scans and replay/ablation workers
4-8 cores:   ECAN reducers and worker-level scheduler
2-8 cores:   bridge fitting, logging, checkpointing
```

The future METTA-IL backend should preserve the same worker-placement policy, even if the surface syntax changes from PETTA predicates to **par** forms.

19 Evaluation: useful autocatalytic generativity

19.1 Benchmark philosophy

The system’s strength is not ordinary task-optimized machine learning, but *interesting generativity*. The chemistry should evolve networks of rules and artifacts that are alive in a minimal operational sense: they maintain themselves, catalyze each other’s production, generate novel useful variants, survive perturbation and transfer, and contribute causally to external work.

A benchmark suite for this purpose should not ask only:

Did the soup solve the task?

It should ask:

Did the soup evolve active autocatalytic subnetworks whose ablation damages useful generative behavior, whose serial transfer preserves function, whose rare fringe contributes new useful structures, and whose overused habits are released before they congeal into sterile repetition?

This distinction is crucial. A fixed circuit, a memorized solution, or an over-specialized policy can solve a narrow benchmark while lacking the self-maintaining generativity that algorithmic chemistry is supposed to bring. The benchmark suite therefore treats external utility as only one axis of success. The other axes measure autocatalysis, coupling between autocatalysis and utility, generativity, heredity, resilience, and anti-rigidity.

19.2 Pareto objective, not a single fitness function

A simple scalar objective such as

$$\text{Score} = U + \lambda A$$

where U is external utility and A is degree of autocatalysis is too easy to pervert. The system can score well by keeping the two achievements separate: a task circuit for U and a decorative autocatalytic loop for A .

Instead maintain a Pareto archive over at least the following axes:

A = active autocatalysis,
 U = external utility,
 C = autocatalysis–utility coupling,
 G = generativity / novelty / diversity,
 H = heredity / transfer stability,
 R = resilience / anti-rigidity,
 K = cost / complexity.

Maximize A, U, C, G, H, R and minimize K and brittleness. Apply hard filters such as

$$A \geq A_{\min}, \quad C \geq C_{\min}, \quad H \geq H_{\min}, \quad U_{\text{heldout}} \geq U_{\min}.$$

19.2.1 Autocatalysis–utility coupling

A useful coupling score is:

$$C(S) = \frac{U(\text{soup}) - U(\text{soup} \setminus S)}{U(\text{soup}) + \epsilon} \cdot \frac{\text{RewardFlux}(S)}{\text{RewardFlux}(\text{soup}) + \epsilon}.$$

Here $U(\text{soup} \setminus S)$ is measured by ablating the candidate ACS or suppressing its members and rerunning or replaying matched task episodes. The second factor measures how much reward-attributed reaction flux actually passes through S . This score asks whether the autocatalytic set is doing the useful work.

19.3 Core diagnostics

Every benchmark in the suite should produce the same dashboard.

19.3.1 ACS / RAF metrics

```
active_ACS_count
largest_active_ACS_size
ACS_births_per_1000_ticks
ACS_lifetime_distribution
active_ACS_fraction
catalysis_coverage
food_generation_coverage
defect_score_distribution
```

19.3.2 Usefulness metrics

```
external_task_score
heldout_task_score
reward_flux_through_ACS
ACS_ablation_delta_U
non_ACS_ablation_delta_U
```

19.3.3 Generativity metrics

```
valid_novel_artifacts
artifact_diversity
behavioral_diversity
rule_family_diversity
rare_tail_enrichment_E20
number_of_new_reusable_intermediates
```

19.3.4 Heredity metrics

```
parent_daughter_JSD
serial_transfer_success
dilution_recovery_time
composition_drift_per_generation
```

19.3.5 Anti-rigidity metrics

```
top_rule_ablation_drop
top_k_path_ablation_drop
alternative_path_count
task_distribution_shift_score
sclerosis_index
```

19.3.6 Adaptive-continuity and anti-precedence metrics

These metrics target the question that motivates value-relative anti-precedence (§19.5): does the system preserve transferable function while continually renewing the implementations through which that function is realized?

```
family_share_entropy          ; H(S_g) over pattern families
family_excess_canalization_mean ; mean O_g(t)
family_excess_canalization_max ; worst-offender O_g(t)
implementation_entropy_given_function ; H(Z | Y)
functional_continuity_score    ; P(Y_{t..t+tau} in V)
implementation_turnover_score  ; rate of change of dominant Z given Y
protected_release_score        ; gain from protected release
adaptive_regret                ; integrated value vs ideal trajectory
post_shift_recovery_auc        ; recovery after task / env shift
incumbent_lockin_auc           ; persistence of brittle incumbents
variant_takeover_under_continued_function
                                ; new family overtakes while function holds
```

In formula form, the implementation entropy is

$$\text{ImplementationEntropy} = H(Z_t | Y_t),$$

where Y_t is functional state and Z_t is implementation state. The functional continuity score over horizon τ is

$$\text{FunctionalContinuity}_\tau = P(Y_{t:t+\tau} \in \mathcal{V}),$$

where $\mathcal{V}$ is the viability condition (e.g. continued ACS-utility coupling above $C_{\min}$, continued held-out transfer, continued artifact-family validity). The protected-release score is

$$\text{ProtectedReleaseScore} = \Delta H(Z | Y) + \Delta \text{Recovery} - \Delta \text{UtilityLoss},$$

measured as the change relative to a baseline run without protected release. The adaptive evolutionary regret is

$$\mathcal{R}_{\text{evo}} = \int \left[M^*(t) - M(t) + \lambda_{\text{lock}} \text{LockIn}(t) + \lambda_{\text{decoy}} \text{Decoy}(t) - \lambda_{\text{var}} \text{VariantShare}(t) \right] dt,$$

where $M(t)$ is utility-coupled generative output (proof success rate, valid-artifact yield, service throughput, or membrane-conversion analogue), $M^*(t)$ its viable-future ideal, and LockIn, Decoy, VariantShare are tracked separately.

19.3.7 Attention-economy metrics

These measure whether ECAN is doing useful work, in addition to or instead of contributing usefully to the ACS and utility metrics.

<code>hot_pool_precision</code>	= #hot atoms used in useful events / #hot atoms
<code>promotion_recall</code>	= #useful atoms promoted before needed / #useful atoms eventually needed
<code>pool_churn</code>	= atoms migrating per 1000 ticks (too low = stale; too high = thrashing)
<code>mm2_candidate_waste</code>	= #candidates discarded by sampler / #candidates generated
<code>attention_concentration_HHI</code>	= $\sum_A \text{sti}(A)^2$ (Herfindahl on STI)
<code>attention_acs_overlap</code>	= fraction of STI mass on active ACS members
<code>ecan_overhead_fraction</code>	= ECAN kernel time / total tick time

Operationally, `hot_pool_precision` is the most useful single diagnostic: a hot pool that holds many atoms which never participate in fired candidates is a sign that promotion is too generous, the hot-pool quotas are off, or the Hebbian neighborhood is too dense.

19.3.8 Near-ACS defect score

Adapt the near-ACS defect score from the geoteleomic origin-of-life work (*Reverse-Engineering the Origin of Life*, Goertzel):

$$D(H; t) = w_{cat} d_{cat}(H) + w_{food} d_{food}(H) + w_{act} d_{act}(H; t) + w_{mass} d_{mass}(H; t).$$

For algorithmic chemistry:

$$\begin{aligned} d_{cat} &= \text{missing catalyst support,} \\ d_{food} &= \text{missing food-generated closure,} \\ d_{act} &= \text{insufficient repeated firing,} \\ d_{mass} &= \text{insufficient abundance, attention or trace mass.} \end{aligned}$$

$D = 0$ denotes an active ACS-like local patch. $D > 0$ denotes an almost-ACS motif. The fringe distribution over these motifs should be monitored, because the rare fringe may be disproportionately productive even when most births come from the common head.

19.3.9 Rare-tail enrichment

For $p \in \{0.10, 0.20, 0.30\}$ define:

$$E(p) = \frac{\text{birth precursor share among rarest } p \text{ fraction of motif types}}{\text{occupancy share of rarest } p \text{ fraction of motif types}}.$$

If $E(p) > 1$, the rare fringe is productive relative to its occupancy. If $E(p) \approx 1$, the tail is decorative. If $E(p) < 1$, the rare fringe is less productive than expected.

19.3.10 Sclerosis index

A rigid circuit often has concentrated flux, high ablation fragility, and little productive rare tail. A useful autocatalytic ecology may also be head-dominated, but it should remain resilient and tail-productive. A rough sclerosis index is:

$$\text{Sclerosis} = \frac{\text{HHI}(\text{reaction flux}) \cdot \text{AblationFragility}}{1 + E_{20}}.$$

This does not punish concentration alone. It punishes brittle concentration without rare-tail productivity.

19.4 Benchmark series

The suite ramps from pure ignition to useful symbolic generativity and then toward self-improving program ecologies. Each stage should be simple enough to instrument exhaustively but rich enough to require emergent networks rather than a single fixed rule.

19.4.1 Benchmark 0: ACS ignition and habit-trap calibration

Tune the chemistry and diagnostics before external task pressure is added.

World:

no meaningful external task yet

Food:

primitive symbolic atoms
simple constructors
random seed rules

Goal:

produce active, persistent autocatalytic rule sets

External reward:

none, or weak reward for maintaining nontrivial active closure

Main question:

can the soup ignite active ACSs at all?

Run conditions:

A: native ActPC only

B: ActPC + SB / geoteleomic guidance

C: ActPC + precedence (positive reuse bias)

D: ActPC + precedence + broad anti-resonance (old baseline)

D': ActPC + precedence + value-relative

anti-precedence + functional protection (recommended)

This benchmark identifies the phase diagram of the soup. It is also where one checks that the detector is not confusing trivial loops with genuine active closure, and where the broad-baseline and recommended anti-precedence variants can first be compared on a clean ignition problem.

19.4.2 Benchmark 1: productive grammar ecology

Recommended first useful benchmark.

```
World:
  symbolic artifact factory

Food:
  primitive tokens
  constructors
  simple rewrite operators
  type symbols
  base grammar fragments

Products:
  strings, trees, small MeTTa expressions, typed symbolic artifacts

External reward:
  produce valid, novel, nontrivial artifacts from a family,
  not one target artifact
```

Complexity ramp:

```
Level 1: finite-state grammar strings
Level 2: balanced parentheses / simple context-free grammar
Level 3: typed expression trees
Level 4: small executable MeTTa-like expressions that evaluate without error
Level 5: small expressions satisfying randomized semantic properties
```

A useful ACS may contain:

```
generators      -> create candidate expressions
validators      -> reject malformed expressions
repair rules    -> transform near-valid into valid
abstraction rules -> create reusable grammar motifs
samplers        -> vary motifs productively
```

A simple utility function is:

$$U = \#ValidNovelArtifacts + \alpha \text{SemanticUsefulness} + \beta \text{Diversity} - \gamma \text{Triviality}.$$

Do not reward exact strings. Reward valid families and reusable generative motifs.

19.4.3 Benchmark 2: autocatalytic abstraction discovery

A stripped-down food/poison style benchmark without a full organism.

```
World:
  stream of objects with symbolic features

Food:
  raw feature atoms:
    color, shape, texture, smell, context, time

Products:
  learned categories, meta-guards, causal/predictive rules
```

External reward:
predict hidden outcomes or choose symbolic labels/actions

Complexity ramp:

Level 1: single-feature rule
Level 2: XOR-like feature conjunctions
Level 3: noisy feature conjunctions
Level 4: delayed outcomes
Level 5: context-dependent delayed outcomes
Level 6: nonstationary concept drift

Example hidden regularities:

red + round + smooth -> beneficial
red + round + rough -> harmful
purple + sweet + active-after-ingestion -> beneficial
purple + sweet + inactive-after-ingestion -> harmful

The goal is not to build a good classifier but to evolve an ecology of abstraction-making rules in which feature-combiners catalyze category rules; category rules catalyze prediction rules; prediction errors catalyze refinements; and successful categories become food for higher-level categories.

19.4.4 Benchmark 3: lemma / proof autocatalysis

Perhaps the cleanest benchmark for useful symbolic generativity.

World:
tiny theorem-proving or equational-reasoning environment

Food:
axioms, primitive inference rules, rewrite identities

Products:
lemmas, proof macros, derived rewrite rules, proof plans

External reward:
solve randomly generated proof goals,
shorten proofs,
generate reusable lemmas

Complexity ramp:

Level 1: propositional tautology fragments
Level 2: term-rewrite simplification
Level 3: equality proofs in tiny algebraic theories
Level 4: induction-free list identities
Level 5: small program invariants or type-checking lemmas

Lemmas are naturally catalytic: they enable proof steps without being consumed. A lemma ecology may look like:

Lemma A enables proof macro M
Macro M derives Lemma B
Lemma B enables simplification rule R
R makes Macro M applicable in more contexts

External reward should be sampled from a stream of related proof goals, not one fixed theorem. Useful metrics: proof success, proof-length reduction, lemma reuse, lemma-ACS ablation delta, heldout transfer, and serial-transfer JSD over the lemma/rewrite-rule composition.

19.4.5 Benchmark 4: program-fragment repair and synthesis ecology

Connects naturally to MOSES/GCoDD/CENF but should remain focused on generative ecology rather than leaderboard-style program induction.

World:

small typed programs with holes, bugs, or missing transformations

Food:

primitive operations, type constructors, examples, tests

Products:

repair rules, program fragments, reusable subroutines,
candidate programs

External reward:

repair or synthesize small programs across a task family

Complexity ramp:

Level 1: list/string transformations

Level 2: tree transformations

Level 3: symbolic arithmetic simplifiers

Level 4: small planner fragments

Level 5: meta-repair: rules that repair repair rules

Example useful ACS:

type-checker fragments catalyze repair fragments;
repair fragments produce candidate programs;
candidate programs expose reusable subfragments;
subfragments become catalysts for future repairs;
CENF/meta-guards compress recurring conditions.

Anti-rigidity checks: new task families after training, heldout primitive combinations, ablation of top repair rules, changed type signatures, reward for reuse and diversity rather than pass rate alone.

19.4.6 Benchmark 5: blackboard protocol alchemy

A minimal test of network functionality without introducing a society of organisms or machines.

World:

shared blackboard of jobs, messages, resources, and products

Food:

incoming request tokens, resource tokens, primitive transformations

Products:

completed jobs, routing patterns, resource-allocation motifs,
reusable service protocols

External reward:
 maintain useful service under changing demand

Example request stream:

```
simplify expression
classify object
prove small lemma
generate valid artifact
repair small program
```

Rules can transform requests, decompose requests, create helper jobs, route jobs to specialized rule clusters, cache intermediate concepts, and recycle failed attempts. The main metric:

$$\frac{\text{useful service throughput causally routed through active ACSs}}{\text{total useful service throughput}}.$$

19.4.7 Benchmark 6: serial transfer and daughter-soup inheritance

A protocol applied to Benchmarks 1–5.

```
1. identify active useful ACSs
2. dilute the soup
3. transfer only:
    food set + selected ACS members + small random background
4. run on fresh task samples
5. measure:
    recovery time
    parent/daughter JSD
    utility retention
    novelty retention
    new ACS births
```

Rigid circuits often fail this test. Compositional ecologies should recover function and preserve recognizable organization under dilution and transfer.

19.4.8 Benchmark 7: adaptive continuity under implementation change

A nonstationary protocol layered on top of Benchmarks 1–5 to test whether the system can preserve function while changing implementation. This is the algorithmic analogue of the shock and takeover protocols in the v8 Lane-seep simulation of *Precedence*, *Anti-Precedence*, and *Open-Ended Intelligence*, where the recommended value-relative protected-release condition consistently improved adaptive scores relative to broad biphasic memory and conversion-bridge baselines.

The general protocol:

```
phase 1 (warmup):
    run benchmark normally until an active useful ACS family
    establishes itself
phase 2 (shift):
    perturb the environment so the established implementation
    is partially invalidated, while a neighboring implementation
    becomes accessible:
        - new grammar / semantic predicate (Benchmark 1)
        - concept drift on feature conjunctions (Benchmark 2)
```

- new theorem distribution (Benchmark 3)
- new type signatures or bug families (Benchmark 4)
- new request mix or resource cost (Benchmark 5)

phase 3 (recovery):

measure whether function is preserved while implementation changes

Primary metrics, computed before/during/after the shift:

```
functional_continuity_score
implementation_entropy_given_function
implementation_turnover_score
post_shift_recovery_auc
incumbent_lockin_auc
variant_takeover_under_continued_function
adaptive_regret
protected_release_score
```

Recommended condition contrasts:

SB only	
SB + broad anti-resonance	(old baseline)
SB + family-level anti-precedence	(no protection)
SB + family-level anti-precedence + functional protection	(recommended F'')
SB + family-level anti-precedence + functional protection + structured release	(recommended F''')

The hypothesis under test: F'' and F''' should preserve heldout task score and ACS-utility coupling under the shift better than the broad-anti-resonance baseline, while raising `implementation_entropy_given_function` and `variant_takeover_under_continued_function`, and lowering `incumbent_lockin_auc`.

19.5 Geoteleomic guidance, precedence, and value-relative anti-precedence

19.5.1 From broad anti-resonance to value-relative anti-precedence

The geoteleomic origin-of-life modeling in *Reverse-Engineering the Origin of Life* (Goertzel) and the systems-theoretic anti-precedence analysis of *Precedence, Anti-Precedence, and Open-Ended Intelligence* (Goertzel, 2026) together suggest a layered control principle for algorithmic chemistry. We adopt their terminology of *precedence* and *anti-precedence*, and use “resonance / anti-resonance” or “morphic resonance” only as historical asides.

The basic intuition has three layers:

1. **Geoteleomic guidance.** Useful viable-future endpoints (active useful ACSs, transferable artifact families, replicating program ecologies) define a Schrödinger-Bridge / Doob-transform tilt $\Delta\psi_r$ on rule-selection probabilities. This is where viable futures live.
2. **Precedence.** A rule, family, or pathway that has recently been productive should be slightly easier to traverse again. This is what makes individuation possible: a recurring motif acquires stability, recognizability, and coherence. Use traces $h_r(t)$ drive a positive precedence multiplier.

3. **Value-relative anti-precedence.** A pathway, rule family, ACS, or workflow is penalized *only* when its dominance exceeds its contribution to viable future transformability, with shared functional machinery explicitly protected and pressure redirected into nearby implementations. This is what prevents successful individuation from collapsing into a brittle high-utility circuit.

Earlier drafts of this document used a broad overuse penalty $\xi \Phi(h_r(t); \theta)$ on individual rules. That mechanism is too blunt: it penalizes *use* as such, and tends to damage validators, repair rules, abstraction tools, and other shared functional machinery exactly because they are heavily reused. We retain the broad-overuse form as a simple baseline (kept in §16 and Appendix T for ablation), but the recommended mechanism in this document is value-relative, family-level, function-protected anti-precedence.

19.5.2 Pattern families

Anti-precedence operates at the level of *pattern families*, not individual rules. A family $g \in \mathcal{G}$ groups together rules, ACSs, workflows, or meta-guard clusters that share an implementation identity. The architecture already supports this through **rule-family**, **rule-neighbor**, ACS macro-atoms, and the rule-space graph. We make families first-class objects:

- families have leaky usage traces $U_g(t)$;
- realized share $S_g(t) = U_g(t) / (\sum_h U_h(t) + \epsilon)$;
- value-justified share $S_g^*(t)$ derived from viability-weighted transformability;
- excess-canalization signal $O_g(t)$;
- neighbor-release stimulus $R_g(t)$.

Concrete schema atoms are listed in the appendices (**pattern-family** G17, **family-share** G17 0.34, **family-value** G17 0.21, **family-excess-canalization** G17 0.11, etc.).

19.5.3 Excess canalization

Realized family use:

$$U_g(t) = \int_0^t e^{-\lambda_U(t-s)} dN_g(s), \quad S_g(t) = \frac{U_g(t)}{\sum_h U_h(t) + \epsilon}.$$

Value-justified share, given a value field $V_g(t)$ on families:

$$S_g^*(t) = \frac{\exp(\beta V_g(t))}{\sum_h \exp(\beta V_h(t))}.$$

Excess canalization (the core anti-precedence signal):

$$O_g(t) = [S_g(t) - S_g^*(t) - \theta_O]_+.$$

A family is penalized only when its realized dominance exceeds its value-justified dominance by more than a tolerance θ_O . A successful family that genuinely supports viable, transferable, generative function is allowed to dominate: it is not penalized for being common, but only for being *disproportionately* common relative to what it contributes.

19.5.4 What “value” means here

Value $V_g(t)$ for a pattern family is not external task reward alone. Following the open-ended-intelligence reading, it is *viability-weighted transformability*: the contribution of the family to continued organized self-maintenance while keeping alternative viable implementations open. In the diagnostics of §19 this becomes a weighted combination of metrics the benchmark suite already tracks:

$$V_g \approx \frac{w_U \Delta U_g + w_C C_g + w_G G_g + w_H H_g + w_R R_g^{\text{res}}}{J_g + \text{Cost}_g + \epsilon} - w_K K_g - w_B B_g.$$

The factors are: ΔU_g external utility contribution; C_g ACS–utility coupling by ablation; G_g generativity (novel valid artifacts); H_g heredity / serial-transfer stability; R_g^{res} resilience to perturbation or task shift; J_g recent flux; K_g runtime cost / structural complexity; B_g brittleness or lock-in indicator. The benchmark suite already produces all of these (§19); the change here is that they become *inputs to the anti-precedence controller*, not just evaluation outputs.

19.5.5 Functional protection

Anti-precedence must not damage shared functional machinery. Each rule r carries a functional-protection coefficient

$$c_r(t) \in [0, 1]$$

measuring the degree to which r contributes to function rather than merely to implementation identity. A practical surrogate is ablation-tested utility per unit flux, normalized:

$$c_r(t) \approx \text{clip}\left(\frac{\Delta U_r(t)}{(J_r(t) + \epsilon) c_0}, 0, 1\right).$$

High c_r rules include validators reused across many artifact families, proof-checking lemmas, repair rules, meta-guards that improve heldout prediction, and abstraction constructors supporting multiple ACSs. Low c_r rules are mostly implementation-specific: one specialized workflow, one task-specific hack, one brittle policy rule, one overused branch of a proof strategy.

Anti-precedence then acts mainly on the implementation-specific component, $(1 - c_r)O_{\pi(r)}$.

19.5.6 Structured release

Instead of merely shrinking an overused family, redirect probability mass and attention into nearby alternatives. Using the same neighbor graph that the Wasserstein natural-gradient module already maintains (§13), define a similarity kernel $K(g, h)$ between families, e.g.

$$K(g, h) \propto \exp(-d(g, h)/\tau),$$

where $d(g, h)$ blends source edit distance, gCoDD distance, guard-set distance, behavioral distance, and cost. The release stimulus to family h is

$$R_h(t) = \sum_{g \in \mathcal{G}} K(g, h) O_g(t).$$

19.5.7 The recommended effective rule selection

Combining these pieces, the recommended effective rule-selection probability is

$$\pi_r^{eff}(t) \propto \pi_r^{PC}(t) \exp\left(\Delta\psi_r(t) + \eta_{id} h_r(t) + \eta_{func} c_r(t) H^{func}(t) - \xi(1 - c_r(t)) O_{\pi(r)}(t)^p + \rho R_{\pi(r)}(t)\right),$$

with components:

$$\begin{aligned} \pi_r^{PC}(t) &= \text{native ActPC rule-selection probability,} \\ \Delta\psi_r(t) &= \text{SB / geoteleomic potential increment,} \\ h_r(t) &= \text{identity precedence: recent use of } r, \\ H^{func}(t) &= \text{functional-memory trace,} \\ c_r(t) &\in [0, 1]: \text{functional-protection coefficient,} \\ O_{\pi(r)}(t) &= \text{excess canalization of } r\text{'s family,} \\ R_{\pi(r)}(t) &= \text{neighbor-release stimulus into } \pi(r), \\ \eta_{id}, \eta_{func}, \xi, \rho &= \text{strength parameters.} \end{aligned}$$

In plain language: stabilize working motifs, preserve functional machinery, penalize over-dominant implementations, and redirect pressure into nearby alternatives rather than random novelty.

19.5.8 Anti-precedence on rule-space transport

The same idea applies at the metric level. The Wasserstein natural-gradient module of §13 can use a history-sensitive ground cost

$$\omega_{ij}^{eff}(t) = \omega_{ij} \exp\left(-\eta \bar{h}_{ij}(t) + \xi(1 - \bar{c}_{ij}(t)) O_{\pi(i)}(t)^p - \rho R_{\pi(j)}(t) - \lambda_{SB} \Delta\psi_{ij}(t)\right),$$

where $\bar{c}_{ij}$ is an average functional-protection coefficient on the edge. This makes transport away from over-entrenched families and toward close alternatives cheaper, while functional machinery stays inexpensive to use and SB guidance cheapens directions toward viable endpoints. None of this is random noise: it is a value-relative information-geometric anti-overfitting force.

19.5.9 The simple-baseline form

For ablation and minimal-implementation use, the older broad-anti-resonance variant

$$\pi_r^{eff}(t) \propto \pi_r^{PC}(t) \exp(\Delta\psi_r(t) + \eta h_r(t) - \xi \Phi(h_r(t); \theta)), \quad \Phi(h; \theta) = [h - \theta]_+^2,$$

is retained as a baseline condition. The benchmark suite below explicitly compares it against the family-level, function-protected variant.

19.6 Experimental matrix

For each benchmark, run at least the following conditions. The first group varies the geoteleomic / precedence / anti-precedence axis, the second varies the attention mechanism, and the third combines them with the recommended value-relative anti-precedence rule. Cross axes for the most informative ablation. The naming convention here uses “precedence” and “anti-precedence” as the primary terminology, with “resonance / anti-resonance” kept only as a historical alias for the broad-baseline variants.

Condition	Description
A	ACTPC-CHEM only (static top- k retrieval, no ECAN)
B	ACTPC-CHEM + external reward
C	ACTPC-CHEM + external reward + ACS archive criterion
D	ACTPC-CHEM + SB / geoteleomic guidance
E	ACTPC-CHEM + SB + precedence (positive reuse bias)
F	ACTPC-CHEM + SB + precedence + broad biphasic anti-resonance (old baseline)
G	ACTPC-CHEM + SB + broad anti-resonance only (old baseline)
F'	ACTPC-CHEM + SB + family-level <i>value-relative</i> anti-precedence (no protection)
F''	ACTPC-CHEM + SB + family-level value-relative anti-precedence + functional protection (recommended)
F'''	ACTPC-CHEM + SB + family-level value-relative anti-precedence + functional protection + structured release
H	ACTPC-CHEM + ECAN-lite pool manager
I	ACTPC-CHEM + ECAN-lite + ACS dividends
J	ACTPC-CHEM + ECAN-lite + ACS dividends + broad-overuse rent (old baseline)
J'	ACTPC-CHEM + ECAN-lite + ACS dividends + value-relative excess-canalization rent + functional protection
K	ACTPC-CHEM + info-geometric ECAN on warm/hot subgraph
L	ACTPC-CHEM + info-geometric ECAN + SB / geoteleomic guidance
M	ACTPC-CHEM + info-geometric ECAN + protected anti-precedence

The key paired comparisons we expect to be most informative are:

- F'' vs. F (value-relative anti-precedence vs. broad biphasic): does targeting excess canalization rather than use as such avoid the loss of useful function?
- F''' vs. F'': does structured release into nearby implementations strictly improve over plain functional protection?
- J' vs. J: same question, but inside the ECAN attention economy rather than directly on rule selection.
- M vs. L: does protected anti-precedence add value on top of info-geometric ECAN with SB guidance?

This matrix tests several distinct hypotheses: whether SB / geoteleomic guidance helps ignition of useful ACSs; whether positive precedence alone creates habit traps; whether broad anti-resonance damages functional machinery while suppressing them; whether *value-relative* anti-precedence with functional protection releases over-entrenched implementations without sacrificing transferable function; whether ECAN-managed pool migration improves hot-pool precision and ACS formation over static top- k ; whether ACS dividends and rare-tail reserves bias the system toward useful generativity rather than rigid task circuits; and whether information-geometric updates over the warm/hot attention subgraph provide an additional benefit at acceptable cost.

19.7 Minimum practical suite and recommended first benchmarks

The shortest useful suite is:

1. Pure ACS ignition / habit-trap calibration.
2. Productive grammar ecology.

3. Autocatalytic abstraction discovery.
4. Lemma/proof ecology.
5. Program repair ecology.
6. Serial-transfer and ablation protocol applied to items 2–5.

This gives the ramp:

self-maintenance → useful generative production → abstraction
 → symbolic reasoning → program/tool generation
 → heritable useful repertoires.

The first concrete benchmarks to start with should be productive grammar ecology and lemma/proof ecology. They are small enough to instrument thoroughly, symbolic enough to fit METTA/MORK naturally, generative enough to avoid ordinary ML framing, and rich enough for real autocatalytic networks to matter.

For productive grammar ecology, a workable first concrete reward setup:

Input food:

atoms: A, B, C
 constructors: pair, list, apply, lambda-ish wrapper
 primitive validators
 primitive mutation/rewrite operators

Task stream:

each episode samples a small grammar or type constraint

Reward:

+1 for valid artifact
 +bonus for novelty
 +bonus for satisfying semantic predicate
 +bonus if artifact later helps produce another valid artifact
 -penalty for duplicates, trivial forms, huge forms

Autocatalysis:

artifact motifs can catalyze rewrite rules;
 validators can catalyze repair rules;
 repair rules can produce more valid motifs;
 motifs can become meta-guards for generators.

19.8 Anti-cheat and anti-rigidity rules

To prevent the soup from becoming a rigid circuit:

1. Use task distributions, not single tasks.
2. Reward families of artifacts, not exact outputs.
3. Require heldout generalization.
4. Require active ACS detection from event logs.

5. Require ACS–utility coupling by ablation.
6. Use serial transfer and dilution tests.
7. Track rare-tail productivity.
8. Add *value-relative anti-precedence* against over-entrenched implementation families, with functional protection for shared machinery and structured release into neighboring implementations (§19.5). The blunt broad-anti-resonance form is retained only as an ablation baseline, because it tends to penalize the very functional machinery the system is supposed to preserve.
9. Penalize single-point fragility, not concentration alone.
10. Require *functional continuity under implementation change* across nonstationary task variants: when the environment shifts, the system should preserve transferable function while letting the implementation evolve.

The last two points are important. A successful ecology may have a concentrated head of common motifs plus a long tail of rarer motifs. That is not a failure. It becomes a failure only when the head is brittle, the tail no longer produces useful novelty, or the head’s dominance is no longer matched by its contribution to viable future transformability.

20 Implementation and scaling plan

20.1 Minimal initial implementation

The first working implementation can omit gCoDD, CENF, PLN, AIRIS, and OT. It should implement:

- atom schemas;
- deterministic MM2 candidate matching;
- host-side softmax sampler;
- host-side environment step for grid bug;
- KL or cross-entropy over outcome classes;
- trace and online logit update;
- cleanup and debugging snapshots.

20.2 PeTTA wrapper runtime as the first worker backend

The first scalable implementation should not wait for METTA-IL. After the single-chamber prototype works, wrap the chamber as a PeTTA/SWI-Prolog worker while keeping all worker metadata as MORK facts. The PeTTA runtime is then a temporary backend for the portable Worker IR.

A worker has the following backend-neutral contract:

```

input:
    worker id
    worker kind
    backend tag: petta or metta-il
    context / deme / window / bridge problem
    snapshot id
    hot pool or work item
    budget
    random seed

execution:
    one bounded local computation:
        MM2 chamber tick sequence, or
        structural-evolution batch, or
        ACS scan, or
        ECAN aggregation, or
        bridge fit, or
        replay/ablation batch

output:
    append-only event log
    stimulus deltas
    rule-credit deltas
    candidate rules
    ACS reports
    bridge potentials
    diagnostics

```

Schematic PETTA/SWI-Prolog implementation:

```

run_worker(W) :-
    worker_kind(W, Kind),
    run_worker_by_kind(Kind, W).

run_worker_by_kind(context_chamber, W) :-
    worker_context(W, C),
    worker_snapshot(W, Snap),
    worker_seed(W, Seed),
    restore_snapshot(Snap),
    run_single_threaded_mm2_chamber(C, Seed, Events),
    write_event_log(W, Events).

run_worker_by_kind(moses_deme, W) :- run_moses_deme_batch(W).
run_worker_by_kind(acs_scan, W) :- run_acs_scan(W).
run_worker_by_kind(ecan_epoch, W) :- run_ecan_epoch_reducer(W).
run_worker_by_kind(bridge_fit, W) :- run_bridge_fit(W).

```

The future METTA-IL implementation should execute the same worker facts, not reinterpret the architecture. Migration should therefore be mostly a backend replacement:

```

PeTTa predicate:
    run_worker_by_kind(context_chamber, W)

```

MeTTa-IL worker form:

```
(run-worker W)
```

Shared meaning:

```
read Worker IR, run chamber, write event log
```

The minimum equivalence test is: given the same snapshot, seed, hot pool, and worker configuration, a direct chamber run and a PETTA worker run must produce the same chosen rules, trace records, stimulus markers, and local event ordering. When METTA-IL becomes available, the same test should compare the PETTA and METTA-IL backends modulo backend tags and harmless ID allocation differences.

20.3 Scaling sequence

Do not begin with a giant soup. A sensible sequence is:

Stage 0: mechanical sanity

```
rules:          1k - 5k
hot rules/tick:  32 - 128
candidates/tick: 128
firings/tick:    1 - 4
demes:          0 - 4
ECAN:           none yet (static top-k retrieval)
purpose:         validate phases, traces, cleanup, action output
worker backend:  none or PeTTa with one context-chamber worker;
                 deterministic trace equivalence test
```

Stage 1: online ActPC only

```
rules:          10k - 30k
warm rules:     1k - 3k
hot rules/tick: 128 - 512
candidates/tick: 256 - 1024
firings/tick:   4 - 8
demes:          0 - 8
ECAN:           none yet (static top-k retrieval)
purpose:         validate stochastic selection and credit assignment
worker backend:  PeTTa par-groups with independent context chambers;
                 append-only event logs and reducers
```

Stage 2: ECAN-lite + structural evolution

```
rules:          50k - 150k
warm rules:     5k - 15k
hot rules/tick: 512 - 1024
candidates/tick: 1024 - 2048
firings/tick:   8 - 16
demes:          16 - 32
pop/deme:       128 - 256
ECAN:           STI/LTI, wages, base/cost rent, sparse Hebbian
                 diffusion, pool migration
purpose:         validate MOSES/gCoDD/CENF proposals; show ECAN
                 improves hot-pool precision over static top-k
worker backend:  PeTTa runs context chambers, MOSES demes, and ACS
                 scans as worker kinds; ECAN begins worker scheduling
```

Stage 3: ACS-aware ECAN + serious single-server run

rules:	150k - 500k
warm rules:	15k - 50k
hot rules/tick:	1024 - 2048
candidates/tick:	2048 - 4096
firings/tick:	16 - 32
demes:	32 - 64
pop/deme:	256 - 512
ECAN:	ACS dividends, rare-tail reserves, family-level value-relative excess-canalization rent + functional protection + neighbor release, ACS macro-atoms
purpose:	real chemistry experiment with attention economy; first run of adaptive-continuity benchmark (Bench 7) to compare broad vs protected anti-precedence
worker backend:	PetTa scheduler manages runtime budgets across chambers, demes, scans, replay batches, and bridges
Stage 4: info-geometric ECAN + scale test	
rules:	500k - 2M+
warm rules:	50k - 150k
hot rules/tick:	2048 - 8192
candidates/tick:	4096 - 16384
firings/tick:	32 - 64
demes:	64 - 128
pop/deme:	512 - 1024
ECAN:	mirror-descent + optional Wasserstein on bounded warm/hot subgraph; coupling to ActPC via KL
purpose:	stress and scaling after profiling succeeds
worker backend:	migrate selected worker families from PetTa to MeTTa-IL when available; compare backend traces

21 Failure modes and mitigations

Failure mode	Symptom	Mitigation
Candidate explosion	thousands of candidates per tick	active signatures, top- <i>k</i> caps, rule-family budgets, retired status.
Novelty addiction	seeks surprise without reward	use learning-progress epistemic reward, entropy with small coefficient, goal weighting.
Rule bloat	rules grow without improving behavior	cost penalty, GCoDD node count, trial retirement, CENF only with support.
Duplicate rules	repeated evaluation of equivalent rules	canonical hash, ENF normalization, source-hash lookup before insertion.
Brittle specialization	too many narrow rules	merge rules, meta-guards, PLN generalization, prior pull.

Failure mode	Symptom	Mitigation
Overgeneral abstraction	meta-guard misclassifies cases	confidence threshold, negative examples, allow inlining/splitting.
Delayed credit confusion	wrong trace gets reward	event windows, eligibility decay, causal refinement, multiple-hypothesis traces.
Slow MORK queries	scans over rule soup	ground-prefix schemas, conjunct order, split execs, compiled triggers.
OT instability	logits become extreme or NaN	damping, clipping, small components, fallback to simple update.
Outer-loop overreach	MOSES floods soup with trial rules	insertion budget, quarantine, promotion thresholds, duplicate checks.
Habit traps / sterile loops	autocatalysis without utility, or rigid flux through a few paths	value-relative anti-precedence at family level, rare-tail tracking, sclerosis index, serial-transfer tests.
Decorative ACSs	high A , low C	coupling-by-ablation filter, hard $C \geq C_{\min}$ Pareto criterion.
Attention monopoly	one cluster dominates hot pool; low rare-tail enrichment; high ablation fragility	excess-canalization rent at family level, hot-pool novelty quota, rare-tail reserve, InverseHebbian competition, ablation-sensitive stimulus, structured release into neighboring families.
False anti-precedence release	useful core / functional machinery is penalized merely because it is heavily reused (validators, repair rules, abstraction tools)	functional-protection coefficient c_r ; ablation-tested V_g and c_r ; minimum dwell time on protection status; restore broad-anti-resonance only as ablation baseline.
Family misclustering	unrelated rules grouped into one family; release pressure misdirected; protected status leaks across function boundaries	multi-distance $d(g, h)$ (gCoDD, behavior, guard set, cost); split families when intra-family behavioral distance exceeds threshold; require shared ACS membership or shared meta-guard for strong protection.
Protected monopoly	high c_r shields a brittle incumbent from release	include heldout / serial-transfer evidence in c_r ; add explicit lock-in penalty B_g in family value V_g ; cap c_r when the incumbent has not produced novel valid artifacts for many epochs.

Failure mode	Symptom	Mitigation
Release thrashing	system constantly shifts implementations; mutant share oscillates without stabilizing	slow trace on O_g (low λ_O decay), hysteresis on excess-canalization threshold, minimum dwell time before re-penalizing the same family.
Decorative diversity	many variants, rising mutant share, but no functional continuity	require ACS–utility coupling and serial-transfer stability before crediting variants; track <code>functional_continuity_score</code> .
SB over-constraint on identity	SB / geoteleomic bridge cheapens one specific implementation rather than the function	train bridge on functional summaries (artifact-family validity, ACS membership, heldout outcome) rather than on rule IDs or implementation fingerprints.
Attentional noise	high novelty, low utility, no durable ACSs, hot pool churns excessively	raise rent on low-utility novelty, raise LTI threshold for repeated promotion, strengthen ACTPC-CHEM support term in selection score.
ECAN overhead	ECAN update dominates runtime; Hebbian graph too dense; pool migration thrashes	cap association edges, bound diffusion frontier, run info-geometry periodically only, separate high-frequency ECAN-lite from low-frequency ECAN-geom.
ECAN/ACTPC-CHEM drift	attention and rule probability point in opposite directions for too long	soft KL coupling $KL(p a_R)$ and $KL(a_R p)$, joint diagnostic dashboard, periodic alignment checks.
MM2 doing ECAN work	per-tick latency spikes when ECAN-related <code>execs</code> fire; numeric updates appear in MM2 traces	move arithmetic, sampling, and top- k retrieval into grounded kernels; MM2 should only consume (<code>pool-hot \$C \$R</code>) and emit stimulus markers (§16.12).
PETTA-specific semantics leak	Prototype depends on PETTA-private predicates or Prolog-side mutable state that has no Worker IR equivalent.	Keep architectural state in Worker IR facts, event logs, reducer outputs, and MORK atoms. Treat PETTA predicates as backend adapters only.
PETTA scheduler bottleneck	Hardware threads are available but idle because worker dispatch or reduction is serialized.	Use coarse-grained workers, thread pools or worker processes, bounded message queues, batched reducers, and ECAN worker-level priorities.

Failure mode	Symptom	Mitigation
Shared hot-pool mutation from worker threads	Runs are not replayable; ACS causality depends on backend thread scheduling.	Never let PETTA threads or METTA-IL <code>par</code> branches mutate one hot pool concurrently. Use snapshots, single-threaded chambers, append-only logs, and reducers.
Non-portable worker contracts	Moving from PETTA to METTA-IL requires redesigning workers.	Define workers through backend-neutral facts: worker kind, snapshot, budget, seed, output log, status, and reducer. Generate PETTA predicates and METTA-IL forms from the same facts.
Fine-grained PETTA or <code>par</code> over-parallelism	Millions of tiny backend tasks dominate runtime and drown useful chemistry.	Use worker parallelism only for context chambers, demes, ACS windows, replay batches, ECAN epochs, bridge jobs, and reducers. Fine-grained candidate scoring stays inside chambers or grounded kernels.
Worker-family attention monopoly	One worker family consumes nearly all runtime budget.	Apply ECAN rent and quotas at the worker level; maintain novelty and rare-tail worker quotas; track utility per CPU millisecond.

22 Conclusion

The most practical route to algorithmic chemistry on MORK is to treat the chemistry as a population of small, weighted, compiled rewrite-rule molecules, with an explicit attention economy mediating between that chemistry and the rest of the cognitive system. The online ACTPC-CHEM loop should remain local, indexed, stochastic, and mostly numeric through grounded kernels. The structural loop should be slower and more creative, using MOSES, gCoDD, CENF, AIRIS, and PLN to propose new molecules and abstractions. Sitting between them, ECAN maintains STI/LTI currencies and a sparse Hebbian/InverseHebbian association graph, gathers stimulus from every other component, and decides moment-by-moment which atoms are cold, warm, or hot. ACTPC-CHEM decides which rules predict and act well; ECAN decides which atoms get enough attention and memory to matter.

The execution substrate should follow the same separation. MM2 is not asked to become a global multithreaded symbolic runtime. It is used as the deterministic local chemistry inside one bounded reaction chamber. The parallel worker graph can be prototyped now in PETTA/SWI-Prolog and later ported to METTA-IL/Rholang, provided both backends consume the same Worker IR and produce the same event logs, reducer outputs, and ECAN stimulus records. In slogan form: MM2 supplies ordered local chemistry; PETTA supplies the initial parallel wrapper; METTA-IL supplies the eventual native parallel wrapper.

The most important architectural commitment is to maintain two coupled but distinct distributions — ACTPC-CHEM’s rule probability $p_t(R)$ and ECAN’s attention $a_t(A)$ — and to update

them through their own appropriate machinery. ACTPC-CHEM updates p via prediction error, reward, and (optionally) Wasserstein natural-gradient on a rule graph. ECAN updates a via wages, rent, sparse Hebbian diffusion, pool migration, and (optionally) mirror-descent or Wasserstein steps on a bounded warm/hot subgraph. A soft KL regularizer couples them without collapsing them into a single distribution. This separation is what prevents the system from collapsing into a brittle high-utility circuit and what gives novelty, diagnostic atoms, and rare-tail ACS precursors a place to live.

The recommended hardware is CPU- and RAM-heavy bare metal, not a GPU box; the soup should be partitioned into a large cold reservoir, a smaller warm active set, and a small hot MM2 working set per cognitive tick, with ECAN migrating atoms among them. The machine should be viewed less as a GPU training box and more as a large, fast, memory-rich symbolic chemistry reactor with an explicit circulatory system.

The benchmark suite should certify success not when the soup solves a single task but when the soup evolves active, useful, generative autocatalytic subnetworks: ablation-sensitive, transfer-stable, tail-productive, and resistant to congealing into rigid habit, with ECAN demonstrably improving hot-pool precision and ACS formation over a static top- k baseline. This is the benchmark philosophy most aligned with the strengths of ACTPC-CHEM/MORK algorithmic chemistry, and the implementation plan above is designed to make those structures detectable from the very first stages.

Finally, the design adopts a three-layer control stack for guiding rule selection and attention. *Geoteleomic guidance* (SB / Doob tilt $\Delta\psi_r$) supplies a least-surprise pull toward viable futures. *Precedence* ($\eta_{\text{id}}h_r$ plus a functionally-protected memory term $\eta_{\text{func}}c_rH^{\text{func}}$) lets recurring useful motifs individuate and become recognizable. *Value-relative anti-precedence* ($-\xi(1-c_r)O_{\pi(r)}^p + \rho R_{\pi(r)}$) prevents successful individuation from collapsing into brittle high-utility circuits, by penalizing implementation families only when their realized share exceeds their viability-justified share, while protecting shared functional machinery and redirecting pressure into nearby implementations. The earlier broad-anti-resonance form is retained as an ablation baseline, but the recommended setting throughout is family-level, value-relative, function-protected anti-precedence: this is the design correction adopted from the v8 Lane-seep simulation results, in which protected release outperformed both broad biphasic memory and a conversion-bridge alone in shock and takeover protocols. Slogan form: *ActPC-Chem decides which rules predict and act well; ECAN decides which atoms get enough attention and memory to matter; protected anti-precedence decides when a successful implementation has become too dominant relative to the future transformability it enables.*

A Atom schemas

A.1 General atom design rules

All hot-path atoms should follow these rules:

1. The first symbol is a relation name, such as `obs`, `rule`, `trigger`, or `cand`.
2. The second and third positions should usually be grounded keys, such as context id, signature id, rule id, tick id, or phase id.
3. Variables should appear toward the suffix of an expression.
4. Avoid deep nested expressions on hot paths. Use multiple shallow facts.
5. Keep source rules and compiled rules separate.

6. Store enough metadata to garbage collect transient atoms deterministically.

The chemistry framing makes it tempting to write a single big atom per “molecule.” Resist this. The hot path needs many small atoms whose front prefixes are selective; one giant nested atom defeats prefix indexing.

A.2 Identifier conventions

C<n>	context id
T<n>	tick id
R<n>	rule id
A<n>	action id
E<n>	trace/event id
O<n>	object id
MG<n>	meta-guard id
G<n>	guard id
N<n>	gCoDD node id
D<n>	MOSES deme id
S<n>	signature id
P<n>	pattern id
W<n>	worker id
CH<n>	chamber id
PG<n>	par-group id
SNAP<n>	snapshot id
LOG<n>	event-log id
RED<n>	reducer id
SH<n>	shard id
WT<n>	worker-template id
WF<n>	worker-family id

IDs should be short symbols or interned hashes. A host-side allocator should map long human names to short stable symbols.

A.3 Persistent fact schemas

A.3.1 Context and tick facts

```
(ctx C42)
(ctx-agent C42 Bug7)
(ctx-episode C42 EP9)
(ctx-status C42 active)
(tick C42 T193)
(phase C42 sense)
(goal C42 get-food)
(goal-weight C42 get-food 1.0)
```

A.3.2 Portable worker IR facts

These facts describe backend-neutral workers. They can be executed by the near-term PETTA wrapper or by the future METTA-IL backend. The backend tag states which runtime should currently execute the worker; it is not part of the worker’s scientific semantics.

```

(worker W17)
(worker-kind W17 context-chamber) ; context-chamber, moses-deme,
                                   ; acs-scan, ecan-epoch,
                                   ; bridge-fit, replay-batch,
                                   ; airis-job, pln-job, reducer
(worker-backend W17 petta)         ; petta, metta-il, host
(worker-status W17 ready)          ; ready, running, done, failed, retired

(worker-context W17 C42)
(worker-chamber W17 CH9)
(worker-snapshot W17 SNAP17)
(worker-hot-pool W17 HP42)
(worker-output-log W17 LOG99)
(worker-budget W17 cpu-ms 50)
(worker-budget W17 memory-mb 512)
(worker-seed W17 12345)
(worker-priority W17 0.71)

(chamber CH9)
(chamber-context CH9 C42)
(chamber-status CH9 active)
(chamber-mm2-mode CH9 single-threaded)
(chamber-current-tick CH9 T193)
(chamber-event-log CH9 LOG99)

(snapshot SNAP17)
(snapshot-context SNAP17 C42)
(snapshot-source-tick SNAP17 T192)
(snapshot-kind SNAP17 context-local)

(event-log LOG99)
(event-log-worker LOG99 W17)
(event-log-context LOG99 C42)
(event-log-start-tick LOG99 T193)
(event-log-status LOG99 open)

(par-group PG3)
(par-backend PG3 petta)
(par-member PG3 W17)
(par-member PG3 W18)
(par-member PG3 W19)
(par-status PG3 ready)

(reducer RED5)
(reducer-kind RED5 ecan-stimulus)
(reducer-input-log RED5 LOG99)
(reducer-input-log RED5 LOG100)
(reducer-output RED5 STIMBATCH17)
(reducer-status RED5 ready)

(worker-template WT_context_chamber)
(worker-template-kind WT_context_chamber context-chamber)
(worker-family WF_context_chambers)

```

```
(worker-family-member WF_context_chambers W17)
```

The PETTA backend reads these facts and dispatches Prolog predicates. The future METTA-IL backend should read the same facts and dispatch `par` workers. The facts, not the backend code, define the portable worker contract.

A.3.3 Observation and feature facts

Use relation-first, context-second, signature-third ordering.

```
(obs C42 see-food 09 2 3)
(obs C42 see-poison 05 1 4)
(obs C42 energy low)
(obs C42 holding empty)

(feat C42 09 color-red)
(feat C42 09 shape-round)
(feat C42 09 texture-smooth)

(active-sig C42 see-food)
(active-sig C42 color-red)
(active-sig C42 shape-round)
```

The exact signature design is domain-specific. The key requirement is that a small set of `active-sig` facts can narrow candidate rules.

A.3.4 Rule metadata

```
(rule R17)
(rule-kind R17 act)           ; act, predict, classify, causal, meta, gcodd
(rule-status R17 active)      ; active, trial, retired, quarantined
(rule-sig R17 see-food)
(rule-layer R17 lower)
(rule-owner R17 hand-coded)   ; hand-coded, actpc, moses, airis, pln
(rule-cost R17 3.0)
(rule-logit R17 0.41)
(rule-prior-logit R17 0.00)
(rule-use-count R17 842)
(rule-success-count R17 522)
(rule-last-fired R17 T192)
(rule-version R17 4)
```

A.3.5 Compiled rule facts

```
(trigger see-food R17)
(rule-guard R17 sees-item)
(rule-guard R17 item-is-foodlike)
(template R17 act-grab)
(template-arg R17 target obj)
(template-arg R17 x x)
(template-arg R17 y y)
```

For very small prototypes, `template` can be a direct action symbol. For larger systems, `template` should point to a compiled template object.

A.3.6 Rule source facts

Do not use these for hot-path matching. They are for inspection, rewriting, and compilation.

```
(rule-src R17
  (= > (and (obs $C see-food $Obj $X $Y)
            (goal $C get-food))
      (act $C grab $Obj)))

(rule-gcodd-root R17 N991)
(rule-src-hash R17 Habc123)
```

A.3.7 Candidate, chosen, and action facts

These are transient and scoped by context/tick.

```
(cand C42 T193 R17 09 2 3)
(cand-score C42 T193 R17 0.72)
(cand-logit C42 T193 R17 0.41)

(chosen C42 T193 R17 09 2 3)
(action C42 T193 A88 grab 09)
(action-status C42 T193 A88 pending)
```

A.3.8 Prediction and outcome facts

```
(pred C42 T193 outcome got-food 0.73)
(pred C42 T193 outcome no-change 0.10)
(pred C42 T193 outcome got-poison -0.05)

(outcome C42 T193 got-food)
(reward C42 T193 env 1.0)
(reward C42 T193 epistemic 0.14)
(reward C42 T193 instrumental 0.86)
(error C42 T193 kl 0.22)
(error C42 T193 xent 0.31)
```

A.3.9 Trace and credit facts

```
(trace E88 C42 T193 R17 A88)
(trace-bind E88 obj 09)
(trace-bind E88 x 2)
(trace-bind E88 y 3)
(trace-status E88 open)
(trace-age E88 0)
(trace-discount E88 1.0)

(credit E88 R17 immediate 0.82)
```

```
(credit E88 R17 delayed 0.15)
(credit-total R17 C42 T193 0.97)
```

A.3.10 Rule-space graph facts

These support Wasserstein updates, structural mutation, and nearest-neighbor exploration.

```
(rule-neighbor R17 R22 0.08)
(rule-neighbor R17 R35 0.21)
(rule-distance R17 R22 edit 2.0)
(rule-distance R17 R22 behavior 0.17)
(rule-family R17 food-grab)
(rule-parent R22 R17 specialize)
```

A.3.11 Pattern-family facts (*anti-precedence*)

These reify pattern families as first-class objects so that the ECAN excess-canalization machinery (§19.5) can operate on them. A pattern family groups rules, ACSs, workflows, or meta-guard clusters that share an implementation identity.

```
(pattern-family G17)
(pattern-family-kind G17 rule-family) ; rule-family, acs, workflow,
                                         ; meta-guard-cluster

(pattern-family-member G17 R17)
(pattern-family-member G17 R22)
(pattern-family-parent G18 G17 mutate)
(pattern-family-neighbor G17 G18 0.12)
(pattern-family-neighbor G17 G44 0.31)

; Family-level traces and shares
(family-use-trace G17 3.82) ; U_g(t)
(family-share G17 0.34) ; S_g(t)
(family-value G17 0.21) ; V_g(t), viability-weighted
(family-value-share G17 0.18) ; S*_g(t) from softmax(beta V)
(family-excess-canalization G17 0.11) ; O_g(t)

; Neighbor release into nearby families
(family-release-neighbor G17 G18 0.07)
(family-release-neighbor G17 G44 0.03)
(family-release-into G18 0.07) ; aggregate R_h received

; Functional-protection annotation per rule
(functional-protection R17 0.82) ; c_r in [0,1]
(functional-role R17 validator) ; validator, repair-rule,
                                ; generator, abstraction,
                                ; meta-guard, samplers,
                                ; implementation-specific

(functional-role R22 repair-rule)
(functional-role R35 generator)
(functional-role R44 implementation-specific)

; Lock-in indicator (used in V_g)
(family-lockin G17 0.27) ; B_g
```

When the ECAN controller detects a family with high V_g and high O_g , it writes structural-request atoms instead of merely applying rent:

```
(structural-request C42 T193
  over-entrenched-family G17)
(structural-request C42 T193
  preserve-function-vary-implementation G17)
```

These are consumed by MOSES/gCoDD/CENF in the next outer-loop epoch.

A.3.12 Meta-guard and abstraction facts

```
(meta-guard MG12)
(meta-guard-status MG12 active)
(meta-guard-member MG12 color-red)
(meta-guard-member MG12 shape-round)
(meta-guard-member MG12 texture-smooth)
(meta-guard-support MG12 183)
(meta-guard-confidence MG12 0.78)

(guard-has C42 09 MG12)
(rule-guard R44 MG12)
```

A.3.13 gCoDD and MOSES facts

```
(gcodd-node N991 and)
(gcodd-edge N991 N992 0)
(gcodd-edge N991 N993 1)
(gcodd-leaf N993 grounded-predicate-is-foodlike)
(gcodd-hash N991 Hdef456)

(deme D4)
(deme-status D4 active)
(deme-root-rule D4 R17)
(deme-exemplar D4 R44)
(deme-fitness D4 R44 0.91)
(deme-model D4 FG8)
```

B Rule representation

B.1 Three-level representation

Every nontrivial rule should exist in three representations:

1. **Source rule:** readable METTA or METTA-like expression.
2. **Canonical program:** gCoDD or another canonical DAG used by MOSES, hashing, deduplication, and structural evolution.
3. **Compiled hot-path atoms:** small indexed facts used by MM2.

```

Human-readable source
    |
    | compile_rule_source_to_gcodd
    v
Canonical gCoDD / CENF representation
    |
    | compile_gcodd_to_mm2_atoms
    v
Compiled trigger/guard/template facts in MORK

```

B.2 Example source rule

```

(rule-src R17
  (=> (and (obs $C see-food $Obj $X $Y)
            (goal $C get-food)
            (not (obs $C holding $Something))))
    (act $C grab $Obj)))

```

B.3 Example compiled representation

```

(rule R17)
(rule-kind R17 act)
(rule-status R17 active)
(rule-sig R17 see-food)
(trigger see-food R17)
(rule-guard R17 goal-get-food)
(rule-guard R17 hand-empty)
(template R17 act-grab)
(template-arg R17 target obj)

```

B.4 Why not match directly on source rules?

Source rules can be large, nested, and variable-rich. That is useful for humans and structural evolution, but bad for MORK hot-path performance. The compiled representation makes the first few bytes of each query selective:

```

(obs C42 see-food ...)
(trigger see-food R17)
(rule-status R17 active)

```

This avoids the common failure mode of scanning every rule for every observation.

C Inner loop: MM2 phase architecture

C.1 Phase overview

A single tick consists of these phases:

```
00 init-tick
10 sense
20 derive-features
30 activate-signatures
40 match-candidates
50 score-candidates
60 sample-or-select
70 fire-action-or-prediction
80 environment-step
90 observe-outcome
100 compute-error
110 assign-credit
120 update-rule-weights
130 structural-trigger
140 cleanup
```

Not every phase must be pure MM2. Some phases are MM2 rewrites; others are sink or source calls. The important point is that phases are explicit, local, and interruptible.

C.2 Phase atoms

```
(todo C42 T193 sense)
(done C42 T193 sense)
(todo C42 T193 match)
(done C42 T193 match)
```

C.3 Pseudo-MM2 notation

The following snippets are pseudo-MM2. They are meant to specify compiler output and scheduler behavior. Exact syntax may need adjustment to the current MORK implementation.

C.4 Chamber contract: MM2 is local and single-threaded

The following pseudo-MM2 phase rules describe the inside of one context chamber. A chamber executes these phases single-threadedly over one context-local hot pool. The output is an ordered event log. This event order is used by trace attribution, delayed credit, replay, ACS/RAF detection, and ablation.

Parallelism is outside the chamber. In the first scalable prototype, PETTA may run many chambers and auxiliary workers concurrently. Later, METTA-IL should express the same concurrent worker batch using Rholang-style `par`. Neither backend should run many MM2 rewrites against the same mutable hot pool.

The chamber input contract is:

```
context id
tick id or tick allocator
context snapshot
pool-hot facts for this context
budget
random seed
phase todo atoms
```


The chamber output contract is:

```
ordered event log
chosen actions
traces and trace bindings
prediction and error facts
credit totals
stimulus-event markers
structural-request atoms
cleanup summary
```

C.5 Phase 00: initialize tick

```
(exec (00 init)
  (, (ctx $C)
    (ctx-status $C active)
    (next-tick $C $T))
  (, (tick $C $T)
    (todo $C $T sense)
    (todo $C $T derive-features)
    (todo $C $T activate-signatures)))
```

C.6 Phase 10: ingest observations

This phase will often be a grounded source or sink because observations come from the environment, a replay buffer, or a continuous predictive-coding module.

```
(exec (10 sense-request)
  (, (todo $C $T sense))
  (0 (+ (sense-request $C $T))))
```

The host responds by adding facts such as:

```
(obs C42 see-food 09 2 3)
(obs C42 energy low)
(done C42 T193 sense)
```

C.7 Phase 20: derive features

```
(exec (20 derive-color-red)
  (, (todo $C $T derive-features)
    (obs $C see-item $Obj $X $Y)
    (raw-feature $C $Obj rgb-redish))
  (, (feat $C $Obj color-red)))

(exec (20 derive-foodlike-mg)
  (, (todo $C $T derive-features)
    (feat $C $Obj color-red)
    (feat $C $Obj shape-round)
    (feat $C $Obj texture-smooth)
    (meta-guard-status MG12 active))
  (, (guard-has $C $Obj MG12)))
```

C.8 Phase 30: activate signatures

```
(exec (30 active-see-food)
  (, (todo $C $T activate-signatures)
    (obs $C see-food $Obj $X $Y))
  (, (active-sig $C see-food)))

(exec (30 active-energy-low)
  (, (todo $C $T activate-signatures)
    (obs $C energy low))
  (, (active-sig $C energy-low)))
```

When signatures are activated, finish the phase:

```
(exec (31 finish-activate)
  (, (todo $C $T activate-signatures))
  (0 (- (todo $C $T activate-signatures))
    (+ (done $C $T activate-signatures))
    (+ (todo $C $T match))))
```

C.9 Phase 40: match candidates

The most selective fact should appear first. Usually this is a context-local observation or active signature.

```
(exec (40 match-see-food)
  (, (todo $C $T match)
    (active-sig $C see-food)
    (obs $C see-food $Obj $X $Y)
    (pool-hot $C $R)
    (trigger see-food $R)
    (rule-status $R active))
  (, (cand $C $T $R $Obj $X $Y)))
```

The `pool-hot` fact is the chamber gate. It is written before the chamber runs, by ECAN or by a reducer. PETTA and METTA-IL backends both treat it as part of the worker input snapshot. MM2 consumes it but does not compute it.

Bad pattern ordering:

```
; Avoid: scans all triggers before considering current context.
(exec (bad)
  (, (trigger $Sig $R)
    (obs $C $Sig $Obj $X $Y))
  (, (cand $C $T $R $Obj $X $Y)))
```

C.10 Phase 50: score candidates

Candidate scoring may be partially symbolic and partially grounded. The MM2 part attaches obvious metadata; the host calculates final scores.

```
(exec (50 score-request)
  (, (todo $C $T score)
    (cand $C $T $R $Obj $X $Y))
```

```
(rule-logit $R $L)
(rule-cost $R $Cost))
(, (score-input $C $T $R $L $Cost $Obj $X $Y)))
```

Host-side SCORE_CANDIDATES consumes `score-input` facts and emits:

```
(cand-score C42 T193 R17 0.72)
(cand-logit C42 T193 R17 0.41)
(done C42 T193 score)
(todo C42 T193 sample)
```

C.11 Phase 60: sample or select

Weighted random choice should be host-side.

```
(exec (60 sample-request)
(, (todo $C $T sample))
(0 (+ (sample-request $C $T))))
```

The host adds one or more chosen applications:

```
(chosen C42 T193 R17 09 2 3)
(done C42 T193 sample)
(todo C42 T193 fire)
```

C.12 Phase 70: fire action and trace

```
(exec (70 fire-request)
(, (todo $C $T fire)
(chosen $C $T $R $Obj $X $Y))
(0 (+ (fire-request $C $T $R $Obj $X $Y))))
```

Host response:

```
(action C42 T193 A88 grab 09)
(trace E88 C42 T193 R17 A88)
(trace-bind E88 obj 09)
(trace-bind E88 x 2)
(trace-bind E88 y 3)
(done C42 T193 fire)
(todo C42 T193 env-step)
```

C.13 Phase 80: environment step

```
(exec (80 env-step-request)
(, (todo $C $T env-step)
(action $C $T $A $Kind $Arg))
(0 (+ (env-step-request $C $T $A $Kind $Arg))))
```

Host response:

```
(outcome C42 T193 got-food)
(reward C42 T193 env 1.0)
(done C42 T193 env-step)
(todo C42 T193 compute-error)
```

C.14 Phase 100: compute error

```
(exec (100 error-request)
      (, (todo $C $T compute-error))
      (0 (+ (error-request $C $T))))
```

Host response:

```
(pred C42 T193 outcome got-food 0.73)
(error C42 T193 kl 0.22)
(error C42 T193 xent 0.31)
(reward C42 T193 epistemic 0.14)
(reward C42 T193 instrumental 0.86)
(done C42 T193 compute-error)
(todo C42 T193 assign-credit)
```

C.15 Phase 110: assign credit

Credit should use immediate reward, prediction error, eligibility traces, and possibly delayed outcomes.

```
(exec (110 credit-request)
      (, (todo $C $T assign-credit))
      (0 (+ (credit-request $C $T))))
```

Host response:

```
(credit E88 R17 immediate 0.82)
(credit-total R17 C42 T193 0.82)
(done C42 T193 assign-credit)
(todo C42 T193 update-weights)
```

C.16 Phase 120: update rule weights

```
(exec (120 update-request)
      (, (todo $C $T update-weights))
      (0 (+ (update-request $C $T))))
```

Host response:

```
(rule-logit R17 0.46)           ; after replacing old value
(rule-use-count R17 843)
(done C42 T193 update-weights)
(todo C42 T193 structural-trigger)
```

C.17 Phase 130: trigger structural evolution

This phase determines whether the outer loop should run for this context, episode, rule family, or deme.

```
(exec (130 structural-trigger-high-error)
  (, (todo $C $T structural-trigger)
    (error $C $T kl $E)
    (gt $E 0.50))
  (, (structural-request $C $T high-error)))

(exec (130 structural-trigger-high-credit)
  (, (todo $C $T structural-trigger)
    (credit-total $R $C $T $Cr)
    (gt $Cr 0.90))
  (, (structural-request $C $T high-credit-rule $R)))
```

The `gt` relation is best implemented as a source predicate or a compiled comparison support fact.

C.18 Phase 140: cleanup

Cleanup deletes transient atoms. Do not delete traces until they are closed or copied into a replay store.

```
(exec (140 cleanup-cand)
  (, (todo $C $T cleanup)
    (cand $C $T $R $Obj $X $Y))
  (0 (- (cand $C $T $R $Obj $X $Y))))

(exec (140 cleanup-chosen)
  (, (todo $C $T cleanup)
    (chosen $C $T $R $Obj $X $Y))
  (0 (- (chosen $C $T $R $Obj $X $Y))))
```

D Grounded kernel interfaces

D.1 Why grounded kernels are required

Pure graph rewriting is not the right substrate for all operations. The following belong in host-side kernels:

- stochastic sampling and random-number generation;
- numerical comparison, softmax, KL, cross-entropy, entropy;
- vector and sparse-matrix operations;
- Wasserstein natural-gradient or optimal-transport solves;
- canonical hashing and hash-consing;
- factor-graph inference and sampling;

- environment stepping and sensor ingestion;
- bulk garbage collection and compaction.

D.2 Kernel API table

Kernel	Input atoms	Output atoms
SENSE	sense-request	obs, raw-feature, done
SCORE_CANDIDATES	score-input, rule metadata	cand-score, cand-logit
SOFTMAX_SAMPLE	cand-score, temperature	chosen
FIRE_TEMPLATE	chosen, template facts	action, trace, trace-bind
ENV_STEP	action	outcome, reward, next observations
COUNT_PATTERNS	pred, outcome, traces	histograms or distribution atoms
KL_XENT	predicted and observed distributions	error, reward epistemic
ASSIGN_CREDIT	traces, rewards, errors	credit, credit-total
UPDATE_LOGITS	credits, logits, costs	updated rule-logit facts
CANON_HASH	rule source or GCODD graph	rule-src-hash, gcodd-hash
RULE_DISTANCE	rule pairs or fragments	rule-neighbor, rule-distance
OT_STEP	rule graph, logits, gradients	updated logits or transport plan
CENF_CLUSTER	guard counts, fitness	meta-guard facts
FG_SAMPLE	factor graph model	sampled knobs/rule fragments
ECAN_GATHER_STIMULUS	events, traces, credits, ACS report, external reward	per-atom stimulus totals
ECAN_PAY_WAGES	stimulus, current STI/LTI	updated sti, lti
ECAN_COLLECT_RENT	STI, costs, overuse traces	updated sti, lti
ECAN_DIFFUSE_STI	STI, Hebbian/InverseHebbian links, frontier	updated sti along bounded frontier
ECAN_UPDATE_HEBBIAN	coactivity over a recent window	hebb, inv-hebb weight updates
ECAN_PROMOTE_DEMOTE	STI, LTI, context, ACS membership, quotas	pool, pool-hot migrations
ECAN_INFO_GEOM_STEP	STI distribution, utility, association graph, ground cost	updated sti (mirror-descent or Wasserstein)
ECAN_FORGET	LTI, protected-set membership	archived or deleted atoms
SNAPSHOT_CONTEXT	context, pool-hot, tick, budget	snapshot, snapshot-context, snapshot-source-tick
PETTA_BUILD_WORKER_BATCH	contexts, demes, scan windows, budgets	worker facts, par-group facts
PETTA_RUN_PAR_GROUP	par-group, worker facts	event-log facts, worker-status updates
RUN_MM2_CHAMBER	worker, chamber, snapshot, hot-pool, seed, budget	event-log, trace, action, stimulus-event, done
REDUCE_EVENT_LOGS	event-log batch	reward summaries, trace summaries, ACS scan inputs
REDUCE_STIMULUS	stimulus-event batch	per-atom stimulus totals, per-worker stimulus totals
SCHEDULE_WORKERS	worker facts, worker STI/LTI, budgets, ready queue	par-group, worker-status updates, worker-budget updates

Kernel	Input atoms	Output atoms
PUBLISH_BACKEND_FORM	Worker IR, backend tag	PETTA predicate call or METTA-IL worker form

D.3 Host-side data structures

The host layer can use the following internal structs. These are not necessarily MORK atoms; they are convenient implementation views.

```

struct ContextId { symbol: String }
struct TickId    { symbol: String }
struct RuleId    { symbol: String }
struct TraceId   { symbol: String }

struct Candidate {
  ctx: ContextId,
  tick: TickId,
  rule: RuleId,
  bindings: Vec<(String, String)>,
  raw_logit: f64,
  score: f64,
}

struct RuleStats {
  rule: RuleId,
  logit: f64,
  prior_logit: f64,
  cost: f64,
  use_count: u64,
  success_count: u64,
  last_fired: Option<TickId>,
}

struct TraceRecord {
  trace: TraceId,
  ctx: ContextId,
  tick: TickId,
  rule: RuleId,
  action: String,
  bindings: Vec<(String, String)>,
  discount: f64,
  status: TraceStatus,
}

```

E Inner loop pseudocode

E.1 Top-level tick

```

procedure ACTPC_TICK(space, ctx):
  tick <- allocate_tick(ctx)
  add(space, (tick ctx tick))

```

```

SENSE(space, ctx, tick)
DERIVE_FEATURES_MM2(space, ctx, tick)
ACTIVATE_SIGNATURES_MM2(space, ctx, tick)

# ECAN ensures active signatures have warm and hot support.
ECAN_RETRIEVE_AND_PROMOTE(space, ctx, active_signatures(ctx))

MATCH_CANDIDATES_MM2(space, ctx, tick) # over hot pool only
SCORE_CANDIDATES(space, ctx, tick)      # uses STI/LTI/ACSBonus
chosen <- SOFTMAX_SAMPLE(space, ctx, tick)

FIRE_CHOSEN(space, ctx, tick, chosen)
ENV_STEP(space, ctx, tick)

COMPUTE_PREDICTION_ERROR(space, ctx, tick)
ASSIGN_CREDIT(space, ctx, tick)
UPDATE_RULE_WEIGHTS(space, ctx, tick)

# Aggregate stimulus and update the attention economy.
stimulus <- ECAN_GATHER_STIMULUS(space, ctx, tick)
ECAN_UPDATE(space, ctx, stimulus, recent_window(ctx))

if SHOULD_RUN_STRUCTURAL_LOOP(space, ctx, tick):
    enqueue_structural_job(ctx, tick)

if tick % info_geom_period == 0:
    ECAN_INFO_GEOM_STEP(space, ctx) # mirror-descent / Wasserstein

CLEANUP_TRANSIENTS(space, ctx, tick)
end procedure

```

E.2 PETTA worker orchestration around single-threaded chambers

The procedure `ACTPC_TICK` describes the inside of one context chamber. The PETTA wrapper schedules many such chambers and auxiliary jobs. This pseudocode is Prolog-like but schematic; exact PETTA syntax may differ.

```

% Build a backend-neutral worker batch.
build_worker_batch(Epoch, Workers) :-
    findall(W, make_context_chamber_worker(Epoch, W), CtxWorkers),
    findall(W, make_moses_deme_worker(Epoch, W), DemeWorkers),
    findall(W, make_acs_scan_worker(Epoch, W), AcswWorkers),
    maybe_make_ecan_epoch_worker(Epoch, EcanWorkers),
    maybe_make_bridge_fit_worker(Epoch, BridgeWorkers),
    append([CtxWorkers, DemeWorkers, AcswWorkers,
            EcanWorkers, BridgeWorkers], Workers).

% PetTa backend: schedule the Worker IR using Prolog predicates.
run_epoch_petta(Epoch) :-
    build_worker_batch(Epoch, Workers),
    schedule_workers(Workers, Selected),
    make_par_group(Epoch, Selected, PG),
    run_par_group_petta(PG),

```



```

    reduce_par_group_outputs(PG).

run_par_group_petta(PG) :-
    par_members(PG, Workers),
    run_workers_in_thread_pool(Workers).

run_worker(W) :-
    worker_kind(W, context_chamber), !,
    worker_context(W, C),
    worker_snapshot(W, Snap),
    worker_seed(W, Seed),
    restore_context_snapshot(Snap),
    run_single_threaded_mm2_chamber(C, Seed, Events),
    write_event_log(W, Events).

run_worker(W) :- worker_kind(W, moses_deme), !, run_moses_deme_batch(W).
run_worker(W) :- worker_kind(W, acs_scan), !, run_acs_scan(W).
run_worker(W) :- worker_kind(W, ecan_epoch), !, run_ecan_epoch_reducer(W).
run_worker(W) :- worker_kind(W, bridge_fit), !, run_bridge_fit(W).

```

E.3 Backend translation principle

A worker should have two renderings: a PETTA rendering and a METTA-IL rendering. Both are generated from the same worker facts.

```

Portable Worker IR:
    (worker W17)
    (worker-kind W17 context-chamber)
    (worker-context W17 C42)
    (worker-snapshot W17 SNAP17)
    (worker-hot-pool W17 HP42)
    (worker-seed W17 12345)

PeTTa rendering:
    run_worker(w17).

MeTTa-IL rendering:
    (run-worker W17)

Shared semantics:
    restore SNAP17;
    run one single-threaded MM2 chamber for C42;
    consume HP42;
    emit LOG99.

```

Any backend-specific optimization must preserve this shared semantics.

E.4 Scoring candidates

The score is not the same as the rule logit. It blends ACTPC-CHEM’s model-side belief about the rule with ECAN’s attention-side evidence that the rule deserves processor time, plus context relevance, novelty, ACS membership, precedence / value-relative anti-precedence (§19.5), and cost.

The recommended combined-score template is:

$$\begin{aligned}
\text{Score}(R; C) = & \ell_R^{\text{ActPC}}/T + \alpha_s \log(\text{STI}(R) + \varepsilon) + \alpha_l \log(\text{LTI}(R) + \varepsilon) \\
& + \gamma \text{Rel}(R, C) + \delta \text{ACSBonus}(R) \\
& + \eta_{\text{id}} h_R + \eta_{\text{func}} c_R H^{\text{func}} \\
& - \xi (1 - c_R) O_{\pi(R)}^p + \rho R_{\pi(R)} \\
& - \kappa \text{cost}(R) + \nu \text{Novelty}(R, C),
\end{aligned}$$

where ℓ_R^{ActPC} is the ACTPC-CHEM rule logit; STI and LTI are ECAN attention values; $\text{Rel}(R, C)$ is context relevance; $\text{ACSBonus}(R)$ rewards membership or catalytic role in active useful ACSs; h_R is identity precedence (recent use trace of R); $c_R \in [0, 1]$ is functional protection; $O_{\pi(R)}$ is the excess canalization of R 's pattern family; H^{func} is a functional-memory trace; $R_{\pi(R)}$ is the neighbor-release stimulus into that family; and T is a sampling temperature. The simple broad-anti-resonance baseline $-\xi \Phi(h_R; \theta)$ is retained as an ablation only.

```

procedure SCORE_CANDIDATES(space, ctx, tick):
  inputs <- query(space, (score-input ctx tick ?R ?L ?Cost ...))
  for each input in inputs:
    R <- input.rule
    L <- input.logit                                # ActPC-side logit
    cost <- input.cost
    g <- family_of(R)

    sti <- get_sti(space, R)
    lti <- get_lti(space, R)
    rel <- compute_context_relevance(space, ctx, R)
    novelty <- compute_novelty_bonus(space, ctx, R)
    acs_bonus <- compute_acs_bonus(space, R)

    h_R <- get_overuse_trace(space, R)                # identity precedence
    c_R <- get_functional_protection(space, R)        # in [0, 1]
    H_fun <- get_functional_memory_trace(space, R)
    O_g <- get_family_excess_canalization(space, g)
    R_g <- get_family_release_into(space, g)

    # Recommended (family_protected) form:
    precedence_pos = eta_id * h_R
    functional_pos = eta_func * c_R * H_fun
    anti_precedence = xi * (1.0 - c_R) * (O_g ^ p_overuse)
    release_bonus = rho_release * R_g

    # Baseline broad-anti-resonance term, used only when
    # config.anti_precedence.mode == "broad".
    broad_overuse_pen = threshold_overuse(h_R, theta_overuse)

    inhibition <- compute_inhibition(space, ctx, R)

    if config.anti_precedence.mode == "family_protected":
      score <- L / temperature
        + alpha_sti * log(sti + eps)
        + alpha_lti * log(lti + eps)
        + gamma_rel * rel

```

```

        + delta_acs * acs_bonus
        + precedence_pos
        + functional_pos
        - anti_precedence
        + release_bonus
        - lambda_cost * cost
        + beta_novelty * novelty
        - inhibition
    else: # mode == "broad" (legacy ablation)
        score <- L / temperature
        + alpha_sti * log(sti + eps)
        + alpha_lti * log(lti + eps)
        + gamma_rel * rel
        + delta_acs * acs_bonus
        + mu_resonance * h_R
        - xi_anti_resonance * broad_overuse_pen
        - lambda_cost * cost
        + beta_novelty * novelty
        - inhibition

    add_or_replace(space, (cand-score ctx tick R score))
    mark_done(space, ctx, tick, score)
end procedure

```

In a first prototype, several of these coefficients can be set to zero; the minimal version is just $L/T - \lambda \text{cost} + \nu \text{novelty}$, to which $\alpha_s \log(\text{STI})$ and $\delta \text{ACSBonus}$ should be added once ECAN and the ACS detector are wired in. Once stable, the precedence and value-relative anti-precedence terms become the principal levers against sticky high-utility habits, with broad-overuse $\Phi(h_R; \theta)$ kept only as a baseline ablation.

E.5 Softmax sampling

```

procedure SOFTMAX_SAMPLE(space, ctx, tick):
    candidates <- query(space, (cand-score ctx tick ?R ?S))
    if candidates is empty:
        add(space, (chosen ctx tick R_NOOP none 0 0))
        return R_NOOP

    temperature <- get_temperature(space, ctx)
    max_score <- max(c.score for c in candidates)

    weights <- []
    for c in candidates:
        w <- exp((c.score - max_score) / max(temperature, eps))
        weights.append(w)

    selected <- sample_categorical(candidates, weights)
    add(space, (chosen ctx tick selected.rule selected.bindings...))
    return selected
end procedure

```

E.6 Prediction distribution

There are multiple ways to define predicted distribution $p_t(m)$. For a first implementation, use the candidate-action distribution implied by softmax. For a richer implementation, simulate or enumerate the outcomes produced by selected prediction rules.

```
procedure COMPUTE_PREDICTION_ERROR(space, ctx, tick):
  pred_counts <- empty map pattern -> float

  for each (cand-score ctx tick R score):
    p_R <- softmax_probability(R)
    outcomes <- get_rule_outcome_predictions(space, R)
    if outcomes is empty:
      outcomes <- default_action_outcome_predictions(R)
    for (pattern, prob) in outcomes:
      pred_counts[pattern] += p_R * prob

  obs_counts <- derive_observed_distribution(space, ctx, tick)

  pred <- normalize_with_floor(pred_counts, epsilon)
  obs <- normalize_with_floor(obs_counts, epsilon)

  kl <- 0
  xent <- 0
  for pattern in union(keys(pred), keys(obs)):
    q <- obs.get(pattern, epsilon)
    p <- pred.get(pattern, epsilon)
    kl += q * (log(q) - log(p))
    xent += -q * log(p)

  add_distribution_atoms(space, ctx, tick, pred, obs)
  add_or_replace(space, (error ctx tick kl kl))
  add_or_replace(space, (error ctx tick xent xent))

  r_int <- -kl
  r_ep <- compute_epistemic_reward(obs, pred)
  add_or_replace(space, (reward ctx tick instrumental r_int))
  add_or_replace(space, (reward ctx tick epistemic r_ep))
end procedure
```

E.7 Epistemic reward

A simple epistemic signal rewards uncertainty reduction rather than raw surprise. Raw surprise alone can lead to pathological novelty seeking. A safer starter formula is:

$$r_t^{ep} = \max(0, e_{t-1} - e_t) + \eta H(p_t), \quad (3)$$

where $H(p_t)$ is entropy and η is small. This rewards learning progress plus a mild preference for maintaining exploration.

```
procedure COMPUTE_EPISTEMIC_REWARD(ctx, tick, obs, pred):
  prev_error <- get_previous_error(ctx, kind="kl")
  curr_error <- KL(obs, pred)
  learning_progress <- max(0, prev_error - curr_error)
```

```

    entropy_bonus <- entropy(pred)
    return learning_progress + eta_entropy * entropy_bonus
end procedure

```

E.8 Credit assignment

```

procedure ASSIGN_CREDIT(space, ctx, tick):
    env_reward <- get_reward(space, ctx, tick, "env", default=0.0)
    r_int <- get_reward(space, ctx, tick, "instrumental", default=0.0)
    r_ep <- get_reward(space, ctx, tick, "epistemic", default=0.0)

    total_signal <- alpha_env * env_reward
                  + alpha_int * r_int
                  + alpha_ep * r_ep

    baseline <- get_context_baseline(space, ctx)
    advantage <- total_signal - baseline

    traces <- query_open_traces(space, ctx)
    for tr in traces:
        age <- tick_age(tr.tick, tick)
        elig <- pow(gamma_trace, age) * tr.discount
        credit <- elig * advantage
        add_or_replace(space, (credit tr.id tr.rule delayed credit))
        accumulate(space, (credit-total tr.rule ctx tick credit))

        if age > max_trace_age or trace_resolved(tr):
            add_or_replace(space, (trace-status tr.id closed))

    update_context_baseline(space, ctx, total_signal)
end procedure

```

E.9 Weight update: simple online version

Use this before implementing optimal transport. It is robust, easy to test, and fast.

$$\ell_R \leftarrow (1 - \rho)\ell_R + \rho\ell_R^{\text{prior}} + \eta \cdot \text{clip}(A_R, -a_{\max}, a_{\max}) - \lambda \cdot \text{cost}(R), \quad (4)$$

where ℓ_R is the rule logit and A_R is accumulated advantage/credit.

```

procedure UPDATE_RULE_WEIGHTS(space, ctx, tick):
    credits <- query(space, (credit-total ?R ctx tick ?Cr))
    for (R, Cr) in credits:
        old <- get_rule_logit(space, R)
        prior <- get_rule_prior_logit(space, R)
        cost <- get_rule_cost(space, R)

        clipped <- clamp(Cr, -advantage_clip, advantage_clip)
        new <- (1 - prior_pull) * old
              + prior_pull * prior
              + eta_logit * clipped
              - lambda_cost_update * cost

```

```

    new <- clamp(new, min_logit, max_logit)
    replace(space, (rule-logit R old), (rule-logit R new))
    increment(space, (rule-use-count R))
end procedure

```

E.10 Delayed rewards

For delayed poison or delayed benefit, the trace table is the key. Traces remain open across many ticks.

```

procedure OPEN_TRACE(rule, action, ctx, tick, bindings):
  E <- allocate_trace_id()
  add(space, (trace E ctx tick rule action))
  add(space, (trace-status E open))
  add(space, (trace-age E 0))
  add(space, (trace-discount E 1.0))
  for (name, value) in bindings:
    add(space, (trace-bind E name value))
  return E
end procedure

procedure AGE_TRACES(space, ctx):
  for each open trace E in ctx:
    age <- get(trace-age E)
    replace(space, (trace-age E age), (trace-age E age+1))
    if age+1 > max_trace_age:
      replace(space, (trace-status E open), (trace-status E expired))
end procedure

```

For causal discovery, preserve a compact event window:

```

(event E88 ctx C42 tick T193 kind ate-item object 09)
(event-feature E88 color-red)
(event-feature E88 shape-round)
(event-feature E88 smell-sweet)
(event-followup E88 T240 outcome sickness)

```

F Wasserstein natural-gradient module

F.1 Purpose

The simple logit update moves each rule independently. The Wasserstein module moves probability mass along a graph of similar rules. This gives smoother updates: a successful specialized rule can lend probability mass to nearby variants, and a failed rule can transfer mass to close alternatives rather than forcing global random exploration.

F.2 Rule-space graph

The graph has nodes R_i and edges between rules that are structurally or behaviorally close.

```

(rule-neighbor R17 R22 0.08)    ; small number means close
(rule-neighbor R17 R35 0.21)

```

Possible edge costs include source edit distance, GCoDD graph edit distance, guard-set Jaccard distance, template/action distance, behavioral distance on a replay buffer, and abstraction/family distance. A practical ground metric is a weighted mixture:

$$\omega_{ij} = w_s d_{source}(i, j) + w_g d_{gcodd}(i, j) + w_b d_{behavior}(i, j) + w_c d_{cost}(i, j). \quad (5)$$

F.3 Measure-dependent Laplacian

Let p_i be the current probability of rule i . For an edge (i, j) , define conductance

$$c_{ij}(p) = \frac{p_i + p_j}{2 \max(\omega_{ij}, \epsilon)}. \quad (6)$$

Then the graph Laplacian is

$$L_{ii}(p) = \sum_{j \in N(i)} c_{ij}(p), \quad L_{ij}(p) = -c_{ij}(p) \quad \text{if } (i, j) \in E. \quad (7)$$

This is a practical sparse approximation. Different discrete OT conventions may use slightly different constants; the interface should allow replacement.

F.4 OT step pseudocode

```

procedure OT_STEP(rule_ids, logits, gradients, neighbor_edges):
  # rule_ids: active rules in one context, family, or deme
  # logits:   current unconstrained weights ell_i
  # gradients: dF / d ell_i or negative credit signal
  # edges:    (i, j, omega_ij)

  p <- softmax(logits)

  L <- sparse_zero_matrix(n, n)
  for (i, j, omega) in neighbor_edges:
    conductance <- 0.5 * (p[i] + p[j]) / max(omega, eps)
    L[i,i] += conductance
    L[j,j] += conductance
    L[i,j] -= conductance
    L[j,i] -= conductance

  # Project gradient to zero-sum tangent space.
  g <- gradients - mean(gradients)

  # Solve L phi = g with damping because L is singular.
  # Equivalent to using a pseudoinverse on the zero-sum subspace.
  phi <- conjugate_gradient(L + damping * I, g,
                           constraint="zero_mean",
                           max_iter=ot_max_iter)

  # Natural-gradient direction in logit coordinates.
  # Sign convention: minimize F, or maximize reward.
  delta <- -ot_step_size * phi

  new_logits <- logits + delta

```

```

    new_logits <- clamp_vector(new_logits, min_logit, max_logit)
    return new_logits
end procedure

```

F.5 When to apply OT

Do not apply OT globally every tick. Use it in restricted scopes:

- active candidate set for one context;
- one rule family;
- one MOSES deme;
- rules sharing a signature;
- top- k rules by recent credit or surprise.

F.6 OT host request

```

(ot-request C42 T193 family food-grab)
(ot-param C42 T193 step-size 0.05)
(ot-param C42 T193 damping 0.001)

```

Host response:

```

(rule-logit R17 0.46)
(rule-logit R22 0.31)
(rule-logit R35 -0.04)
(ot-done C42 T193 food-grab)

```

G Outer loop: structural evolution with MOSES, gCoDD, and CENF

G.1 Outer-loop role

The outer loop proposes new molecules for the chemistry. It should not be responsible for every online decision. It receives accumulated evidence and returns candidate rules or abstractions.

```

Inner loop output:
    traces, credits, errors, high-value rules, failed contexts

```

```

Outer loop processing:
    extract fragments
    canonicalize as gCoDD
    mine guard correlations
    build/update factor graph
    sample candidates
    normalize with ENF/CENF
    compile candidates to MM2 facts

```

```

Inner loop input:
    new trial rules, meta-guards, priors, rule-neighbor edges

```


G.2 Structural job types

Job	Meaning
high-credit-rule	Clone, specialize, or abstract a successful rule.
high-error-context	Propose new predictors or causal rules for an unexplained outcome.
unstable-rule	Split a rule into more specific variants.
redundant-family	Merge or retire rules with similar behavior.
feature-correlation	Create a meta-guard from co-occurring guards.
deme-refresh	Rebuild a MOSES factor graph and sample new rules.

G.3 Rule mutation operators

```
SPECIALIZE_CONDITION(rule):
    add one guard that was present in successful traces

GENERALIZE_CONDITION(rule):
    remove one guard that appears unnecessary

SWAP_TEMPLATE(rule):
    replace action or predicted outcome with a nearby template

ABSTRACT_GUARDS(rule):
    replace several guards by a meta-guard

INLINE_METAGUARD(rule):
    replace a meta-guard by its member guards

SPLIT_RULE(rule):
    create variants for high-variance trace clusters

MERGE_RULES(rule_a, rule_b):
    create a generalized rule if behavior is compatible

ADD_DELAY_CONDITION(rule):
    add elapsed-time or event-window guard

ADD_CONTEXT_CONDITION(rule):
    add activity, temperature, goal, agent, or memory condition
```

G.4 Structural loop pseudocode

```
procedure STRUCTURAL_LOOP(space, job):
    evidence <- COLLECT_EVIDENCE(space, job)

    fragments <- EXTRACT_RULE_FRAGMENTS(evidence)
    canonical <- []
    for frag in fragments:
        dag <- COMPILE_FRAGMENT_TO_GCDD(frag)
        dag <- ENF_NORMALIZE(dag)
        canonical.append(dag)
```

```

if job.type in {feature-correlation, deme-refresh}:
    meta_guardes <- CENF_CLUSTER(space, evidence)
    INSERT_META_GUARDS(space, meta_guardes)

deme <- GET_OR_CREATE_DEME(space, job, canonical)
UPDATE_DEME_MODEL(space, deme, evidence)

candidates <- []
for k in 1..outer_batch_size:
    sample <- FG_SAMPLE(deme.model)
    candidate_dag <- BUILD_GCDD_FROM_SAMPLE(deme.sketch, sample)
    candidate_dag <- ENF_NORMALIZE(candidate_dag)
    candidate_dag <- CENF_NORMALIZE(candidate_dag, active_meta_guardes)
    candidate_rule <- COMPILE_GCDD_TO_RULE(candidate_dag)
    candidates.append(candidate_rule)

for rule in candidates:
    if VALIDATE_RULE(rule):
        rid <- INSERT_TRIAL_RULE(space, rule)
        COMPILE_RULE_TO_HOT_PATH(space, rid)
        ADD_RULE_NEIGHBORS(space, rid)
end procedure

```

G.5 CENF meta-guard creation

```

procedure CENF_CLUSTER(space, evidence):
    # Gather guard co-occurrences weighted by trace credit.
    counts <- map pair(guard_a, guard_b) -> float
    singles <- map guard -> float

    for trace in evidence.traces:
        guards <- guards_active_in_trace(trace)
        weight <- max(0, trace.credit)
        for g in guards:
            singles[g] += weight
        for unordered pair (g1, g2) in guards:
            counts[g1,g2] += weight

    # Convert counts to association scores.
    edges <- []
    for (g1, g2), c in counts:
        pmi <- log((c + eps) / ((singles[g1] + eps) *
                                (singles[g2] + eps)))
        if c >= min_support and pmi >= min_pmi:
            edges.append((g1, g2, pmi))

    clusters <- threshold_connected_components(edges)

    meta_guardes <- []
    for cluster in clusters:
        if size(cluster) >= 2 and size(cluster) <= max_meta_guard_size:
            mg <- allocate_meta_guard_id(cluster)
            meta_guardes.append((mg, cluster))

```

```

    return meta_guards
end procedure

```

G.6 Compiling meta-guards

```

procedure INSERT_META_GUARDS(space, meta_guards):
  for (MG, members) in meta_guards:
    add(space, (meta-guard MG))
    add(space, (meta-guard-status MG active))
    for g in members:
      add(space, (meta-guard-member MG g))

  # Create derived guard-has facts for current contexts.
  for ctx_obj in objects_with_all_members(space, members):
    add(space, (guard-has ctx_obj.ctx ctx_obj.obj MG))
end procedure

```

G.7 MOSES factor-graph EDA

A deme has a sketch and variables. Variables may include guard presence bits, meta-guard presence bits, template/action choice, delay threshold, context type, numeric parameters such as temperature or confidence threshold, and choice of grounded predicate leaf.

```

procedure UPDATE_DEME_MODEL(space, deme, evidence):
  examples <- SELECT_EXEMPLARS(evidence, deme)

  vars <- GET_DEME_VARIABLES(deme)
  factors <- MINE_FACTORS(examples, vars)

  model <- empty_factor_graph()
  for v in vars:
    model.add_variable(v, domain(v))

  for factor in factors:
    potential <- estimate_potential(factor, examples,
                                     smoothing=factor_smoothing)
    model.add_factor(factor.vars, potential)

  run_belief_propagation(model)
  store_model(space, deme, model)
end procedure

```

G.8 Sampling candidate rules from a deme

```

procedure BUILD_GCODD_FROM_SAMPLE(sketch, sample):
  dag <- copy(sketch.base_dag)
  for knob in sketch.knobs:
    value <- sample[knob.id]
    dag <- fill_knob(dag, knob, value)
  return dag
end procedure

```

```

procedure COMPILE_GCDDD_TO_RULE(dag):
  source <- gcodd_to_metta_source(dag)
  guards <- extract_guards(dag)
  trigger <- select_primary_trigger(guards)
  template <- extract_template(dag)
  cost <- count_nodes(dag) + grounded_leaf_costs(dag)
  return RuleSource(source, trigger, guards, template, cost)
end procedure

```

G.9 Inserting trial rules

New rules should enter with low or moderate prior weight and a trial status.

```

procedure INSERT_TRIAL_RULE(space, rule):
  hash <- CANON_HASH(rule.source)
  existing <- query(space, (rule-src-hash ?R hash))
  if existing not empty:
    return existing[0]

  R <- allocate_rule_id()
  add(space, (rule R))
  add(space, (rule-kind R rule.kind))
  add(space, (rule-status R trial))
  add(space, (rule-owner R moose))
  add(space, (rule-src R rule.source))
  add(space, (rule-src-hash R hash))
  add(space, (rule-cost R rule.cost))
  add(space, (rule-logit R trial_initial_logit))
  add(space, (rule-prior-logit R trial_prior_logit))
  return R
end procedure

```

H AIRIS and PLN integration

H.1 Design principle

AIRIS and PLN should not bypass the chemistry. They propose rules, abstractions, and constraints. Those proposals enter the same weighted rule soup as other molecules and are tested by prediction error and reward.

H.2 AIRIS causal rule proposal

AIRIS-style refinement is triggered by a surprising transition.

```

procedure AIRIS_PROPOSE(space, ctx, tick):
  surprise <- get_error(space, ctx, tick, "kl")
  if surprise < airis_threshold:
    return []

  prev_events <- get_recent_events(space, ctx, window=airis_window)
  outcome <- get_outcome(space, ctx, tick)

```

```

action <- get_action(space, ctx, tick)

candidate_conditions <- abduce_conditions(prev_events, action, outcome)

rules <- []
for cond_set in candidate_conditions:
    rule <- make_causal_rule(
        conditions=cond_set,
        action=action.kind,
        predicted_outcome=outcome.kind,
        delay=estimate_delay(prev_events, outcome))
    rules.append(rule)

return rules
end procedure

```

Example output:

```

(rule-src R900
(=> (and (event $E ate-item $Obj)
        (event-feature $E color-red)
        (event-feature $E shape-round)
        (activity-after $E mostly-still)
        (elapsed $E greater-than 50))
(predict $E sickness)))

```

H.3 PLN abstraction proposal

PLN-style inference proposes a generalized rule from observed cases and analogies.

```

procedure PLN_PROPOSE(space, job):
    examples <- collect_positive_and_negative_examples(space, job)
    background <- retrieve_relevant_ltm(space, examples)

    hypotheses <- []
    hypotheses += inductive_generalizations(examples)
    hypotheses += analogical_mappings(examples, background)
    hypotheses += abductive_explanations(examples, background)

    rules <- []
    for h in hypotheses:
        if h.confidence >= pln_min_confidence:
            rule <- translate_pln_hypothesis_to_rule(h)
            rule.owner <- "pln"
            rule.prior_logit <- confidence_to_logit(h.confidence)
            rules.append(rule)
    return rules
end procedure

```

H.4 Proposal merger

```

procedure PROCESS_SYMBOLIC_PROPOSALS(space, ctx, tick):
    airis_rules <- AIRIS_PROPOSE(space, ctx, tick)

```

```

pln_rules <- PLN_PROPOSE(space, structural_job(ctx, tick))

all_rules <- airis_rules + pln_rules
for rule in all_rules:
    dag <- COMPILE_RULE_TO_GCDD(rule)
    dag <- ENF_NORMALIZE(dag)
    rule <- GCDD_TO_RULE_SOURCE(dag)
    R <- INSERT_TRIAL_RULE(space, rule)
    COMPILE_RULE_TO_HOT_PATH(space, R)
    ADD_RULE_NEIGHBORS(space, R)
end procedure

```

I Attention economy: ECAN integration

This section specifies the attention-economy layer that sits between the ACTPC-CHEM inner loop and the rest of the cognitive system. Its job is to maintain the cold/warm/hot pool partition of the rule soup, decide moment-by-moment which atoms participate in any given MM2 tick, and consolidate the credit signals coming from every other component into durable retention decisions.

I.1 The two distributions, and what they do

The system maintains two coupled but distinct probability-like distributions over symbolic content:

- $p_t(R)$ — ACTPC-CHEM’s belief that rule R is useful as a model or action under the current context. This is what the inner-loop logits represent and what the Wasserstein update modifies.
- $a_t(A)$ — ECAN’s attention mass on atom A . This is what pool-migration decisions, hot-pool admission, and forgetting are based on. STI is the unnormalized representative of a_t ; LTI is a slower, more durable companion.

These distributions are coupled, not identified. A rule can have high $p_t(R)$ but low STI if the current context does not need it; an atom can have high STI but low $p_t(R)$ if it is novel, diagnostic, or part of a rare-tail near-ACS motif. The coupling is mediated by stimulus flow (ACTPC-CHEM pays ECAN stimulus when a rule reduces prediction error or earns reward) and a soft KL regularizer in the joint loss (§16.10). The separation is what prevents the system from collapsing into a brittle high-utility circuit.

I.2 Working memory and long-term memory under ECAN

Working memory and long-term memory are not separate modules in this design; they are operational consequences of the attention economy.

- *Working memory* = the hot pool plus context-local scratch atoms. Lifetime is one tick or a small number of ticks. STI for working-memory atoms is paid by the active context and decays rapidly when the context moves on.
- *Long-term memory* = atoms with sufficiently high LTI to survive forgetting. Includes durable rules, meta-guards, ACS macro-atoms, and abstractions accumulated by PLN.

In schema terms, working memory atoms are still represented as before (`wm C42 entity 09`, etc.), but their persistence is now under the control of ECAN’s STI dynamics rather than ad-hoc TTL fields.

I.3 Atom and link schemas

Add the following relation-first schemas to those described in §8.

I.3.1 Attention facts

```
(sti R17 0.013)
(lti R17 0.67)
(pool R17 warm)
(pool-hot C42 R17)           ; context-local hot membership
(overuse-trace R17 2.83)     ; h_R, identity-precedence trace
(context-relevance C42 R17 0.77)
```

(pool R17 warm) expresses global pool membership; the context-local (pool-hot C42 R17) expresses that R17 is in the hot pool for context C42. The hot loop should query by the context-local form, since several contexts can share the same warm soup but disagree on what is hot for them.

I.3.2 Worker-level attention facts

Workers are also atoms for ECAN purposes. Worker-level STI allocates processor urgency; worker-level LTI tracks whether a worker template or worker family should persist.

```
(sti W17 0.031)
(lti W17 0.44)
(worker-priority W17 0.71)
(worker-runtime-credit W17 0.18)
(worker-cpu-ms-used W17 47)
(worker-output-utility W17 0.22)
(worker-output-novelty W17 0.05)
(worker-output-accs-yield W17 0.03)
(worker-rent W17 cpu 0.004)
(worker-rent W17 memory 0.001)

(worker-template WT_context_chamber)
(worker-template-kind WT_context_chamber context-chamber)
(sti WT_context_chamber 0.02)
(lti WT_context_chamber 0.70)

(worker-family WF_context_chambers)
(worker-family-member WF_context_chambers W17)
(worker-family-share WF_context_chambers 0.40)
(worker-family-value WF_context_chambers 0.31)
```

I.3.3 Anti-precedence facts (family-level)

These mirror the schemas in §19.5 and the persistent pattern-family facts above. They are the inputs and outputs of the value-relative anti-precedence kernels.

```
; Family-level shares and value
(family-share G17 0.34)           ; S_g(t)
(family-value G17 0.21)          ; V_g(t)
(family-value-share G17 0.18)    ; S*_g(t)
```

```

(family-excess-canalization G17 0.11) ; O_g(t)
(family-release-into G18 0.07) ; R_h(t)

; Per-rule functional protection
(functional-protection R17 0.82) ; c_r in [0,1]
(functional-role R17 validator)

; Diagnostic: per-rule decomposition of the rent paid
(rent-decomp T193 R17 base 0.0010)
(rent-decomp T193 R17 cost 0.0006)
(rent-decomp T193 R17 excess-can 0.0002)
(rent-decomp T193 R17 release-bonus 0.0011)

```

The first block lets the match phase and the scoring kernel read S_g^* , O_g , and c_r as relation-first facts; the `rent-decomp` block is optional but extremely useful for debugging the anti-precedence economy.

1.3.4 Association links

```

(hebb R17 R22 0.37)
(inv-hebb R17 R44 0.12)
(coactive R17 R22 14)
(co-catalytic R17 R22 5)
(context-assoc C42 R17 R22 0.22)

```

The association graph should be sparse. A practical cap is 16–64 Hebbian and 8–32 InverseHebbian edges per atom. ECAN kernels enforce these caps at update time.

1.3.5 Stimulus events

Stimulus is logged explicitly during early development; debugging the attention economy is otherwise extremely difficult.

```

(stimulus T193 R17 0.18 actpc-prediction-gain)
(stimulus T193 R22 0.07 acs-coupling)
(stimulus T193 R30 0.21 external-reward)
(stimulus T193 MG12 0.05 rare-tail-precursor)
(stimulus T193 ACS42 0.31 active-useful-accs)

```

After validation, stimulus events can be summarized into per-rule running totals and compacted.

1.3.6 ACS macro-atoms

When the ACS detector finds a recurrent active useful subnetwork, it reifies it as a macro-atom. The macro-atom is itself subject to STI/LTI dynamics:

```

(acs ACS42)
(acs-member ACS42 R17)
(acs-member ACS42 MG12)
(acs-member ACS42 R22)
(acs-utility ACS42 0.72)
(acs-coupling ACS42 0.46)
(acs-lifetime ACS42 1287)
(sti ACS42 0.04)

```


(lti ACS42 0.83)
(pool ACS42 warm)

This is the algorithmic-chemistry version of ECAN *map formation*: an attentional attractor becomes a manipulable symbolic object that can be reasoned about, planned with, and evolved.

I.4 Stimulus sources

Every other component pays ECAN stimulus on atoms it found useful. The aggregate is what drives wages.

Source	What it stimulates
ACTPC-CHEM	Rules that reduce prediction error, earn reward, or contribute to learning progress. Stimulus is proportional to per-rule credit.
ACS detector	Members, catalysts, products, and rare-tail precursors of active useful ACSs. Modulated by ablation-coupling score so that decorative ACSs do not collect stimulus.
MOSES	High-fitness program/rule fragments, surviving gCoDD roots, CENF meta-guards that compress useful structure, fragments that transfer across task families.
AIRIS	Causal rules that explain surprising transitions and reduce future prediction error.
PLN	Abstractions and analogies that improve heldout prediction or compress rule count.
External reward	Atoms causally implicated in reward delivery, attributed via eligibility traces.
Context relevance	Atoms whose signatures match active observations or goals receive a small contextual stimulus.

A simple ACTPC-CHEM-side stimulus on a fired rule r is

$$\text{stim}_{\text{ACTPC-CHEM}}(r, t) = c_1 \Delta e_t^+(r) + c_2 R_t^+(r) + c_3 \text{Novelty}_t(r) - c_4 \text{cost}(r),$$

where $\Delta e_t^+(r)$ is the positive prediction-error reduction attributed to r . ACS-side stimulus on a member atom A of an active ACS S is

$$\text{stim}_{\text{ACS}}(A, t) = d_1 \text{Member}(A, S) + d_2 \text{Catalyst}(A, S) + d_3 \text{Coupling}(S) + d_4 \text{Transfer}(S),$$

with the coupling factor being the same ablation-based $U(\text{soup}) - U(\text{soup} \setminus S)$ ratio that already appears in the benchmark suite. This makes the attention economy and the benchmark suite point in the same direction: useful ACSs win, both in scoring and in attention.

I.5 Core ECAN dynamics

The core dynamics for STI and LTI are the standard ECAN economic update:

$$\begin{aligned}\text{Wage}_{\text{STI}}(A) &= w_s s_A, \\ \text{Wage}_{\text{LTI}}(A) &= w_l \text{Durable}(s_A), \\ \text{Rent}(A) &= \text{Rent}_{\text{base}}(A) + \text{Rent}_{\text{cost}}(A) + \text{Rent}_{\text{excess-can}}(A),\end{aligned}$$

where s_A is the aggregated stimulus on atom A and $\text{Durable}(s_A)$ extracts the portion of stimulus associated with repeated, transferable, or ablation-supported usefulness. The three rent components are:

- *Base rent*: a flat charge for occupying hot or warm pool slots; prevents permanent monopolies.
- *Cost rent*: scales with the runtime/structural cost of the atom.
- *Excess-canalization rent* (value-relative anti-precedence, §19.5):

$$\text{Rent}_{\text{excess-can}}(A) = r_O (1 - c_A) O_{\pi(A)}(t)^p,$$

where $c_A \in [0, 1]$ is the functional-protection coefficient, $\pi(A)$ is the pattern family of A , and $O_g(t) = [S_g(t) - S_g^*(t) - \theta_O]_+$ is the family’s excess canalization. Atoms in over-dominant families pay rent proportional to how much their family exceeds its value-justified share, but only to the extent that they are *not* shared functional machinery. The legacy broad-overuse rent $r_h \text{cost}(A) [h_A - \theta_h]_+^2$ is retained as an ablation baseline (selected by `anti_precedence.mode = broad`); the recommended setting is `anti_precedence.mode = family_protected`.

The recent-use trace updates as

$$h_A(t+1) = \lambda_h h_A(t) + \text{fired}(A, t) + \zeta \text{coactive}(A, t),$$

and is now used both for identity precedence (the positive $\eta_{\text{id}} h_A$ term in candidate scoring) and for computing family shares S_g via aggregation. Family-level traces, V_g , S_g^* , O_g , c_r , and the neighbor-release stimulus R_g are maintained by the kernels listed in §16.6 of the main document and detailed in the pseudocode below.

After wages, rent, and diffusion, STI is renormalized to the global budget; LTI is updated more slowly and homeostatically:

$$\begin{aligned} \text{STI}(A) &\leftarrow B_{\text{STI}} \cdot \frac{\max(0, \text{STI}(A))}{\sum_B \max(0, \text{STI}(B)) + \varepsilon}, \\ \text{LTI}(A) &\leftarrow (1 - \lambda_L) \text{LTI}(A) + \text{Wage}_{\text{LTI}}(A) - \text{Rent}_{\text{LTI}}(A). \end{aligned}$$

I.6 Sparse Hebbian diffusion

For each atom A in the diffusion frontier (hot pool, warm pool, and a bounded one-hop neighborhood), spread at most a fraction m of its STI along its top- k Hebbian neighbors:

$$\Delta \text{STI}(B) = m \text{STI}(A) \cdot \frac{H(A, B)}{\sum_{B' \in N_H(A)} H(A, B') + \varepsilon},$$

and similarly subtract for InverseHebbian neighbors. Hebbian weights are updated from observed coactivity over a recent window:

$$H_{t+1}(A, B) = (1 - \lambda_H) H_t(A, B) + \eta_H \text{CoAct}(A, B, t),$$

where coactivity may include same-tick firing, same candidate application, same ACS membership, same MOSES individual, or same context-local AF.

Diffusion never runs over the whole MORK. The frontier is bounded by construction; this is where the prefix-friendly schemas of §8 pay off most directly.

I.7 Pool migration

Pool migration is the operational output of ECAN. The default rules:

- **Cold** → **warm**: promote when context signature matches the atom, or the atom is strongly Hebbian-associated with a hot atom, or has high LTI for the current task family, or is a near-ACS member with low defect score, or is requested by MOSES/AIRIS/PLN/ACTPC-CHEM.
- **Warm** → **hot**: under strict per-context, per-signature caps. Selected by a hot score that combines STI, context relevance, ACTPC-CHEM support, ACS bonus, novelty quota, and negative cost.
- **Hot** → **warm**: when STI falls below threshold, the active signature disappears, the atom is unused for too many ticks, or its overuse rent exceeds its stimulus.
- **Warm** → **cold**: when the atom is unused for many ticks, no longer associated with any hot atom, not part of an active or near-active ACS, and not requested by any module.
- **Cold** → **forgotten**: when LTI falls below the retention floor and the atom is not part of any archived ACS, lineage, or rare-tail archive. In experimental work, this should mean archival compaction rather than irreversible deletion.

I.8 Hot-pool quotas

The hot pool should not be filled exclusively by current high-utility atoms. To preserve the rare-tail productivity that the benchmark suite rewards, hot-pool slots are partitioned into soft buckets:

high-utility / high-STI:	60-80% of hot pool
high-novelty:	10-20%
rare-tail near-ACS:	5-15%
SB / geoteleomic guidance:	5-10% (when active)

These are soft quotas, not hard laws; the point is to make sure a few high-utility atoms cannot permanently exclude all novelty.

I.9 ECAN-aware retrieval rules

MM2 retrieval rules consume ECAN pool facts. A typical match phase becomes:

```
(exec (40 match-food-hot)
  (, (todo $C $T match)
    (active-sig $C see-food)
    (obs $C see-food $Obj $X $Y)
    (pool-hot $C $R)
    (trigger see-food $R)
    (rule-status $R active))
  (, (cand $C $T $R $Obj $X $Y)))
```

Compared to the ECAN-free version in §8, the only addition is the `(pool-hot $C $R)` atom. Because that fact is context-prefixed, it acts as a strong selectivity gate before the trigger atoms are even consulted.

I.10 Pseudocode: ECAN-lite update

```

procedure ECAN_UPDATE(C, stimulus, W):

    # 1. Pay wages.
    for A, s in stimulus.items():
        STI[A] += wage_sti * positive(s)
        LTI[A] += wage_lti * durable_component(A, s, W)

    # 2. Update recent-use traces (identity precedence).
    for A in hot_pool(C) union warm_pool(C):
        h[A] = lambda_h * h[A] + direct_use(A, W) + zeta * co_use(A, W)

    # 2b. Update family-level traces and shares
    #      (value-relative anti-precedence).
    for g in pattern_families():
        U[g] = (1.0 - lambda_U) * U[g]
            + sum(direct_use(r, W) for r in members(g))
    total_U = sum(U[g] for g in pattern_families()) + eps
    for g in pattern_families():
        S[g] = U[g] / total_U

    # 2c. Compute family value V_g from blended diagnostics
    #      (slow EMA controlled by anti_precedence.value_smoothing).
    for g in pattern_families():
        V_raw[g] = (
            w_utility          * delta_utility(g, W)
            + w_acs_coupling   * acs_coupling_score(g, W)
            + w_generativity    * generativity_score(g, W)
            + w_transfer        * transfer_score(g, W)
            + w_resilience     * resilience_score(g, W)
        ) / (recent_flux(g, W) + cost(g) + eps) \
            - w_cost          * cost(g) \
            - w_lockin        * lockin_score(g, W)
        V[g] = (1 - alpha_V) * V[g] + alpha_V * V_raw[g]

    # 2d. Value-justified share S*_g and excess canalization O_g.
    Z = sum(exp(beta_V * V[g]) for g in pattern_families())
    for g in pattern_families():
        S_star[g] = exp(beta_V * V[g]) / Z
        O[g] = max(S[g] - S_star[g] - theta_overuse, 0.0)

    # 2e. Functional protection per rule.
    for r in rules():
        c[r] = clip(
            functional_payoff_per_flux(r, W) / c0,
            0.0, 1.0
        )

    # 2f. Neighbor-release stimulus into nearby families.
    for h in pattern_families():
        Release[h] = sum(K[g, h] * O[g] for g in pattern_families())

    # 3. Collect rent.

```

```

for A in hot_pool(C) union warm_pool(C):
    STI[A] -= base_rent(A)
    STI[A] -= cost_rent(A)
    if config.anti_precedence.mode == "family_protected":
        g = family_of(A)
        STI[A] -= r_0 * (1.0 - c[A]) * (O[g] ** p_overuse)
        STI[A] += rho_release * Release[g]
    else:
        # legacy "broad"
        STI[A] -= r_h * cost(A) * max(0, h[A] - theta_h)**2
    LTI[A] -= lti_storage_rent(A)

# 4. Sparse STI diffusion over a bounded frontier.
frontier = bounded_frontier(C)
for A in frontier:
    if should_diffuse(A):
        spread_hebbian_sti(A, frontier)
        spread_inverse_hebbian_inhibition(A, frontier)

# 5. Update association links from coactivity and co-catalysis.
UPDATE_HEBBIAN_LINKS(W)

# 6. Normalize budgets.
normalize_STI_budget()
homeostat_LTI_budget()

# 7. Migrate pools.
PROMOTE_COLD_TO_WARM(C)
PROMOTE_WARM_TO_HOT(C)
DEMOTED_HOT_TO_WARM(C)
DEMOTED_WARM_TO_COLD(C)

# 8. Optional welfare/tax to prevent collapse.
if attention_concentration_too_high():
    apply_attention_tax_and_welfare(C)

# 9. Maintain exploration quotas.
enforce_hot_pool_quotas(C)

# 10. For high-V high-O families, ask the outer loop for
#     preserve-function-vary-implementation variants instead
#     of merely paying rent.
for g in pattern_families():
    if V[g] > theta_high_value and O[g] > theta_high_overuse:
        ECAN_TRIGGER_VARIANT_PROPOSALS(C, g)

```

I.11 Pseudocode: worker-level ECAN

```

procedure ECAN_UPDATE_WORKERS(space, epoch):
    workers = query(space, (worker ?W))
    for W in workers:
        util = get_worker_output_utility(space, W)
        novelty = get_worker_output_novelty(space, W)
        acs_yield = get_worker_output_acs_yield(space, W)

```

```

cpu = get_worker_cpu_ms_used(space, W)
mem = get_worker_memory_mb_used(space, W)

stimulus = w_util * util
          + w_novelty * novelty
          + w_acs * acs_yield
          - w_cpu * cpu
          - w_mem * mem

STI[W] += wage_worker_sti * max(0, stimulus)
LTI[W] += wage_worker_lti * durable_worker_component(W, stimulus)

STI[W] -= worker_base_rent(W)
STI[W] -= worker_cpu_rent_scale * cpu
STI[W] -= worker_mem_rent_scale * mem

normalize_worker_sti_budget()
update_worker_hebbian_links_from_coactivation()
end procedure

procedure SCHEDULE_WORKERS(space, ready_workers, hardware_budget):
  scored = []
  for W in ready_workers:
    score = alpha_s * log(STI[W] + eps)
           + alpha_l * log(LTI[W] + eps)
           + alpha_p * get_worker_priority(space, W)
           - k_cpu * requested_cpu_ms(W)
           - k_mem * requested_memory_mb(W)
    scored.append((W, score))

  selected = []
  remaining = hardware_budget
  for W, score in sort_desc(scored):
    if fits_budget(W, remaining):
      selected.append(W)
      remaining = subtract_budget(remaining, W)

  return selected
end procedure

```

I.12 Information-geometric ECAN

Once the ECAN-lite economy is stable, an information-geometric step can be added on the warm/hot subgraph. View STI, suitably normalized, as a probability distribution

$$a_t(A) = \frac{\text{STI}(A)}{\sum_B \text{STI}(B)}.$$

Let $u_t(A)$ be the utility/stimulus vector. A KL-regularized mirror-descent step solves

$$a_{t+1} = \arg \max_{a \in \Delta(\mathcal{B}_t)} \left[\langle u_t, a \rangle - \lambda \text{KL}(a \parallel a_t) \right],$$

which has the closed-form update

$$a_{t+1}(A) \propto a_t(A) \exp(u_t(A)/\lambda).$$

This is the simplest information-geometric ECAN variant. It is cheap, preserves nonnegativity and normalization automatically, and gives a principled alternative to ad hoc rescaling after wages and rent.

A more advanced step adds a Wasserstein transport term over the association graph, using a ground cost ω_{ij} derived from Hebbian strength, structural similarity, co-catalysis frequency, shared ACS membership, or gCodd/CENF similarity:

$$a_{t+1} = \arg \min_{a \in \Delta(\mathcal{B}_t)} \left[-\langle u_t, a \rangle + \lambda \text{KL}(a \| a_t) + \frac{1}{2h} W_\omega^2(a, a_t) + \chi \text{Overuse}(a) \right].$$

This says: move attention toward utility, but do not move too far from the current attention distribution, prefer moving along cheap association paths, and pay a penalty for stale high-traffic pathways. The transport cost can be made history-sensitive in the same way as the rule-space transport cost in §13:

$$\omega_{ij}^{eff}(t) = \omega_{ij} \exp(-\eta \bar{h}_{ij}(t) + \xi \Phi(\bar{h}_{ij}(t); \theta) - \lambda_{SB} \Delta \psi_{ij}(t)).$$

For scalability, the information-geometric step runs only on a bounded subgraph (typically 512–4096 atoms) and only periodically (every 50–500 ticks). It does not run over the whole MORK.

I.13 Coupling ECAN and ACTPC-CHEM

A useful joint regularizer couples the two distributions softly:

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{ACTPC-CHEM}}(p) + \lambda_1 \text{KL}(p \| a_R) + \lambda_2 \text{KL}(a_R \| p) + \Omega_{\text{resource}}(a),$$

where a_R is the ECAN attention distribution restricted to rule atoms. The two KL directions have different meanings:

- $\text{KL}(p \| a_R)$: do not starve rules that ACTPC-CHEM believes are important.
- $\text{KL}(a_R \| p)$: do not waste too much attention on rules unsupported by the model.

Use this coupling gently. ECAN must be allowed to diverge from ACTPC-CHEM for exploration, diagnostics, and rare-tail ACS formation.

I.14 Division of labor: MM2, METTA, and grounded kernels

A natural temptation is to express the entire ECAN economy in METTA or MM2. Resist it. The ECAN update is dominated by operations that pure graph rewriting handles poorly: numeric arithmetic on STI/LTI vectors, weighted random sampling, sparse top- k retrieval over the association graph, KL-regularized mirror descent, and periodically a Wasserstein transport step. These belong in grounded kernels, just as ACTPC-CHEM’s softmax sampling and KL/cross-entropy already do. MM2’s role with respect to ECAN is much more limited: it consumes the kernels’ output and uses it as a selectivity gate.

The clean division is:

Layer	ECAN responsibilities
MORK	Persistent indexed storage of <i>all</i> ECAN facts: (<code>sti R V</code>), (<code>lti R V</code>), (<code>pool R warm</code>), (<code>pool-hot C R</code>), (<code>hebb A B w</code>), (<code>inv-hebb A B w</code>), (<code>overuse-trace R h</code>), (<code>stimulus T R v src</code>), and ACS macro-atoms. Prefix-indexed retrieval of pool-hot atoms during the match phase, where the relation-and-context prefix gives strong selectivity.
MM2	Two things only. (1) <i>Consume</i> pool decisions: the match phase includes (<code>pool-hot \$C \$R</code>) as a conjunct, gating candidate generation to atoms ECAN has admitted. (2) <i>Emit</i> stimulus markers: when a rule fires successfully, contributes to prediction-error reduction, or is part of a winning candidate set, an <code>exec</code> writes a small (<code>stimulus-event T R kind v</code>) atom that the next <code>ECAN_GATHER_STIMULUS</code> call aggregates. MM2 does not compute STI/LTI updates, rent, diffusion, or pool migration.
METTA	High-level configuration and glue: stimulus-weight tables, ECAN policy parameters, retrieval helpers used outside the hot loop, and human-readable definitions of <code>durable_component</code> , hot-pool quotas, and pool-migration thresholds. METTA can also host small-batch retrieval rules used by structural-loop jobs (which are already off the hot path), but <i>never</i> runs the per-tick ECAN update.
Grounded kernels	Everything numeric: <code>ECAN_GATHER_STIMULUS</code> , <code>ECAN_PAY_WAGES</code> , <code>ECAN_COLLECT_RENT</code> , <code>ECAN_DIFFUSE_STI</code> , <code>ECAN_UPDATE_HEBBIAN</code> , <code>ECAN_PROMOTE_DEMOTE</code> , <code>ECAN_INFO_GEOM_STEP</code> , <code>ECAN_FORGET</code> . These read and write MORK atoms but compute in typed numeric structures host-side. The <code>ECAN_INFO_GEOM_STEP</code> kernel is the only one that uses dense linear algebra, and only on the bounded warm/hot subgraph.

Why this matters: a tempting but wrong design has MM2 `execs` walking the entire association graph each tick to update Hebbian weights, or has a METTA interpreter computing softmaxes over STI. Both will dominate runtime within minutes of the first nontrivial benchmark. Keeping ECAN numeric work in kernels and using MM2 only to gate retrieval and emit stimulus markers preserves the bounded-cost property of the inner loop.

There is one operationally important consequence: the (`pool-hot $C $R`) fact is what makes the MM2 hot loop *actually* hot. Without it, the match phase would have to scan (`pool $R hot`) or rule-status atoms across all contexts. With it — because it is context-prefixed and small — MORK’s PathMap index can return the per-context hot rules in essentially $O(K_{\text{hot}})$ time per active signature.

I.15 Grounded ECAN kernels

The following operations live in host kernels rather than pure MM2:

<code>ECAN_GATHER_STIMULUS</code>	aggregate stimulus from all sources
<code>ECAN_PAY_WAGES</code>	apply <code>wage_sti</code> and <code>wage_lti</code>
<code>ECAN_COLLECT_RENT</code>	base/cost/excess-canalization rent
<code>ECAN_UPDATE_OVERUSE_TRACES</code>	decay and bump <code>h_A</code> (identity precedence)
# Family-level value-relative anti-precedence (recommended path).	
<code>ECAN_UPDATE_FAMILY_TRACES</code>	decay and bump <code>U_g</code> , recompute <code>S_g</code>


```

ECAN_COMPUTE_FAMILY_VALUE      blend benchmark diagnostics into V_g
                                with slow EMA
ECAN_COMPUTE_EXCESS_CANALIZATION
                                O_g = [S_g - S*_g - theta_overuse]_+
ECAN_COMPUTE_FUNCTIONAL_PROTECTION
                                c_r per rule from ablation-tested
                                utility per unit flux
ECAN_APPLY_PROTECTED_RELEASE_RENT
                                rent (1 - c_r) * O_pi(r) per atom
ECAN_PAY_RELEASE_BONUSES       credit rho * R_pi(r) into atoms in
                                neighbor families of overused families
ECAN_TRIGGER_VARIANT_PROPOSALS
                                write structural-request atoms for
                                preserve-function-vary-implementation
                                when V_g and O_g are both high

# Original kernels.
ECAN_SPARSE_DIFFUSE_STI        bounded top-k Hebbian/InverseHebbian spread
ECAN_UPDATE_HEBBIAN            coactivity-based H/I weight updates
ECAN_NORMALIZE_BUDGETS         STI budget renormalization, LTI homeostat
ECAN_PROMOTE_COLD_TO_WARM      retrieval-based promotion
ECAN_PROMOTE_WARM_TO_HOT       bucketed top-k under quotas
ECAN_DEMOTE_AND_FORGET         HotWarmCold demotions and archival
ECAN_INFO_GEOM_STEP            mirror-descent / Wasserstein update
ECAN_CREATE_ACS_MACRO_ATOMS    reify recurrent useful ACSs
ECAN_FIND_ATTENTIONAL_ATTRACTORS detect attractor candidates

```

The family-level kernels operate on a much smaller set than the per-atom kernels: the number of pattern families $|\mathcal{G}|$ is typically much smaller than the number of rules, and a single family-trace pass amortizes anti-precedence costs over all members of that family. `ECAN_TRIGGER_VARIANT_PROPOSALS` runs at most once per epoch per family, gated by `anti_precedence.min_family_flux_for_overuse`.

I.16 Memory retrieval examples

A retrieval rule that pulls relevant LTM facts into WM, now ECAN-aware:

```

(exec (35 retrieve-food-memory)
  (, (active-sig $C see-food)
    (ltm-case $Case)
    (ltm-case-feature $Case color-red)
    (sti $Case $S)
    (gt $S retrieval_threshold))
  (, (wm $C retrieved-case $Case)))

```

For large systems, this kind of selective retrieval should be host-side and approximate (e.g. top- k over a Hebbian neighborhood of currently active atoms) rather than driven by symbolic comparisons against a threshold.

J MORK-specific performance strategy

J.1 Hot-path query rules

1. Query by current context first.

2. Query by active signature before rule id if signature is more selective.
3. Query by rule id before metadata if a rule has already been chosen.
4. Split disjoint operations into separate `execs`.
5. Avoid nested logical expressions on the hot path.
6. Cache derived features and meta-guards for the current tick.
7. Use trial and active rule statuses to avoid matching retired rules.
8. Use top- k candidate caps per signature.

J.2 Good and bad schemas

```
; Good: relation and grounded keys first.
(obs C42 see-food 09 2 3)
(trigger see-food R17)
(rule-status R17 active)

; Bad: variable-heavy relation that invites global scans.
(rule-applies $R $C $Obj $X $Y)

; Good: compiled source split into small facts.
(rule-sig R17 see-food)
(rule-guard R17 hand-empty)
(template R17 act-grab)

; Bad: one giant rule blob on the hot path.
(rule R17 (=> (and ... large nested expression ...) (...)))
```

J.3 Candidate caps

Without caps, a rich chemistry can produce too many candidates. Use local caps.

```
(max-candidates C42 see-food 32)
(max-candidates C42 default 128)
(rule-family-budget food-grab 64)
```

A host-side scoring kernel can keep only the top- k candidates per signature.

```
procedure CAP_CANDIDATES(space, ctx, tick):
  grouped <- group candidates by signature
  for sig, cands in grouped:
    k <- get_budget(ctx, sig)
    ranked <- sort_by_score_desc(cands)
    for c in ranked[k:]:
      remove(space, c.atom)
      add(space, (cand-pruned ctx tick c.rule sig))
end procedure
```

J.4 Garbage collection

Transient atoms should carry context and tick fields so they can be deleted by prefix-like queries.

```
(transient C42 T193 cand)
(transient C42 T193 score-input)
(transient C42 T193 sample-request)
```

```
procedure CLEANUP_TRANSIENTS(space, ctx, tick):
  for relation in transient_relations:
    atoms <- query_by_ctx_tick_relation(space, relation, ctx, tick)
    for atom in atoms:
      remove(space, atom)
    add(space, (cleanup-done ctx tick))
end procedure
```

J.5 Sharding

Possible shard keys include context id for online execution, rule family for structural evolution, signature for the rule index, deme id for MOSES, and episode id for replay traces. Do not shard a hot candidate set across too many stores unless cross-shard sampling is implemented.

J.6 PETTA / METTA-IL snapshot-log-reducer concurrency

The safe concurrency pattern is:

```
snapshot -> local worker -> append-only log -> reducer -> global update
```

Near-term, the local worker is dispatched by PETTA. Long-term, the same local worker is dispatched by METTA-IL. In both cases:

1. ECAN or a reducer writes **pool-hot** facts for each context.
2. A context-chamber worker snapshots the context and hot pool.
3. MM2 executes the chamber single-threadedly.
4. The chamber emits an ordered event log.
5. Reducers merge logs into global trace summaries, ECAN stimulus, ACS reports, rule proposals, and bridge updates.

Good PETTA pattern:

```
run_par_group_petta(PG) :-
  par_members(PG, Workers),
  run_workers_in_thread_pool(Workers),
  reduce_par_group_outputs(PG).
```

Good future METTA-IL pattern:

```
(par
  (run-worker W1)
  (run-worker W2)
  (run-worker W3))

(reduce-par-group PG)
```

Bad pattern in either backend:

```
parallel branches mutate the same hot pool directly
```

That pattern should be excluded from benchmark runs that measure ACS/RAF structure, because it makes event causality scheduler-dependent.

K Hardware targets and population sizing

K.1 Why a CPU-focused server is preferred

The core ACTPC-CHEM-on-MORK workload is not primarily a dense linear-algebra workload. The inner-loop operations are more like:

1. ingest symbolic observations into a context-local working memory;
2. activate a small set of currently relevant signatures;
3. retrieve top-k rules per active signature;
4. match compact compiled rule triggers against context facts;
5. score and sample candidate rule applications;
6. fire a small number of reactions;
7. emit actions, predictions, traces, and credit-assignment facts;
8. clean scratch atoms and compact old traces.

The performance drivers are therefore:

- CPU parallelism for many independent contexts, demes, candidate batches, and local rewrites;
- RAM capacity for the persistent rule soup, active indices, context facts, trace buffers, factor graphs, gCodd/CENF structures, and caches;
- memory bandwidth and NUMA behavior, because indexed symbolic operations are often memory-bound;
- fast local NVMe for checkpoints, logs, trace archives, and batch evaluation data;
- stable bare-metal performance for profiling.

A GPU server is useful when the system includes neural perception, embedding-based retrieval, learned rule scorers, batch rollouts, or continuous predictive-coding modules. However, an expensive GPU with limited host RAM is a worse match than a CPU-heavy server with large RAM for the symbolic chemistry substrate.

K.2 Recommended machine classes

K.2.1 Starter engineering server

Enough to build and debug the system, validate MM2 phases, measure atom churn, and run moderate algorithmic chemistry experiments.

CPU:	32-64 physical cores
RAM:	256-512 GB ECC
Storage:	2-4 TB local NVMe
Network:	1-10 Gbps sufficient
GPU:	optional; small or absent is fine
Use case:	implementation, profiling, debugging, first learning runs

Suggested conservative population settings:

persistent rules:	30k - 150k
active rules:	2k - 15k
hot rules/context/tick:	128 - 1024
candidates/tick:	256 - 2048
firings/tick:	1 - 16
MOSES demes:	8 - 32
population/deme:	64 - 256
recent traces:	10k - 500k

K.2.2 Default serious single-server target

Recommended rental target for real experiments once the code is stable.

CPU:	64-96 physical cores, preferably AMD EPYC Genoa/Bergamo/Turin
RAM:	512-768 GB ECC DDR5
Storage:	2 x 4 TB local NVMe preferred
Network:	10 Gbps nice but not essential
GPU:	optional; useful for side modules only
Use case:	serious single-node ActPC-Chem/MORK/MOSES runs

Suggested default population settings:

persistent rules:	150k - 500k
active rules:	15k - 50k
hot rules/context/tick:	1024 - 2048
candidates/tick:	2048 - 4096
firings/tick:	16 - 32
MOSES demes:	32 - 64
population/deme:	256 - 512
recent traces:	500k - 2M

K.2.3 Aggressive single-node target

Useful once the default regime is stable and memory management has been stress-tested.

CPU:	128-192 physical cores
RAM:	1.0-1.5 TB ECC RAM
Storage:	4-8 TB local NVMe
Network:	10-25 Gbps helpful for moving checkpoints/logs
GPU:	optional; 1-4 GPUs useful if neural side workloads are significant
Use case:	scale tests, long runs, many demes, large trace buffers

Suggested aggressive population settings:

persistent rules:	500k - 2M+
active rules:	50k - 150k
hot rules/context/tick:	2048 - 8192
candidates/tick:	4096 - 16384
firings/tick:	32 - 64
MOSES demes:	64 - 128
population/deme:	512 - 1024
recent traces:	2M - 10M

Do not start here. Reach this regime only after the smaller settings have stable profiling, bounded scratch-atom churn, reliable checkpointing, and predictable garbage collection.

K.3 Cold, warm, and hot populations

The implementation should distinguish three different population sizes, and ECAN is what migrates atoms among them:

Cold / persistent soup:

Long-term population of rule records, stored, indexed, scored, canonicalized, checkpointed, and occasionally revived. Membership is set when LTI is below the warm-promotion threshold but above the forgetting floor. Cold atoms can have high LTI and low STI.

Warm / active soup:

Rules with enough current attention, relevance, recency, or credit to be eligible for retrieval and use in the near future. ECAN promotes atoms here from cold based on context-signature matches, Hebbian neighborhoods of hot atoms, near-ACS membership, and explicit module requests.

Hot / MM2 working set:

Context-local bounded subset of eligible rules actually admitted into a cognitive tick as candidate reaction participants. ECAN promotes atoms here from warm based on STI, context relevance, ACS membership, and bucket quotas (utility / novelty / rare-tail / SB-guided).

This is the central scaling idea. The whole soup should not be chemically active inside MM2 at every point in time. A large persistent soup is the reservoir; the hot loop is a small reaction chamber; ECAN is the circulatory system that decides what enters the chamber. A reasonable ratio among the three levels is:

cold : warm : hot $\sim$ 1000 : 100 : 1

More concretely:

Regime	Cold rules	Warm rules	Hot rules/context/tick
Smoke test	1k–5k	256–1k	32–128
Small run	10k–30k	1k–3k	128–512
Default run	150k–500k	15k–50k	1024–2048
Aggressive run	500k–2M+	50k–150k	2048–8192

The Hebbian and InverseHebbian association graphs add a smaller but non-trivial memory cost on top of the rule records: at 16–64 Hebbian and 8–32 InverseHebbian edges per atom, a 500k-atom warm soup carries $\sim$ 30M to $\sim$ 50M association edges. At 64 bytes per edge record that is on the order of a few GB — comfortable on the recommended hardware, but a real budget item that should be tracked alongside trace memory.

K.4 Inner MM2-loop sizing

K.4.1 Recommended hot-loop caps

The inner loop should be explicitly capped. A default serious setup is:

```

active_signatures_per_context = 8 - 32
rules_per_signature_cap      = 32 - 64
hot_rules_per_context        = 1024 - 2048
candidate_cap_per_tick       = 2048 - 4096
fire_cap_per_tick            = 16 - 32
working_memory_fact_cap      = 8192 - 32768
scratch_atom_cap_per_tick    = 25000 - 50000
trace_ttl_ticks              = 4096 - 8192

# ECAN-related caps
ecan_max_hebb_edges_per_atom = 16 - 64
ecan_max_inv_edges_per_atom  = 8 - 32
ecan_diffusion_depth         = 1 - 2 hops
ecan_max_spread_fraction     = 0.05 - 0.20 of STI
ecan_info_geom_subgraph      = 512 - 4096 (when enabled)
ecan_info_geom_period_ticks  = 50 - 500 (when enabled)

# Hot-pool quotas (soft)
hot_quota_utility            = 0.60 - 0.80
hot_quota_novelty            = 0.10 - 0.20
hot_quota_rare_tail          = 0.05 - 0.15
hot_quota_sb_guided          = 0.05 - 0.10

```

```

# Portable worker / PeTTa orchestration caps
worker_backend_initial       = petta
worker_backend_target        = metta-il
worker_context_chambers     = 32 - 128
worker_moses_demes          = 16 - 64
worker_acs_scans             = 4 - 32
worker_replay_batches        = 4 - 64
worker_bridge_jobs           = 0 - 8
worker_ecan_epochs           = 1 - 4
worker_max_par_group_size    = 32 - 256
worker_default_chamber_budget_ms = 50
worker_reduce_period_ticks   = 1 - 10
allow_shared_hot_pool_mutation = false

```

These numbers are not additional MM2 threads inside one chamber. They are coarse-grained workers. A 96-core server might run dozens of context-chamber workers plus several demes and scan workers concurrently, but each chamber worker still runs one ordered MM2 loop over its assigned hot pool.

Candidate generation should be bounded *before* it explodes. The system should not try every rule against every observation.

K.4.2 MM2 tick skeleton with bounding

```

procedure INNER_TICK(context C):

  OBS = read_current_observations(C)

  ACTIVE_SIGS = grounded_active_signature_extractor(OBS)
  ACTIVE_SIGS = top_k(ACTIVE_SIGS, K_SIGS_PER_CONTEXT)

```

```

candidate_buffer = empty

for sig in ACTIVE_SIGS:

    RULES_SIG = retrieve_rules_by_trigger(sig)
    RULES_SIG = top_k(
        RULES_SIG,
        K_RULES_PER_SIG,
        key = attention + logit + recent_credit - cost_penalty)

    OBS_SIG = observations_with_signature(C, sig)

    for obs in OBS_SIG:
        CANDS = match_compiled_triggers(obs, RULES_SIG)
        append CANDS to candidate_buffer

        if size(candidate_buffer) >= CANDIDATE_CAP:
            break

    if size(candidate_buffer) >= CANDIDATE_CAP:
        break

SCORED = grounded_score_candidates(candidate_buffer)
CHOSEN = softmax_or_topk_sample(SCORED, FIRE_CAP)

for app in CHOSEN:
    fire_rewrite_application(app)
    write_trace(app)
    update_short_term_attention(app)

delete_scratch_candidates(C)
compact_expired_traces(C)

```

K.4.3 Why not all rules hot?

If the system has R rules, O observations, and naive matching tries all rules against all observations, the hot loop tends toward an $O(RO)$ scan. This is fatal once R reaches hundreds of thousands. The scalable form converts the practical hot-loop size from the global rule count to:

```
0(active_signatures * rules_per_signature_cap * observations_per_signature)
```

which is bounded by configuration. This is why the schema design in §8 insists on relation-and-grounded-key prefixes: it is what makes the bounded form actually bounded.

K.5 Persistent rule-soup memory sizing

A compact rule record should be split into multiple small indexed atoms rather than one giant expression. A typical rule may require:

```

rule metadata atoms:      5 - 10
trigger/index atoms:     1 - 5
template/compiled atoms: 2 - 10

```


statistics atoms:	5 - 20
gCoDD/CENF references:	1 - 5
optional neighbor links:	0 - 32

A rough total is 15–50 atoms per persistent rule, before traces and neighbor expansion. Thus:

100k rules	-> 1.5M - 5M rule-related atoms
500k rules	-> 7.5M - 25M rule-related atoms
1M rules	-> 15M - 50M rule-related atoms

This is plausible on a large server, but only if symbol interning is efficient, hash-consing avoids duplicate structure, traces are bounded and compressed, scratch atoms are aggressively deleted, cold rules can be frozen, checkpointed, or unloaded, and indexes are profiled and tuned.

A good rule of thumb is that the persistent rule soup should not consume more than about 40–50 percent of RAM. The rest is needed for active indices, working memory, trace buffers, demes, factor graphs, checkpoints, logs, and OS cache.

K.6 MOSES/gCoDD/CENF outer-loop sizing

The outer structural-evolution layer should run at a slower rhythm than the inner ACTPC-CHEM loop. It creates, mutates, abstracts, canonicalizes, and retires rules.

K.6.1 Default MOSES parameters

MOSES_demes	= 32 - 64
population_per_deme	= 256 - 512
elite_per_deme	= 32 - 64
structural_epoch_ticks	= 200 - 1000
EDA_variables_per_deme	= 64 - 256
factor_arity_cap	= 4 initially
nary_factor_cap	= 128 - 512
CENF_period_generations	= 5 - 20
migration_period_generations	= 10 - 50

A good first serious choice is:

MOSES_demes	= 32
population_per_deme	= 256
elite_per_deme	= 32
structural_epoch_ticks	= 200
EDA_variables_per_deme	= 192
factor_arity_cap	= 4

K.6.2 Outer-loop rhythm

Every inner tick:
 update rule logits, traces, local attention, and immediate credit.

Every 10-50 ticks:
 run delayed credit assignment and pattern-count updates.

Every 200-1000 ticks:

```

run structural epoch:
  select high-credit traces/rules;
  extract rule fragments;
  mutate and recombine gCoDD/CENF forms;
  evaluate candidates;
  insert new rules with low/moderate priors;
  retire weak/cold active rules.

```

Every several structural epochs:

```

  run CENF/meta-guard updates;
  update factor graph structures;
  migrate exemplars among demes.

```

K.7 Trace and checkpoint requirements

Trace management can dominate memory unless capped. The trace system should support three levels:

Hot traces:
Detailed, recent, RAM-resident traces for delayed credit assignment.

Warm trace summaries:
Aggregated statistics by rule, signature, context, reward pattern, prediction-error pattern, and causal discrepancy.

Cold trace archives:
Compressed logs on NVMe for offline analysis, reproducibility, debugging, and later batch learning.

Suggested caps:

Starter:

```

recent_trace_cap = 100k - 250k
trace_ttl_ticks  = 1024 - 4096

```

Default:

```

recent_trace_cap = 500k - 2M
trace_ttl_ticks  = 4096 - 8192

```

Aggressive:

```

recent_trace_cap = 2M - 10M
trace_ttl_ticks  = 8192+

```

Checkpointing should write the persistent rule table and canonical IDs; rule logits, costs, attention values, and utility statistics; active/warm rule indices; MOSES deme states and factor graphs; CENF/meta-guard definitions; recent trace summaries; random seeds and configuration; and environment/simulator state if applicable.

K.8 Role of GPUs

A GPU is optional but can help substantially if assigned the right jobs.

Good GPU tasks:

```
continuous predictive-coding perception/action modules
image/audio/sensor encoders
embedding generation for states, contexts, and rules
approximate nearest-neighbor retrieval over vectorized rules
learned rule utility scoring
batch environment rollouts
batch MOSES fitness evaluation
some factor-graph or sampling kernels
LLM-assisted rule proposal or explanation, if used locally
```

Poor GPU tasks:

```
whole-soup symbolic matching
fine-grained irregular graph rewrites
massive random atom insertion/deletion
pointer-heavy MORK index traversal
unbounded scratch-atom churn
```

The symbolic substrate should remain CPU/RAM-centric. GPU work should be batched and coarse-grained.

K.9 Bare metal versus cloud VM

Bare metal is preferred for the first serious MORK/MM2 profiling runs. Bare-metal advantages: predictable NUMA and memory-bandwidth behavior; no noisy-neighbor effects; direct local NVMe performance; easier CPU pinning and process layout; clearer profiling of MORK indexes and garbage collection; often cheaper for long-running high-RAM CPU workloads. Cloud VM advantages: easier provisioning and teardown; simpler scaling experiments; easier snapshotting and integration with managed storage; useful for short tests or when bare-metal procurement is slow. For the main research run, prefer bare metal if available. Use cloud VMs for short experiments, integration testing, and reproducibility checks.

K.10 NUMA and process layout

On a dual-socket or many-core EPYC system, NUMA matters. The implementation should avoid a single shared global hot data structure that all threads mutate randomly. Better patterns:

1. shard contexts by NUMA node;
2. shard MOSES demes by NUMA node;
3. keep context-local scratch atoms local;
4. batch cross-shard migrations;
5. use per-shard candidate buffers and trace buffers;
6. merge global statistics periodically, not every tick;
7. pin worker threads to cores or NUMA regions during profiling.

A natural layout:

```
NUMA node 0:
  contexts 0..K
  demes 0..D0
  local rule-trigger cache
  local scratch and trace buffers
```

```

NUMA node 1:
    contexts K+1..M
    demes D0+1..D
    local rule-trigger cache
    local scratch and trace buffers

Global service:
    cold rule store
    checkpoint writer
    global statistics aggregator
    rule retirement/insertion manager

```

K.10.1 Worker placement

When the PETTA or METTA-IL runtime maps workers onto hardware threads, it should use the following placement preferences:

1. context-chamber workers near their context-local MORK view;
2. MOSES deme workers near their deme state and factor graph;
3. ACS scan workers near event-log shards;
4. ECAN local reducers near the stimulus logs they consume;
5. bridge workers near summary trajectory buffers;
6. global reducers on fewer threads, running less frequently.

A default 96-core allocation target:

```

48-64 cores: context-chamber workers
8-16 cores:  MOSES / CENF / structural workers
8-16 cores:  ACS scans and replay/ablation workers
4-8 cores:   ECAN reducers and worker-level scheduler
2-8 cores:   bridge fitting, logging, checkpointing

```

K.11 Autotuning policy

All caps should be adaptive. A simple policy:

```

procedure AUTOTUNE(metrics):

    if p95_tick_ms > tick_budget_ms:
        K_RULES_PER_SIG      *= 0.8
        CANDIDATE_CAP        *= 0.8
        FIRE_CAP              = max(1, floor(0.8 * FIRE_CAP))
        STRUCTURAL_EPOCH     *= 1.25

    if p95_tick_ms < 0.5 * tick_budget_ms
       and memory_used < memory_soft_cap
       and candidate_cap_not_saturated:
        K_RULES_PER_SIG      *= 1.1

```

```

CANDIDATE_CAP      *= 1.1

if candidate_count == CANDIDATE_CAP too often:
    improve trigger specificity
    reduce rules_per_signature
    add better active-signature gating

if rule_entropy < entropy_floor:
    increase temperature
    increase epistemic bonus
    increase mutation rate slightly

if memory_used > memory_soft_cap:
    retire cold rules
    compress old traces
    freeze low-utility demes
    checkpoint and unload cold clusters

if new_rule_acceptance_rate < threshold:
    reduce mutation volume
    improve MOSES proposal features
    increase selection pressure

if prediction_error_plateau persists:
    trigger AIRIS/PLN/MOSES structural proposal epoch

# ECAN-specific autotuning

if hot_pool_precision < precision_floor:
    increase warm_to_hot threshold
    tighten hot_quota_utility
    reduce K_rules_per_sig

if rare_tail_enrichment < 1.0:
    increase hot_quota_rare_tail
    increase ACS dividend weight on rare-tail precursors
    decrease anti_precedence.overuse_threshold
        (release over-canalized families sooner)

if attention_concentration_HHI > monopoly_threshold:
    increase anti_precedence.anti_scale
    increase anti_precedence.release_scale
    increase InverseHebbian weight from dominant atoms
    widen hot_quota_novelty band

if ecan_overhead_fraction > overhead_budget:
    reduce max_hebb_edges_per_atom
    reduce ecan_diffusion_depth
    increase info_geom_period_ticks
    skip info_geom step when warm pool is small

if pool_churn > churn_threshold:
    increase warm_idle_ticks_before_demote
    smooth ECAN wages over a longer window

```

raise STI promotion thresholds slightly

K.12 Operational acceptance criteria

Before scaling beyond Stage 2 of the scaling sequence (§20.3), the system should satisfy:

1. p95 tick latency remains under the configured budget for several hours;
2. scratch atom count returns to baseline after cleanup;
3. trace memory is bounded and old traces compress correctly;
4. rule insertion and retirement do not fragment indexes pathologically;
5. checkpoint + restore reproduces rule logits and active indices;
6. MOSES demes can be paused, resumed, migrated, and retired;
7. CENF/meta-guard updates do not break canonical IDs;
8. candidate cap saturation is rare or diagnostically understood;
9. CPU utilization is high without catastrophic lock contention;
10. memory bandwidth, not a single lock or global queue, is the main bottleneck.

L Evaluation: useful autocatalytic generativity

L.1 Benchmark philosophy

The system’s strength is not ordinary task-optimized machine learning, but *interesting generativity*. The chemistry should evolve networks of rules and artifacts that are alive in a minimal operational sense: they maintain themselves, catalyze each other’s production, generate novel useful variants, survive perturbation and transfer, and contribute causally to external work.

A benchmark suite for this purpose should not ask only:

Did the soup solve the task?

It should ask:

Did the soup evolve active autocatalytic subnetworks whose ablation damages useful generative behavior, whose serial transfer preserves function, whose rare fringe contributes new useful structures, and whose overused habits are released before they congeal into sterile repetition?

This distinction is crucial. A fixed circuit, a memorized solution, or an over-specialized policy can solve a narrow benchmark while lacking the self-maintaining generativity that algorithmic chemistry is supposed to bring. The benchmark suite therefore treats external utility as only one axis of success. The other axes measure autocatalysis, coupling between autocatalysis and utility, generativity, heredity, resilience, and anti-rigidity.

L.2 Pareto objective, not a single fitness function

A simple scalar objective such as

$$\text{Score} = U + \lambda A$$

where U is external utility and A is degree of autocatalysis is too easy to pervert. The system can score well by keeping the two achievements separate: a task circuit for U and a decorative autocatalytic loop for A .

Instead maintain a Pareto archive over at least the following axes:

A = active autocatalysis,
 U = external utility,
 C = autocatalysis–utility coupling,
 G = generativity / novelty / diversity,
 H = heredity / transfer stability,
 R = resilience / anti-rigidity,
 K = cost / complexity.

Maximize A, U, C, G, H, R and minimize K and brittleness. Apply hard filters such as

$$A \geq A_{\min}, \quad C \geq C_{\min}, \quad H \geq H_{\min}, \quad U_{\text{heldout}} \geq U_{\min}.$$

L.2.1 Autocatalysis–utility coupling

A useful coupling score is:

$$C(S) = \frac{U(\text{soup}) - U(\text{soup} \setminus S)}{U(\text{soup}) + \epsilon} \cdot \frac{\text{RewardFlux}(S)}{\text{RewardFlux}(\text{soup}) + \epsilon}.$$

Here $U(\text{soup} \setminus S)$ is measured by ablating the candidate ACS or suppressing its members and rerunning or replaying matched task episodes. The second factor measures how much reward-attributed reaction flux actually passes through S . This score asks whether the autocatalytic set is doing the useful work.

L.3 Core diagnostics

Every benchmark in the suite should produce the same dashboard.

L.3.1 ACS / RAF metrics

```

active_ACS_count
largest_active_ACS_size
ACS_births_per_1000_ticks
ACS_lifetime_distribution
active_ACS_fraction
catalysis_coverage
food_generation_coverage
defect_score_distribution
  
```

L.3.2 Usefulness metrics

```

external_task_score
heldout_task_score
reward_flux_through_ACS
ACS_ablation_delta_U
non_ACS_ablation_delta_U
  
```

L.3.3 Generativity metrics

```
valid_novel_artifacts
artifact_diversity
behavioral_diversity
rule_family_diversity
rare_tail_enrichment_E20
number_of_new_reusable_intermediates
```

L.3.4 Heredity metrics

```
parent_daughter_JSD
serial_transfer_success
dilution_recovery_time
composition_drift_per_generation
```

L.3.5 Anti-rigidity metrics

```
top_rule_ablation_drop
top_k_path_ablation_drop
alternative_path_count
task_distribution_shift_score
sclerosis_index
```

L.3.6 Attention-economy metrics

These measure whether ECAN is doing useful work, in addition to or instead of contributing usefully to the ACS and utility metrics.

```
hot_pool_precision      = #hot atoms used in useful events
                        / #hot atoms

promotion_recall         = #useful atoms promoted before needed
                        / #useful atoms eventually needed

pool_churn               = atoms migrating per 1000 ticks
                        (too low = stale; too high = thrashing)

mm2_candidate_waste     = #candidates discarded by sampler
                        / #candidates generated

attention_concentration_HHI = sum_A sti(A)^2 (Herfindahl on STI)

attention_acs_overlap    = fraction of STI mass on active ACS members

ecan_overhead_fraction  = ECAN kernel time / total tick time
```

Operationally, `hot_pool_precision` is the most useful single diagnostic: a hot pool that holds many atoms which never participate in fired candidates is a sign that promotion is too generous, the hot-pool quotas are off, or the Hebbian neighborhood is too dense.

L.3.7 Near-ACS defect score

Adapt the near-ACS defect score from the geoteleomic origin-of-life work (*Reverse-Engineering the Origin of Life*, Goertzel):

$$D(H; t) = w_{cat} d_{cat}(H) + w_{food} d_{food}(H) + w_{act} d_{act}(H; t) + w_{mass} d_{mass}(H; t).$$

For algorithmic chemistry:

$$\begin{aligned} d_{cat} &= \text{missing catalyst support,} \\ d_{food} &= \text{missing food-generated closure,} \\ d_{act} &= \text{insufficient repeated firing,} \\ d_{mass} &= \text{insufficient abundance, attention or trace mass.} \end{aligned}$$

$D = 0$ denotes an active ACS-like local patch. $D > 0$ denotes an almost-ACS motif. The fringe distribution over these motifs should be monitored, because the rare fringe may be disproportionately productive even when most births come from the common head.

L.3.8 Rare-tail enrichment

For $p \in \{0.10, 0.20, 0.30\}$ define:

$$E(p) = \frac{\text{birth precursor share among rarest } p \text{ fraction of motif types}}{\text{occupancy share of rarest } p \text{ fraction of motif types}}.$$

If $E(p) > 1$, the rare fringe is productive relative to its occupancy. If $E(p) \approx 1$, the tail is decorative. If $E(p) < 1$, the rare fringe is less productive than expected.

L.3.9 Sclerosis index

A rigid circuit often has concentrated flux, high ablation fragility, and little productive rare tail. A useful autocatalytic ecology may also be head-dominated, but it should remain resilient and tail-productive. A rough sclerosis index is:

$$\text{Sclerosis} = \frac{\text{HHI}(\text{reaction flux}) \cdot \text{AblationFragility}}{1 + E_{20}}.$$

This does not punish concentration alone. It punishes brittle concentration without rare-tail productivity.

L.4 Benchmark series

The suite ramps from pure ignition to useful symbolic generativity and then toward self-improving program ecologies. Each stage should be simple enough to instrument exhaustively but rich enough to require emergent networks rather than a single fixed rule.

L.4.1 Benchmark 0: ACS ignition and habit-trap calibration

Tune the chemistry and diagnostics before external task pressure is added.

World:
 no meaningful external task yet

Food:
 primitive symbolic atoms
 simple constructors
 random seed rules

Goal:
 produce active, persistent autocatalytic rule sets

External reward:
 none, or weak reward for maintaining nontrivial active closure

Main question:
 can the soup ignite active ACSs at all?

Run conditions:

A: native ActPC only
 B: ActPC + SB / geoteleomic guidance
 C: ActPC + precedence (positive reuse bias)
 D: ActPC + precedence + broad anti-resonance (legacy baseline)
 D': ActPC + precedence + value-relative
 anti-precedence + functional protection (recommended)

This benchmark identifies the phase diagram of the soup. It is also where one checks that the detector is not confusing trivial loops with genuine active closure, and where the broad and protected anti-precedence variants can first be compared on a clean ignition problem.

L.4.2 Benchmark 1: productive grammar ecology

Recommended first useful benchmark.

World:
 symbolic artifact factory

Food:
 primitive tokens
 constructors
 simple rewrite operators
 type symbols
 base grammar fragments

Products:
 strings, trees, small MeTTa expressions, typed symbolic artifacts

External reward:
 produce valid, novel, nontrivial artifacts from a family,
 not one target artifact

Complexity ramp:

Level 1: finite-state grammar strings
 Level 2: balanced parentheses / simple context-free grammar

```
Level 3: typed expression trees
Level 4: small executable MeTTa-like expressions that evaluate without error
Level 5: small expressions satisfying randomized semantic properties
```

A useful ACS may contain:

```
generators      -> create candidate expressions
validators      -> reject malformed expressions
repair rules    -> transform near-valid into valid
abstraction rules -> create reusable grammar motifs
samplers       -> vary motifs productively
```

A simple utility function is:

$$U = \#ValidNovelArtifacts + \alpha \text{SemanticUsefulness} + \beta \text{Diversity} - \gamma \text{Triviality}.$$

Do not reward exact strings. Reward valid families and reusable generative motifs.

L.4.3 Benchmark 2: autocatalytic abstraction discovery

A stripped-down food/poison style benchmark without a full organism.

```
World:
  stream of objects with symbolic features

Food:
  raw feature atoms:
    color, shape, texture, smell, context, time

Products:
  learned categories, meta-guards, causal/predictive rules

External reward:
  predict hidden outcomes or choose symbolic labels/actions
```

Complexity ramp:

```
Level 1: single-feature rule
Level 2: XOR-like feature conjunctions
Level 3: noisy feature conjunctions
Level 4: delayed outcomes
Level 5: context-dependent delayed outcomes
Level 6: nonstationary concept drift
```

Example hidden regularities:

```
red + round + smooth -> beneficial
red + round + rough  -> harmful
purple + sweet + active-after-ingestion -> beneficial
purple + sweet + inactive-after-ingestion -> harmful
```

The goal is not to build a good classifier but to evolve an ecology of abstraction-making rules in which feature-combiners catalyze category rules; category rules catalyze prediction rules; prediction errors catalyze refinements; and successful categories become food for higher-level categories.

L.4.4 Benchmark 3: lemma / proof autocatalysis

Perhaps the cleanest benchmark for useful symbolic generativity.

```
World:
    tiny theorem-proving or equational-reasoning environment

Food:
    axioms, primitive inference rules, rewrite identities

Products:
    lemmas, proof macros, derived rewrite rules, proof plans

External reward:
    solve randomly generated proof goals,
    shorten proofs,
    generate reusable lemmas
```

Complexity ramp:

```
Level 1: propositional tautology fragments
Level 2: term-rewrite simplification
Level 3: equality proofs in tiny algebraic theories
Level 4: induction-free list identities
Level 5: small program invariants or type-checking lemmas
```

Lemmas are naturally catalytic: they enable proof steps without being consumed. A lemma ecology may look like:

```
Lemma A enables proof macro M
Macro M derives Lemma B
Lemma B enables simplification rule R
R makes Macro M applicable in more contexts
```

External reward should be sampled from a stream of related proof goals, not one fixed theorem. Useful metrics: proof success, proof-length reduction, lemma reuse, lemma-ACS ablation delta, heldout transfer, and serial-transfer JSD over the lemma/rewrite-rule composition.

L.4.5 Benchmark 4: program-fragment repair and synthesis ecology

Connects naturally to MOSES/GCoDD/CENF but should remain focused on generative ecology rather than leaderboard-style program induction.

```
World:
    small typed programs with holes, bugs, or missing transformations

Food:
    primitive operations, type constructors, examples, tests

Products:
    repair rules, program fragments, reusable subroutines,
    candidate programs

External reward:
    repair or synthesize small programs across a task family
```

Complexity ramp:

```
Level 1: list/string transformations
Level 2: tree transformations
Level 3: symbolic arithmetic simplifiers
Level 4: small planner fragments
Level 5: meta-repair: rules that repair repair rules
```

Example useful ACS:

```
type-checker fragments catalyze repair fragments;
repair fragments produce candidate programs;
candidate programs expose reusable subfragments;
subfragments become catalysts for future repairs;
CENF/meta-guards compress recurring conditions.
```

Anti-rigidity checks: new task families after training, heldout primitive combinations, ablation of top repair rules, changed type signatures, reward for reuse and diversity rather than pass rate alone.

L.4.6 Benchmark 5: blackboard protocol alchemy

A minimal test of network functionality without introducing a society of organisms or machines.

```
World:
    shared blackboard of jobs, messages, resources, and products

Food:
    incoming request tokens, resource tokens, primitive transformations

Products:
    completed jobs, routing patterns, resource-allocation motifs,
    reusable service protocols

External reward:
    maintain useful service under changing demand
```

Example request stream:

```
simplify expression
classify object
prove small lemma
generate valid artifact
repair small program
```

Rules can transform requests, decompose requests, create helper jobs, route jobs to specialized rule clusters, cache intermediate concepts, and recycle failed attempts. The main metric:

$$\frac{\text{useful service throughput causally routed through active ACSs}}{\text{total useful service throughput}}.$$

L.4.7 Benchmark 6: serial transfer and daughter-soup inheritance

A protocol applied to Benchmarks 1–5.

1. identify active useful ACSs
2. dilute the soup
3. transfer only:
 - food set + selected ACS members + small random background
4. run on fresh task samples
5. measure:
 - recovery time
 - parent/daughter JSD
 - utility retention
 - novelty retention
 - new ACS births

Rigid circuits often fail this test. Compositional ecologies should recover function and preserve recognizable organization under dilution and transfer.

L.5 Geoteleomic guidance, precedence, and value-relative anti-precedence

This subsection mirrors the conceptual exposition in §19.5 of the main document, with implementation-oriented detail. We summarize the equations so the appendix is self-contained.

L.5.1 Three-layer control

The recommended rule-selection biasing has three layers:

1. **Geoteleomic guidance** ($\Delta\psi_r$): a SB / Doob tilt toward viable-future endpoints (active useful ACSs, transferable artifact families, replicating program ecologies).
2. **Precedence** ($+\eta_{\text{id}}h_r$, plus a functional-memory term $+\eta_{\text{func}}c_rH^{\text{func}}$): use traces make recently productive paths slightly easier to traverse; functional machinery is rewarded for being reused.
3. **Value-relative anti-precedence** ($-\xi(1 - c_r)O_{\pi(r)}^p + \rho R_{\pi(r)}$): penalize a pattern family only when its realized share exceeds its value-justified share, with shared functional machinery protected and pressure redirected into nearby implementations.

L.5.2 Family-level quantities

The ECAN controller maintains, per family $g \in \mathcal{G}$:

$$\begin{aligned}
 U_g(t) &= \int_0^t e^{-\lambda_U(t-s)} dN_g(s), & S_g(t) &= \frac{U_g(t)}{\sum_h U_h(t) + \epsilon}. \\
 V_g(t) &\approx \frac{w_U \Delta U_g + w_C C_g + w_G G_g + w_H H_g + w_R R_g^{\text{res}}}{J_g + \text{Cost}_g + \epsilon} - w_K K_g - w_B B_g. \\
 S_g^*(t) &= \frac{\exp(\beta V_g(t))}{\sum_h \exp(\beta V_h(t))}, & O_g(t) &= [S_g(t) - S_g^*(t) - \theta_O]_+. \\
 R_h(t) &= \sum_g K(g, h) O_g(t), & K(g, h) &\propto \exp(-d(g, h)/\tau).
 \end{aligned}$$

The neighbor metric $d(g, h)$ blends source edit distance, gCoDD distance, guard-set distance, behavioral distance, and cost — the same components used in the rule-space graph for OT.

L.5.3 Functional-protection coefficient

Per rule r :

$$c_r(t) \approx \text{clip}\left(\frac{\Delta U_r(t)}{(J_r(t) + \epsilon) c_0}, 0, 1\right).$$

A more robust estimator uses ablation-tested utility per unit flux, averaged across task families and contexts. Validators, repair rules, abstraction tools, and shared meta-guards typically end up with c_r near 1; implementation-specific specialized rules end up near 0.

L.5.4 Effective rule-selection probability

The recommended form:

$$\pi_r^{eff}(t) \propto \pi_r^{PC}(t) \exp\left(\Delta\psi_r(t) + \eta_{\text{id}} h_r(t) + \eta_{\text{func}} c_r(t) H^{\text{func}}(t) - \xi (1 - c_r(t)) O_{\pi(r)}(t)^p + \rho R_{\pi(r)}(t)\right).$$

The legacy broad-anti-resonance form

$$\pi_r^{eff}(t) \propto \pi_r^{PC}(t) \exp(\Delta\psi_r(t) + \eta h_r(t) - \xi \Phi(h_r(t); \theta)), \quad \Phi(h; \theta) = [h - \theta]_+^2,$$

is retained as a baseline for ablation only.

L.5.5 Anti-precedence on rule-space transport

The OT module of §13 can use the history-sensitive ground cost

$$\omega_{ij}^{eff}(t) = \omega_{ij} \exp\left(-\eta \bar{h}_{ij}(t) + \xi (1 - \bar{c}_{ij}(t)) O_{\pi(i)}(t)^p - \rho R_{\pi(j)}(t) - \lambda_{SB} \Delta\psi_{ij}(t)\right),$$

where $\bar{c}_{ij}$ averages the functional-protection coefficient on the edge.

L.5.6 Trace updates

$$h_r(t+1) = (1 - \lambda_h) h_r(t) + I_r(t), \quad U_g(t+1) = (1 - \lambda_U) U_g(t) + \sum_{r \in g} I_r(t).$$

V_g is updated with a slow EMA (config: `anti_precedence.value_smoothing`) so that anti-precedence does not chase noise.

L.6 Experimental matrix

For each benchmark, run at least the following conditions. The first group varies the geoteleomic / precedence / anti-precedence axis along which the most important design refinement (§19.5) operates. The second group varies the attention mechanism. Cross both axes for the most informative ablation.

Condition	Description
A	ACTPC-CHEM only (static top- k retrieval, no ECAN)
B	ACTPC-CHEM + external reward
C	ACTPC-CHEM + external reward + ACS archive criterion
D	ACTPC-CHEM + SB / geoteleomic guidance
E	ACTPC-CHEM + SB + precedence (positive reuse bias)
F	ACTPC-CHEM + SB + precedence + broad biphasic anti-resonance (legacy baseline)
G	ACTPC-CHEM + SB + broad anti-resonance only (legacy baseline)
F'	ACTPC-CHEM + SB + family-level value-relative anti-precedence (no protection)
F''	ACTPC-CHEM + SB + family-level value-relative anti-precedence + functional protection (recommended)
F'''	ACTPC-CHEM + SB + family-level value-relative anti-precedence + functional protection + structural protection
H	ACTPC-CHEM + ECAN-lite pool manager
I	ACTPC-CHEM + ECAN-lite + ACS dividends
J	ACTPC-CHEM + ECAN-lite + ACS dividends + broad-overuse rent (legacy baseline)
J'	ACTPC-CHEM + ECAN-lite + ACS dividends + value-relative excess-canalization rent + functional protection
K	ACTPC-CHEM + info-geometric ECAN on warm/hot subgraph
L	ACTPC-CHEM + info-geometric ECAN + SB / geoteleomic guidance
M	ACTPC-CHEM + info-geometric ECAN + protected anti-precedence

The most informative paired contrasts:

- F'' vs. F: does targeting excess canalization rather than use as such avoid the loss of useful function?
- F''' vs. F'': does structured release into nearby implementations strictly improve over plain functional protection?
- J' vs. J: same comparison inside the ECAN attention economy.
- M vs. L: does protected anti-precedence add value on top of info-geometric ECAN with SB guidance?
- Bench 7 (adaptive continuity under implementation change), F''' vs. F: should be the strongest single comparison if the v8 Lane-seep result transfers.

L.7 Benchmark harness pseudocode

```

procedure RUN_BENCHMARK(config):
    initialize_soup(config.food_rules, config.random_seed_rules)

    for epoch in 1..N:
        task_batch = sample_tasks(config.task_distribution)

        for task in task_batch:
            reset_working_memory(task)

```



```

for tick in 1..T:

    observations = read_task_state(task)

    active_signatures = extract_active_signatures(observations)

    candidates = bounded_mm2_candidate_generation(
        active_signatures,
        hot_rule_caps
    )

    chosen = score_and_sample_candidates(candidates)

    fire_reactions(chosen)

    update_traces(chosen)

    log_events(
        tick, task, chosen_rules,
        inputs, outputs, catalysts,
        rewards, prediction_errors
    )

    update_actpc_weights()

    maybe_apply_resonance_antiresonance()

    maybe_apply_structural_mutation_or_moses_epoch()

windows = sliding_windows(event_log)

for W in windows:

    actual_graph = build_actual_reaction_graph(W)

    near_acs_motifs = enumerate_or_sample_near_acs(actual_graph)

    acs_list = detect_active_useful_acs(
        actual_graph, food_set, activity_thresholds
    )

    for S in acs_list:

        A = active_autocatalysis_score(S, W)
        U = external_utility_score(S, W)
        C = utility_coupling_score(S, W)
        G = generativity_score(S, W)
        H = heredity_score(S, W)
        R = resilience_score(S, W)
        K = complexity_cost(S)

        if passes_minimum_filters(A, C, H):
            pareto_archive.add((S, A, U, C, G, H, R, K))

```

```

    if epoch % transfer_period == 0:
        run_serial_transfer_tests(pareto_archive)

    if epoch % ablation_period == 0:
        run_ablation_tests(pareto_archive)

return final_metrics, pareto_archive

```

L.8 ACS detection pseudocode

```

procedure DETECT_ACTIVE_USEFUL_ACS(actual_graph, food_set):

    candidates = mine_dense_catalytic_subgraphs(actual_graph)

    useful_sets = []

    for H in candidates:

        d_food = food_generation_defect(H, food_set)
        d_cat  = catalysis_defect(H)
        d_act  = activity_defect(H)
        d_mass = abundance_or_attention_defect(H)

        D = w_food*d_food + w_cat*d_cat + w_act*d_act + w_mass*d_mass

        if D == 0:

            U_full = measure_utility(current_soup)

            U_without_H = measure_utility_after_ablation(H)

            coupling = (U_full - U_without_H) / (U_full + epsilon)

            if coupling >= coupling_threshold:
                useful_sets.append(H)

    return useful_sets

```

L.9 Minimum practical suite and recommended first benchmarks

The shortest useful suite is:

1. Pure ACS ignition / habit-trap calibration.
2. Productive grammar ecology.
3. Autocatalytic abstraction discovery.
4. Lemma/proof ecology.
5. Program repair ecology.
6. Serial-transfer and ablation protocol applied to items 2–5.

This gives the ramp:

self-maintenance → useful generative production → abstraction
→ symbolic reasoning → program/tool generation
→ heritable useful repertoires.

The first concrete benchmarks to start with should be productive grammar ecology and lemma/proof ecology. They are small enough to instrument thoroughly, symbolic enough to fit METTA/MORK naturally, generative enough to avoid ordinary ML framing, and rich enough for real autocatalytic networks to matter.

For productive grammar ecology, a workable first concrete reward setup:

```
Input food:
  atoms: A, B, C
  constructors: pair, list, apply, lambda-ish wrapper
  primitive validators
  primitive mutation/rewrite operators

Task stream:
  each episode samples a small grammar or type constraint

Reward:
  +1 for valid artifact
  +bonus for novelty
  +bonus for satisfying semantic predicate
  +bonus if artifact later helps produce another valid artifact
  -penalty for duplicates, trivial forms, huge forms

Autocatalysis:
  artifact motifs can catalyze rewrite rules;
  validators can catalyze repair rules;
  repair rules can produce more valid motifs;
  motifs can become meta-guards for generators.
```

L.10 Anti-cheat and anti-rigidity rules

To prevent the soup from becoming a rigid circuit:

1. Use task distributions, not single tasks.
2. Reward families of artifacts, not exact outputs.
3. Require heldout generalization.
4. Require active ACS detection from event logs.
5. Require ACS–utility coupling by ablation.
6. Use serial transfer and dilution tests.
7. Track rare-tail productivity.

8. Add *value-relative anti-precedence* against over-entrenched implementation families, with functional protection for shared machinery and structured release into neighboring implementations (§19.5). The blunt broad-anti-resonance form is retained only as an ablation baseline.
9. Penalize single-point fragility, not concentration alone.
10. Require *functional continuity under implementation change* via the adaptive-continuity benchmark (Bench 7).

The last two points are important. A successful ecology may have a concentrated head of common motifs plus a long tail of rarer motifs. That is not a failure. It becomes a failure only when the head is brittle, the tail no longer produces useful novelty, or the head's dominance is no longer matched by its contribution to viable future transformability.

M Implementation and scaling plan

M.1 Repository layout

```
actpc_mork/  
  README.md  
  schemas/  
    atoms.md  
    rule_schema.md  
    trace_schema.md  
  mm2/  
    phases.mm2  
    match_rules.mm2  
    cleanup.mm2  
    examples_bug.mm2  
  host/  
    Cargo.toml  
    src/  
      lib.rs  
      ids.rs  
      kernels.rs  
      scoring.rs  
      sampling.rs  
      error.rs  
      credit.rs  
      update.rs  
      ot.rs  
      env_bug.rs  
      gcodd.rs  
      cenf.rs  
      moses.rs  
      airis.rs  
      pln_bridge.rs  
      tests.rs  
  experiments/  
    bug_simple.toml  
    bug_features.toml  
    bug_delayed_poison.toml  
    grammar_ecology.toml
```

```

    lemma_ecology.toml
petta_wrapper/
    worker_ir.petta
    scheduler.petta
    chamber_adapter.petta
    reducers.petta
    ecan_worker_budget.petta
    backend_rendering.petta
metta_il_target/
    workers.metta-il
    par_epoch.metta-il
    reducers.metta-il
    worker_templates.metta-il
runtime/
    worker_ir.rs
    chamber_runner.rs
    event_log.rs
    snapshot.rs
    reducers.rs
    backend_adapter.rs
logs/
notebooks/

```

M.2 Minimal initial implementation

The first working implementation can omit gCoDD, CENF, PLN, AIRIS, and OT. It should implement:

- atom schemas;
- deterministic MM2 candidate matching;
- host-side softmax sampler;
- host-side environment step for grid bug;
- KL or cross-entropy over outcome classes;
- trace and online logit update;
- cleanup and debugging snapshots.

M.3 Implementation order for an LLM coder

1. Define atom serialization and query helpers.
2. Define ID allocators for ctx, tick, rule, action, trace.
3. Implement simple virtual bug environment.
4. Add hand-coded rules and compiled trigger/template facts.
5. Implement MM2 or pseudo-MM2 phase files for match/fire/cleanup.
6. Implement SCORE_CANDIDATES and SOFTMAX_SAMPLE.
7. Implement ENV_STEP, KL_XENT, ASSIGN_CREDIT, UPDATE_LOGITS.
8. Add logging snapshots.
9. Add worker, chamber, snapshot, event-log, par-group, reducer, and backend

- schemas. Treat these as the portable Worker IR.
10. Implement a PeTTa wrapper runtime over the Worker IR. It should support context-chamber workers first, then MOSES demes, ACS scans, ECAN epochs, and bridge jobs.
 11. Wrap the deterministic single-context MM2 loop as a PeTTa context-chamber worker. Prove that one PeTTa worker produces the same trace as the original single-chamber prototype.
 12. Add PeTTa par-groups using SWI-Prolog thread pools, worker processes, or equivalent coarse-grained dispatch. Workers emit event logs; reducers merge logs. No PeTTa worker may mutate a shared hot pool concurrently.
 13. Add backend rendering: the same Worker IR can be rendered either as a PeTTa predicate call or as a future MeTTa-IL worker form.
 14. Add delayed traces.
 15. Add meta-guard mining.
 16. Add ECAN-lite: STI/LTI facts, wages, base/cost rent, sparse Hebbian diffusion, pool migration, and worker-level STI/LTI for context-chamber and deme workers. MM2 retrieval queries pool-hot facts. Worker scheduling reads worker-level ECAN scores.
 17. Add source-rule compiler and canonical hash.
 13. Add simple structural mutation.
 14. Add MOSES/factor-graph sampler. MOSES pays ECAN stimulus.
 15. Add AIRIS/PLN proposal bridges. AIRIS/PLN pay ECAN stimulus.
 16. Add OT_STEP as optional update strategy for rule logits.
 17. Add ACS detection over event logs.
 18. Wire ACS dividends and rare-tail reserves into ECAN stimulus.
 19. Add overuse traces, family-level traces \$U_g\$, and the value-relative excess-canalization rent (recommended), with the older broad-overuse rent as a config-flag baseline. Add functional-protection coefficients \$c_r\$ and the structured-release kernel.
 20. Add information-geometric ECAN (mirror descent first, then optional Wasserstein) on bounded warm/hot subgraph.
 21. Wire ACS detection to Pareto archive and benchmark harness.
 22. Run Benchmarks 0 (ignition) and 1 (grammar) under conditions A-G.

M.4 Scaling sequence

Do not begin with a giant soup. A sensible sequence is:

Stage 0: mechanical sanity

rules:	1k - 5k
hot rules/tick:	32 - 128
candidates/tick:	128
firings/tick:	1 - 4
demes:	0 - 4
ECAN:	none yet (static top-k retrieval)
worker backend:	none or PeTTa with one context-chamber worker; deterministic trace equivalence test
purpose:	validate phases, traces, cleanup, action output

Stage 1: online ActPC only

rules:	10k - 30k
warm rules:	1k - 3k
hot rules/tick:	128 - 512

candidates/tick: 256 - 1024
 firings/tick: 4 - 8
 demes: 0 - 8
 ECAN: none yet (static top-k retrieval)
 purpose: validate stochastic selection and credit assignment
 worker backend: PeTTa par-groups with multiple independent
 context-chamber workers; append-only logs/reducers

Stage 2: ECAN-lite + structural evolution

rules: 50k - 150k
 warm rules: 5k - 15k
 hot rules/tick: 512 - 1024
 candidates/tick: 1024 - 2048
 firings/tick: 8 - 16
 demes: 16 - 32
 pop/deme: 128 - 256
 ECAN: STI/LTI, wages, base/cost rent, sparse Hebbian
 diffusion, pool migration
 purpose: validate MOSES/gCoDD/CENF proposals; show ECAN
 improves hot-pool precision over static top-k
 worker backend: PeTTa runs context chambers, MOSES demes, and ACS
 scans as worker kinds; ECAN begins worker scheduling

Stage 3: ACS-aware ECAN + serious single-server run

rules: 150k - 500k
 warm rules: 15k - 50k
 hot rules/tick: 1024 - 2048
 candidates/tick: 2048 - 4096
 firings/tick: 16 - 32
 demes: 32 - 64
 pop/deme: 256 - 512
 ECAN: ACS dividends, rare-tail reserves,
 family-level value-relative
 excess-canalization rent + functional
 protection + neighbor release,
 ACS macro-atoms
 purpose: real chemistry experiment with attention
 economy; first run of adaptive-continuity
 benchmark (Bench 7) to compare broad vs
 protected anti-precedence
 worker backend: PeTTa scheduler manages runtime budgets across
 chambers, demes, scans, replay batches, and bridges

Stage 4: info-geometric ECAN + scale test

rules: 500k - 2M+
 warm rules: 50k - 150k
 hot rules/tick: 2048 - 8192
 candidates/tick: 4096 - 16384
 firings/tick: 32 - 64
 demes: 64 - 128
 pop/deme: 512 - 1024
 ECAN: mirror-descent + optional Wasserstein on bounded
 warm/hot subgraph; coupling to ActPC via KL
 purpose: stress and scaling after profiling succeeds

worker backend:	migrate selected worker families from PeTTa to MeTTa-IL if available; compare backend outputs
-----------------	---

M.5 Mapping integration milestones to scaling stages

The integration milestones below specify what must work before moving to the next scaling stage.

1. **M0: Atom schema smoke test.** Insert and query contexts, rules, observations, candidates, and traces. (Stage 0.)
2. **M1: Deterministic rule firing.** Hand-coded food rule fires and emits action/trace. (Stage 0.)
3. **M2: Stochastic selection.** Two conflicting rules are sampled according to logits. (Stage 0–1.)
4. **M3: Online learning.** Food-grab rule logit increases; poison-grab rule logit decreases. (Stage 1.)
5. **M4: Delayed reward.** Trace remains open and receives delayed poison credit after many ticks. (Stage 1.)
6. **M5: ECAN-lite pool manager.** STI/LTI facts are written and updated; warm-to-hot promotion is driven by STI; MM2 retrieval queries respect `pool-hot` facts; hot-pool precision improves over the static top- k baseline of Stages 0–1. (Stage 2.)
7. **M6: Meta-guard creation.** Correlated feature conjunction becomes a meta-guard and is used by a compiled rule. (Stage 2.)
8. **M7: AIRIS proposal with stimulus.** A surprising transition creates a trial causal rule and pays ECAN stimulus. (Stage 2.)
9. **M8: MOSES proposal with stimulus.** Deme samples a new rule that survives trial promotion; high-fitness fragments collect ECAN stimulus. (Stage 2.)
10. **M9: OT update.** Wasserstein update on rule logits improves stability versus independent logit update on a controlled rule graph. (Stage 2–3.)
11. **M10: ACS detection and dividends.** Active autocatalytic subnetworks are detected from event logs, pass food-generation, catalysis, and activity defect checks, and pay ECAN dividends to members and rare-tail precursors. (Stage 3.)
12. **M11: Anti-resonance.** Overuse traces are maintained; overuse rent breaks a synthetic high-utility habit-trap scenario without destroying productive concentration. (Stage 3.)
13. **M12: Information-geometric ECAN.** Mirror-descent ECAN step on a bounded warm/hot subgraph improves promotion precision and rare-tail enrichment over ECAN-lite alone. (Stage 3–4.)
14. **M13: Coupling and transfer.** ACS ablation produces measurable utility delta; serial transfer preserves function; removing the ECAN layer (or replacing it with static top- k) degrades ACS formation and rare-tail enrichment. (Stage 3–4.)

M.6 Promotion from trial to active

```
procedure PROMOTE_OR_RETIRE_TRIAL_RULES(space):
  for R in query(space, (rule-status ?R trial)):
    uses <- get_use_count(R)
    if uses < min_trial_uses:
      continue

    avg_credit <- average_recent_credit(R)
    avg_error_delta <- average_error_delta(R)
    duplicate <- has_active_duplicate(R)

    if duplicate:
      replace_status(R, trial, retired)
    else if avg_credit > promote_credit_threshold and
      avg_error_delta < promote_error_threshold:
      replace_status(R, trial, active)
      add(space, (rule-promoted R current_tick))
    else if avg_credit < retire_credit_threshold:
      replace_status(R, trial, retired)
      add(space, (rule-retired R current_tick))
```

N Worked example: virtual bug food and poison

N.1 Initial facts

```
(ctx C1)
(ctx-status C1 active)
(goal C1 get-food)
(goal-weight C1 get-food 1.0)

(rule R1)
(rule-kind R1 act)
(rule-status R1 active)
(rule-sig R1 see-item)
(trigger see-item R1)
(template R1 act-grab)
(rule-logit R1 0.0)
(rule-cost R1 1.0)

(rule R2)
(rule-kind R2 act)
(rule-status R2 active)
(rule-sig R2 see-item)
(trigger see-item R2)
(template R2 act-turn-left)
(rule-logit R2 0.0)
(rule-cost R2 1.0)
```

N.2 Specialized learned rules

```
(rule R10)
(rule-kind R10 act)
(rule-status R10 active)
(rule-sig R10 see-food)
(trigger see-food R10)
(template R10 act-grab)
(rule-logit R10 1.4)
(rule-cost R10 2.0)
```

```
(rule R11)
(rule-kind R11 act)
(rule-status R11 active)
(rule-sig R11 see-poison)
(trigger see-poison R11)
(template R11 act-avoid)
(rule-logit R11 1.2)
(rule-cost R11 2.0)
```

N.3 Feature-combination abstraction

```
(meta-guard MG1)
(meta-guard-member MG1 color-red)
(meta-guard-member MG1 shape-round)
(meta-guard-member MG1 texture-smooth)
(meta-guard-confidence MG1 0.82)
```

```
(rule R20)
(rule-kind R20 classify)
(rule-status R20 active)
(rule-sig R20 color-red)
(trigger color-red R20)
(rule-guard R20 MG1)
(template R20 classify-foodlike)
(rule-logit R20 0.9)
```

N.4 Delayed conditional poison

```
(rule R30)
(rule-kind R30 causal)
(rule-status R30 trial)
(rule-sig R30 ate-item)
(trigger ate-item R30)
(rule-guard R30 red-round-sweet)
(rule-guard R30 mostly-still-after-eating)
(rule-guard R30 elapsed-gt-50)
(template R30 predict-sickness)
(rule-logit R30 0.2)
(rule-owner R30 airis)
```

N.5 End-to-end example tick

1. Environment adds:
(obs C1 see-food 09 2 3)
(feat C1 09 color-red)
(feat C1 09 shape-round)
(feat C1 09 texture-smooth)
2. MM2 activates:
(active-sig C1 see-food)
(guard-has C1 09 MG1)
3. MM2 matches:
(cand C1 T1 R10 09 2 3)
4. SCORE_CANDIDATES emits:
(cand-score C1 T1 R10 1.35)
5. SOFTMAX_SAMPLE emits:
(chosen C1 T1 R10 09 2 3)
6. FIRE_TEMPLATE emits:
(action C1 T1 A1 grab 09)
(trace E1 C1 T1 R10 A1)
(trace-bind E1 obj 09)
7. ENV_STEP emits:
(outcome C1 T1 got-food)
(reward C1 T1 env 1.0)
8. KL_XENT emits:
(error C1 T1 kl 0.10)
(reward C1 T1 instrumental -0.10)
(reward C1 T1 epistemic 0.03)
9. ASSIGN_CREDIT emits:
(credit E1 R10 immediate 0.79)
(credit-total R10 C1 T1 0.79)
10. UPDATE_LOGITS replaces:
(rule-logit R10 1.4)
with
(rule-logit R10 1.45)

N.6 Acceptance criteria for the virtual bug

Task A: simple food/poison

Success: average reward improves above random baseline by 5x.

Success: food-grab rule logit > poison-grab rule logit after training.

Task B: feature-combination classification

Success: no single feature is sufficient, but meta-guard rule emerges.

Success: learned policy generalizes to held-out objects with same pattern.

Task C: delayed conditional poison

Success: causal rule includes time delay and activity condition.

Success: bug learns to keep moving after eating conditional item.

Task D: structural evolution

Success: MOSES-proposed rules survive trial more often than random rules.

Success: duplicate rules are canonicalized and not repeatedly evaluated.

O Compilation pipeline

O.1 Compile source rule to gCoDD

```
procedure COMPILE_SOURCE_TO_GCODD(source_rule):
  ast <- parse_metta_like_source(source_rule)
  dag <- empty_dag()

  function lower(node):
    if node is grounded predicate call:
      return dag.leaf(label=node.predicate, args=node.static_args)
    if node is variable:
      return dag.var(node.name)
    if node is logical connective:
      children <- [lower(c) for c in node.children]
      return dag.node(node.operator, children)
    if node is template/action:
      children <- [lower(c) for c in node.children]
      return dag.node("template", children)

  root <- lower(ast)
  root <- hash_cons(root)
  root <- ENF_NORMALIZE(root)
  return root
```

O.2 Compile gCoDD to hot-path atoms

```
procedure COMPILE_GCODD_TO_HOT_PATH(space, R, root):
  guards <- extract_guard_nodes(root)
  trigger <- choose_primary_trigger(guards)
  template <- extract_template(root)
  cost <- compute_cost(root)

  add_or_replace(space, (rule-sig R trigger))
  add_or_replace(space, (trigger trigger R))
  add_or_replace(space, (rule-cost R cost))

  for guard in guards:
    if guard != trigger:
      add(space, (rule-guard R guard))

  add_or_replace(space, (template R template.kind))
```

```
for arg in template.args:
    add(space, (template-arg R arg.name arg.binding_name))
```

O.3 Choose primary trigger

The trigger should maximize selectivity and stability.

```
procedure CHOOSE_PRIMARY_TRIGGER(guards):
    best <- None
    best_score <- -infinity
    for g in guards:
        selectivity <- estimate_selectivity(g)      # lower frequency is better
        stability <- estimate_stability(g)          # avoid noisy feature triggers
        cost <- estimate_match_cost(g)
        score <- w_sel * selectivity + w_stab * stability - w_cost * cost
        if score > best_score:
            best <- g
            best_score <- score
    return best
```

P Logging, observability, and testing

P.1 Event log facts

```
(log-worker LOG99 W17)
(log-backend LOG99 petta)
(log-chamber LOG99 CH9)
(log-par-group LOG99 PG3)
(log-event LOG99 T193 seq 0001 phase match started)
(log-event LOG99 T193 seq 0002 candidate R17 09)
(log-event LOG99 T193 seq 0003 chosen R17)
(log-event LOG99 T193 seq 0004 action A88 grab 09)
(log-event LOG99 T193 seq 0005 reward env 1.0)
(log-event LOG99 T193 seq 0006 credit R17 0.82)
(log-event LOG99 T193 seq 0007 k1 0.22)
(log-event LOG99 T193 seq 0008 update R17 0.41 0.46)
(log-closed LOG99 T193)
```

The older compact notation remains valid as a shorthand for the same local sequence:

```
(log T193 C42 phase match started)
(log T193 C42 candidates 12)
(log T193 C42 chosen R17)
(log T193 C42 reward 1.0)
(log T193 C42 k1 0.22)
(log T193 C42 update R17 0.41 0.46)
```

P.2 Debug snapshots

The host should support snapshots per tick:

```

Snapshot(ctx, tick):
  observations
  active signatures
  candidates with scores
  selected rule
  action
  predicted distribution
  observed outcome
  reward components
  trace ids
  updated logits
  structural jobs enqueued

```

P.3 Invariants to check

1. Every **chosen** atom has a corresponding **cand** atom.
2. Every **action** emitted by a rule has a corresponding trace.
3. Every open trace either closes, expires, or is carried to the next tick.
4. Every active rule has exactly one current **rule-logit** fact.
5. Every active rule has at least one trigger.
6. Every trial rule has a source hash.
7. Every meta-guard has at least two members.
8. Candidate count per context/tick/signature is under budget.
9. Every atom has at most one (**sti A v**) and one (**lti A v**) fact at any time.
10. Total $\sum_A \text{STI}(A)$ is within tolerance of the configured **sti_budget** after each **ECAN_NORMA-LIZE_BUDGETS**.
11. Hebbian and InverseHebbian out-degrees per atom are within configured caps.
12. No atom is in the hot pool of more contexts than the configured hot multi-context cap.
13. Every (**pool-hot \$C \$R**) fact corresponds to an atom in the warm pool with $\text{STI} \geq$ the warm-to-hot promotion threshold (or in a quota bucket that allows it).
14. No atom appears in two different pools simultaneously (cold/warm/forgotten are mutually exclusive global pools).
15. Every context-chamber worker has exactly one chamber and one output event log.
16. Every chamber marked **single-threaded** has strictly increasing local event sequence numbers.
17. No two running workers write to the same context-local hot pool unless one is explicitly a reducer and the chamber using that hot pool is not running.

18. Every **par-group** has a corresponding reduction step or an explicit marker saying that its outputs are independent and need no reduction.
19. Every **pool-hot C R** fact used by a chamber is either in its input snapshot or was written before the chamber began execution.
20. A replay of a chamber from snapshot, seed, and worker configuration reproduces the same chosen rules and trace records.
21. Backend equivalence test: for a small deterministic benchmark, the PETTA backend and the future METTA-IL backend must produce equivalent event logs modulo backend tags and harmless ID allocation differences.

P.4 Unit tests for grounded kernels

Kernel	Tests
SOFTMAX_SAMPLE	deterministic seed reproducibility, probability normalization, temperature behavior.
KL_XENT	zero error when distributions match, floor handling, missing patterns.
UPDATE_LOGITS	positive credit increases logits, cost penalty works, clipping works.
OT_STEP	preserves finite logits, moves mass toward credited nodes, respects disconnected components.
CANON_HASH	equivalent rules hash identically after normalization.
CENF_CLUSTER	correlated guards form meta-guards; weak correlations do not.
FG_SAMPLE	sampled assignments match known simple factor graph marginals.
ECAN_PAY_WAGES	positive stimulus increases STI; durable stimulus also increases LTI; nonpositive stimulus does not raise STI.
ECAN_COLLECT_RENT	base rent reduces STI of hot atoms more than warm; cost rent scales with cost; overuse rent activates above threshold and is zero below.
ECAN_DIFFUSE_STI	total STI is conserved within tolerance; diffusion respects the bounded frontier; out-degree caps are honored.
ECAN_PROMOTE_DEMOTE	hot-pool quotas are respected; high-STI atoms become hot; idle hot atoms demote; low-LTI cold atoms enter the forget candidate set.
ECAN_INFO_GEOM_STEP	mirror-descent update has the expected closed form on small synthetic problems; preserves nonnegativity and normalization; reduces toward utility direction.

Q Guidance for coding LLMs

When a coding LLM implements this design, it should follow these rules:

1. Implement the smallest deterministic path first: observation to candidate to chosen action to trace.
2. Do not implement Wasserstein OT until the simple update passes tests.
3. Do not match on **rule-src** in the inner loop.
4. Do not invent huge nested atoms for hot-path state.

5. Every host kernel should have a pure unit-testable core independent of MORK I/O.
6. Every host kernel should have a thin adapter that reads atoms and writes atoms.
7. Every stochastic function must accept a seed for reproducibility.
8. Every structural rule proposal must pass canonicalization and duplicate checks before insertion.
9. Every transient atom must be scoped by context and tick.
10. Every new rule starts as **trial**, not **active**, unless it is hand-coded bootstrap knowledge.
11. Always prefer Pareto-archive bookkeeping over single-scalar fitness.
12. Wire ACS detection in early; do not defer it to “after the learner works.” ACS structure can be diagnostic of the learner’s health, not just an evaluation afterthought.
13. ECAN-related arithmetic — wages, rent, normalization, diffusion, mirror descent, Wasserstein — belongs in grounded kernels. MM2 should consume (**pool-hot \$C \$R**) and emit (**stimulus-event ...**) markers, nothing more. (See §16.12.)
14. Implement ECAN-lite (STI/LTI, wages, base/cost rent, sparse Hebbian diffusion, pool migration) before any information-geometric step. The mirror-descent variant is a small addition once ECAN-lite is stable; the Wasserstein variant comes later still.
15. Cap Hebbian and InverseHebbian out-degree per atom at insertion time, not in a periodic cleanup. Dense association graphs are the easiest way to silently make ECAN expensive.
16. Keep $p_t(R)$ and $a_t(A)$ separate. Stimulus is the principal coupling channel; the KL regularizer is optional and gentle. Identifying the two distributions defeats the entire point of having both.
17. Do not wait for METTA-IL to implement the first worker runtime. Implement the wrapper as PETTA/SWI-Prolog over the portable Worker IR.
18. Do not bake architecture into PETTA-private predicates. Every durable worker concept must be represented as a Worker IR fact.
19. PETTA may dispatch workers and call kernels. PETTA must not bypass ECAN hot-pool gating or mutate one shared hot pool concurrently.
20. Write backend-rendering tests: given one Worker IR record, produce a PETTA call now and a future METTA-IL form later.
21. Keep event logs backend-neutral. PETTA and METTA-IL should emit the same log schema.
22. Use PETTA or **par** parallelism only for coarse-grained workers. Fine-grained candidate scoring belongs inside chambers or grounded kernels.

R Pseudo-MM2 skeleton

```
; -----  
; PeTTa / MeTTa-IL worker orchestration  
; -----  
; This is schematic. The important point is that concurrency composes  
; coarse-grained workers. It does not run many MM2 rewrites against one  
; shared hot pool.  
  
(par  
  (run-worker W_ctx_001)  
  (run-worker W_ctx_002)  
  (run-worker W_deme_001)  
  (run-worker W_acs_001)  
  (run-worker W_ecan_001))  
  
(reduce-par-group PG17)  
  
; A context worker runs one single-threaded MM2 chamber.  
; The chamber consumes pool-hot facts from its snapshot.  
; It emits ordered event-log facts and stimulus-event facts.  
  
; -----  
; Phase transitions  
; -----  
  
(exec (31 finish-activate)  
  (, (todo $C $T activate-signatures))  
  (0 (- (todo $C $T activate-signatures))  
    (+ (done $C $T activate-signatures))  
    (+ (todo $C $T match))))  
  
(exec (49 finish-match)  
  (, (todo $C $T match))  
  (0 (- (todo $C $T match))  
    (+ (done $C $T match))  
    (+ (todo $C $T score))))  
  
(exec (59 finish-score)  
  (, (done $C $T score))  
  (0 (+ (todo $C $T sample))))  
  
; -----  
; Signature activation  
; -----  
  
(exec (30 sig-see-food)  
  (, (todo $C $T activate-signatures)  
    (obs $C see-food $Obj $X $Y))  
  (, (active-sig $C see-food)))  
  
(exec (30 sig-see-poison)  
  (, (todo $C $T activate-signatures)
```

```

    (obs $C see-poison $Obj $X $Y))
  (, (active-sig $C see-poison)))

; -----
; Candidate matching (ECAN-gated)
; -----
; The (pool-hot $C $R) atom is what makes the loop hot: it is a small,
; context-prefixed selectivity gate written by ECAN_PROMOTE_DEMOTE.
; MM2 only consumes it; it does not compute it.

(exec (40 match-food)
  (, (todo $C $T match)
    (active-sig $C see-food)
    (obs $C see-food $Obj $X $Y)
    (pool-hot $C $R)
    (trigger see-food $R)
    (rule-status $R active))
  (, (cand $C $T $R $Obj $X $Y)))

(exec (40 match-poison)
  (, (todo $C $T match)
    (active-sig $C see-poison)
    (obs $C see-poison $Obj $X $Y)
    (pool-hot $C $R)
    (trigger see-poison $R)
    (rule-status $R active))
  (, (cand $C $T $R $Obj $X $Y)))

; -----
; Stimulus emission (atom-level only)
; -----
; MM2 writes small (stimulus-event ...) markers that the next call to
; ECAN_GATHER_STIMULUS aggregates. MM2 does NOT compute STI/LTI updates.

(exec (75 emit-fire-stimulus)
  (, (chosen $C $T $R $Obj $X $Y)
    (rule-cost $R $Cost))
  (, (stimulus-event $T $R fire-success 1.0)))

(exec (115 emit-credit-stimulus)
  (, (credit-total $R $C $T $Cr))
  (, (stimulus-event $T $R actpc-credit $Cr)))

; -----
; Host-kernel requests
; -----

(exec (50 score-request)
  (, (todo $C $T score)
    (cand $C $T $R $Obj $X $Y))
  (, (score-request $C $T)))

(exec (60 sample-request)
  (, (todo $C $T sample)))

```

```

(, (sample-request $C $T)))

(exec (70 fire-request)
  (, (todo $C $T fire)
    (chosen $C $T $R $Obj $X $Y))
  (, (fire-request $C $T $R $Obj $X $Y)))

(exec (80 env-step-request)
  (, (todo $C $T env-step)
    (action $C $T $A $Kind $Arg))
  (, (env-step-request $C $T $A $Kind $Arg)))

(exec (125 ecan-update-request)
  (, (todo $C $T ecan-update))
  (, (ecan-update-request $C $T)))

```

S Rust-like kernel skeleton

```

// -----
// Worker runtime skeleton
// -----

pub enum WorkerKind {
    ContextChamber,
    MosesDeme,
    AcsScan,
    EcanEpoch,
    BridgeFit,
    ReplayBatch,
    AirisJob,
    PlnJob,
    Reducer,
}

pub struct WorkerSpec {
    pub id: String,
    pub kind: WorkerKind,
    pub context: Option<String>,
    pub snapshot: Option<String>,
    pub hot_pool: Option<String>,
    pub budget_cpu_ms: u64,
    pub budget_memory_mb: u64,
    pub seed: u64,
}

pub struct EventRecord {
    pub log_id: String,
    pub worker_id: String,
    pub context: Option<String>,
    pub tick: String,
    pub seq: u64,
}

```

```

    pub kind: String,
    pub payload: Vec<(String, String)>,
}

pub trait WorkerRuntime<S: AtomSpace> {
    fn run_worker(&mut self, space: &mut S, worker: &WorkerSpec) -> Vec<EventRecord>;
    fn reduce_outputs(&mut self, space: &mut S, logs: Vec<Vec<EventRecord>>);
}

pub fn run_context_chamber_worker<S: AtomSpace>(
    space: &mut S,
    worker: &WorkerSpec,
    cfg: &KernelConfig,
) -> Vec<EventRecord> {
    restore_context_snapshot(space, worker.snapshot.as_ref().unwrap());

    // Run the single-threaded MM2 chamber over the context-local hot pool.
    let events = run_single_threaded_mm2_chamber(
        space,
        worker.context.as_ref().unwrap(),
        worker.seed,
        cfg,
    );

    write_event_log_atoms(space, &worker.id, &events);
    events
}

pub fn reduce_worker_event_logs<S: AtomSpace>(
    space: &mut S,
    logs: Vec<Vec<EventRecord>>,
) {
    let stimulus = reduce_stimulus_events(&logs);
    write_stimulus_totals(space, stimulus);

    let trace_summaries = reduce_trace_summaries(&logs);
    write_trace_summaries(space, trace_summaries);

    let acs_inputs = build_acs_scan_inputs(&logs);
    write_acs_scan_inputs(space, acs_inputs);
}

pub trait AtomSpace {
    fn add(&mut self, atom: Atom);
    fn remove(&mut self, atom: &Atom);
    fn query(&self, pattern: &AtomPattern) -> Vec<Bindings>;
    fn replace(&mut self, old_atom: &Atom, new_atom: Atom) {
        self.remove(old_atom);
        self.add(new_atom);
    }
}

pub struct KernelConfig {
    pub eta_logit: f64,

```

```

    pub lambda_cost: f64,
    pub prior_pull: f64,
    pub temperature: f64,
    pub gamma_trace: f64,
    pub max_trace_age: u64,
    pub advantage_clip: f64,
}

pub fn run_update_request<S: AtomSpace>(space: &mut S,
                                       ctx: &str,
                                       tick: &str,
                                       cfg: &KernelConfig) {
    let credits = read_credit_totals(space, ctx, tick);
    for cr in credits {
        let stats = read_rule_stats(space, &cr.rule);
        let clipped = cr.value.clamp(-cfg.advantage_clip, cfg.advantage_clip);
        let new_logit = (1.0 - cfg.prior_pull) * stats.logit
            + cfg.prior_pull * stats.prior_logit
            + cfg.eta_logit * clipped
            - cfg.lambda_cost * stats.cost;
        write_rule_logit(space, &cr.rule, new_logit);
    }
}

// -----
// ECAN kernel skeleton
// -----

pub struct EcanConfig {
    pub wage_sti: f64,
    pub wage_lti: f64,
    pub base_hot_rent: f64,
    pub base_warm_rent: f64,
    pub cost_rent_scale: f64,
    pub overuse_rent_scale: f64,
    pub overuse_threshold: f64,
    pub overuse_decay: f64,
    pub max_spread_fraction: f64,
    pub max_hebb_edges: usize,
    pub sti_budget: f64,
    pub forget_lti_floor: f64,
}

pub struct EcanContext<'a, S: AtomSpace> {
    pub space: &'a mut S,
    pub ctx: &'a str,
    pub tick: &'a str,
    pub cfg: &'a EcanConfig,
}

// Aggregate stimulus_event atoms into per-atom totals.
pub fn ecan_gather_stimulus<S: AtomSpace>(c: &EcanContext<S>)
    -> HashMap<AtomId, f64> { /* read (stimulus-event ...) atoms, sum by AtomId */
    unimplemented!() }

```

```

pub fn ecan_pay_wages<S: AtomSpace>(c: &mut EcanContext<S>,
                                   stim: &HashMap<AtomId, f64>) {
    for (atom, s) in stim {
        let mut sti = read_sti(c.space, atom);
        let mut lti = read_lti(c.space, atom);
        sti += c.cfg.wage_sti * s.max(0.0);
        lti += c.cfg.wage_lti * durable_component(c.space, atom, *s);
        write_sti(c.space, atom, sti);
        write_lti(c.space, atom, lti);
    }
}

pub fn ecan_collect_rent<S: AtomSpace>(c: &mut EcanContext<S>) {
    for atom in iter_frontier(c.space, c.ctx) {
        let mut sti = read_sti(c.space, &atom);
        let cost = read_cost(c.space, &atom);
        let h = read_overuse_trace(c.space, &atom);
        let in_hot = is_hot(c.space, &atom, c.ctx);
        let base = if in_hot { c.cfg.base_hot_rent } else { c.cfg.base_warm_rent };
        let overuse = (h - c.cfg.overuse_threshold).max(0.0);
        sti -= base
            + c.cfg.cost_rent_scale * cost * sigmoid(sti)
            + c.cfg.overuse_rent_scale * cost * overuse * overuse;
        write_sti(c.space, &atom, sti);
    }
}

pub fn ecan_diffuse_sti<S: AtomSpace>(c: &mut EcanContext<S>) {
    let frontier = bounded_frontier(c.space, c.ctx);
    for atom in &frontier {
        let sti = read_sti(c.space, atom);
        if !should_diffuse(sti) { continue; }
        let neighbors = top_hebbian_neighbors(c.space, atom, c.cfg.max_hebb_edges);
        let denom: f64 = neighbors.iter()
            .map(|(_, w)| w.max(0.0))
            .sum::<f64>() + 1e-12;
        let budget = c.cfg.max_spread_fraction * sti;
        for (b, w) in &neighbors {
            let delta = budget * w.max(0.0) / denom;
            bump_sti(c.space, b, delta);
            bump_sti(c.space, atom, -delta);
        }
        // (analogous block for top_inverse_neighbors)
    }
}

pub fn ecan_normalize_sti<S: AtomSpace>(c: &mut EcanContext<S>) {
    let total: f64 = iter_all_atoms(c.space).map(|a| read_sti(c.space, &a).max(0.0)).sum();
    let scale = c.cfg.sti_budget / (total + 1e-12);
    for atom in iter_all_atoms(c.space) {
        let sti = read_sti(c.space, &atom).max(0.0) * scale;
        write_sti(c.space, &atom, sti);
    }
}

```

```

    }
}

// One full ECAN tick: gather, pay, rent, diffuse, normalize, migrate.
pub fn run_ecan_update<S: AtomSpace>(c: &mut EcanContext<S>) {
    let stim = ecan_gather_stimulus(c);
    ecan_pay_wages(c, &stim);
    ecan_decay_and_bump_overuse_traces(c);
    ecan_collect_rent(c);
    ecan_diffuse_sti(c);
    ecan_update_hebbian(c);
    ecan_normalize_sti(c);
    ecan_homeostat_lti(c);
    ecan_promote_demote_pools(c);
    ecan_enforce_hot_quotas(c);
}

```

T Mathematical summary

T.1 Predicted and observed distributions

$$p_t(m) = P(m \mid R_t, G_t^{in}), \quad q_t(m) = P(m \mid G_t^{out}). \quad (8)$$

T.2 Prediction error

$$e_t = D_{KL}(q_t \parallel p_t). \quad (9)$$

T.3 Reward

$$r_t = \alpha_{env} r_t^{env} + \alpha_{int} r_t^{int} + \alpha_{ep} r_t^{ep}. \quad (10)$$

A simple instrumental reward is $r_t^{int} = -e_t$. A safer epistemic reward is $r_t^{ep} = \max(0, e_{t-1} - e_t) + \eta H(p_t)$.

T.4 Simple logit update

$$\ell_R \leftarrow (1 - \rho) \ell_R + \rho \ell_R^{prior} + \eta A_R - \lambda \cdot \text{cost}(R). \quad (11)$$

T.5 Wasserstein-style update

Given active rules $R_1, \dots, R_n$, logits ℓ , probabilities $p = \text{softmax}(\ell)$, and a rule graph with edge costs ω_{ij} , construct sparse $L(p)$ and solve $L(p)\phi = g$, where g is a projected gradient or negative credit vector. Then update $\ell \leftarrow \ell - h\phi$.

T.6 Resonance / anti-resonance modifier (legacy baseline)

The legacy broad-anti-resonance form, retained as an ablation baseline (config `anti_precedence.mode = broad`):

$$\pi_r^{eff}(t) \propto \pi_r^{PC}(t) \exp(\Delta\psi_r(t) + \eta h_r(t) - \xi \Phi(h_r(t); \theta)), \quad \Phi(h; \theta) = [h - \theta]_+^2.$$

T.7 Value-relative anti-precedence (recommended)

Per pattern family $g \in \mathcal{G}$:

$$\begin{aligned}
U_g(t) &= \int_0^t e^{-\lambda_U(t-s)} dN_g(s), & S_g(t) &= \frac{U_g(t)}{\sum_h U_h(t) + \epsilon}. \\
V_g(t) &\approx \frac{w_U \Delta U_g + w_C C_g + w_G G_g + w_H H_g + w_R R_g^{\text{res}}}{J_g + \text{Cost}_g + \epsilon} - w_K K_g - w_B B_g. \\
S_g^*(t) &= \frac{\exp(\beta V_g(t))}{\sum_h \exp(\beta V_h(t))}, & O_g(t) &= [S_g(t) - S_g^*(t) - \theta_O]_+. \\
R_h(t) &= \sum_g K(g, h) O_g(t), & K(g, h) &\propto \exp(-d(g, h)/\tau). \\
c_r(t) &\approx \text{clip}\left(\frac{\Delta U_r(t)}{(J_r(t) + \epsilon) c_0}, 0, 1\right).
\end{aligned}$$

The recommended effective rule-selection probability:

$$\pi_r^{\text{eff}} \propto \pi_r^{\text{PC}} \exp\left(\Delta\psi_r + \eta_{\text{id}} h_r + \eta_{\text{func}} c_r H^{\text{func}} - \xi(1 - c_r) O_{\pi(r)}^p + \rho R_{\pi(r)}\right).$$

Excess-canalization rent on atom A in pattern family $g = \pi(A)$:

$$\text{Rent}_{\text{excess-can}}(A) = r_O (1 - c_A) O_g(t)^p,$$

balanced by neighbor-release credits $\rho R_g(t)$ on atoms in nearby families.

T.8 Coupling score

$$C(S) = \frac{U(\text{soup}) - U(\text{soup} \setminus S)}{U(\text{soup}) + \epsilon} \cdot \frac{\text{RewardFlux}(S)}{\text{RewardFlux}(\text{soup}) + \epsilon}.$$

T.9 ECAN attention distribution

Normalized STI gives an attention distribution over atoms:

$$a_t(A) = \frac{\text{STI}(A)}{\sum_B \text{STI}(B)}.$$

T.10 ECAN economic update

For each atom A with aggregated stimulus s_A in the current cycle, let $g = \pi(A)$ be its pattern family. The recommended `family_protected` update:

$$\begin{aligned}
\text{STI}(A) &\leftarrow \text{STI}(A) + w_s s_A - \rho_0 \mathbf{1}[A \in \text{hot}] - \rho_1 \text{cost}(A) \sigma(\text{STI}(A)) \\
&\quad - r_O (1 - c_A) O_g(t)^p + \rho_R R_g(t), \\
\text{LTI}(A) &\leftarrow (1 - \lambda_L) \text{LTI}(A) + w_l \text{Durable}(s_A) - \text{Rent}_{\text{LTI}}(A), \\
h_A(t+1) &= \lambda_h h_A(t) + \text{fired}(A, t) + \zeta \text{coactive}(A, t), \\
U_g(t+1) &= (1 - \lambda_U) U_g(t) + \sum_{r \in g} \text{fired}(r, t),
\end{aligned}$$

followed by renormalization $\text{STI}(A) \leftarrow B_{\text{STI}} \text{STI}(A) / \sum_B \max(0, \text{STI}(B))$. The legacy `broad` mode replaces the $-r_O(1 - c_A)O_g^p + \rho_R R_g$ contribution with $-r_h \text{cost}(A) [h_A - \theta_h]_+^2$.

T.11 Mirror-descent ECAN step

For utility field u_t on the active subgraph $\mathcal{B}_t$,

$$a_{t+1}(A) \propto a_t(A) \exp(u_t(A)/\lambda), \quad A \in \mathcal{B}_t.$$

T.12 Wasserstein ECAN step

$$a_{t+1} = \arg \min_{a \in \Delta(\mathcal{B}_t)} \left[-\langle u_t, a \rangle + \lambda \text{KL}(a \| a_t) + \frac{1}{2h} W_{\omega_{eff}}^2(a, a_t) + \chi \text{Overuse}(a) \right],$$

with the recommended history-sensitive ground cost

$$\omega_{ij}^{eff}(t) = \omega_{ij} \exp\left(-\eta \bar{h}_{ij}(t) + \xi (1 - \bar{c}_{ij}(t)) O_{\pi(i)}(t)^p - \rho R_{\pi(j)}(t) - \lambda_{SB} \Delta \psi_{ij}(t)\right),$$

where $\bar{c}_{ij}$ is an averaged functional-protection coefficient on the edge. The legacy form $\omega_{ij}^{eff}(t) = \omega_{ij} \exp(-\eta \bar{h}_{ij} + \xi \Phi(\bar{h}_{ij}; \theta) - \lambda_{SB} \Delta \psi_{ij})$ is used only as a baseline.

T.13 Joint ACTPC-CHEM/ECAN regularizer

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{ACTPC-CHEM}}(p) + \lambda_1 \text{KL}(p \| a_R) + \lambda_2 \text{KL}(a_R \| p) + \Omega_{\text{resource}}(a),$$

where a_R is the ECAN attention distribution restricted to rule atoms.

T.14 Combined selection score

The recommended candidate-scoring formula used by the inner loop:

$$\begin{aligned} \text{Score}(R; C) = & \ell_R^{\text{ActPC}}/T + \alpha_s \log(\text{STI}(R) + \varepsilon) + \alpha_l \log(\text{LTI}(R) + \varepsilon) \\ & + \gamma \text{Rel}(R, C) + \delta \text{ACSBonus}(R) \\ & + \eta_{\text{id}} h_R + \eta_{\text{func}} c_R H^{\text{func}} \\ & - \xi (1 - c_R) O_{\pi(R)}^p + \rho R_{\pi(R)} \\ & - \kappa \text{cost}(R) + \nu \text{Novelty}(R, C). \end{aligned}$$

The legacy baseline replaces the precedence / anti-precedence / release block with $\mu h_R - \xi \Phi(h_R; \theta)$.

U Recommended initial configuration files

U.1 Starter config

```
[server]
target_cores = 64
target_ram_gb = 512
reserve_ram_fraction = 0.35

[soup]
persistent_rule_target = 100000
persistent_rule_hard_cap = 300000
active_rule_target = 10000
active_rule_hard_cap = 30000
```

```

[context]
working_memory_fact_cap = 8192
scratch_atom_cap = 10000
active_signature_cap = 16
rules_per_signature_cap = 32

[inner_loop]
tick_budget_ms = 50
hot_rules_per_context = 512
candidate_cap = 1024
fire_cap = 8
prediction_rollout_cap = 64

[traces]
trace_ttl_ticks = 4096
recent_trace_cap = 250000
compress_old_traces = true

[learning]
logit_update_every_ticks = 1
credit_assignment_every_ticks = 10
structural_epoch_ticks = 200
mutation_fraction = 0.01
new_rule_insert_cap_per_epoch = 5000
retire_fraction_per_epoch = 0.02

[moses]
demes = 32
population_per_deme = 256
elite_per_deme = 32
eda_variables_cap = 192
factor_arity_cap = 4
nary_factor_cap = 256
cenf_every_generations = 10

[ecan]
sti_budget = 1.0
lti_budget = 1.0
wage_sti = 0.02
wage_lti = 0.002
base_hot_rent = 0.001
base_warm_rent = 0.0001
cost_rent_scale = 0.0005
overuse_rent_scale = 0.002      # legacy broad-overuse rent (mode = broad)
overuse_threshold = 5.0        # legacy
overuse_decay = 0.98           # legacy
max_spread_fraction = 0.10
max_inv_spread_fraction = 0.05
diffusion_depth = 2
max_hebb_edges = 32
max_inv_hebb_edges = 16
hebb_decay = 0.995
hebb_learn_rate = 0.01
forget_lti_floor = 0.001

```

```

[anti_precedence]
# "broad" = legacy broad-overuse rent (ablation baseline)
# "family_protected" = recommended value-relative anti-precedence
mode = family_protected
enabled = true
family_trace_decay = 0.98
value_beta = 4.0
overuse_threshold = 0.05
overuse_power = 2.0
anti_scale = 0.20 # xi
release_scale = 0.08 # rho
functional_protection_scale = 0.15 # eta_func
neighbor_release_temperature = 0.25 # tau in K(g, h)
min_family_flux_for_overuse = 10
value_smoothing = 0.95 # alpha_V for slow EMA on V_g
high_value_threshold = 0.30 # for triggering variant proposals
high_overuse_threshold = 0.10 # for triggering variant proposals

[anti_precedence_value]
# Weights for V_g = (w_U DeltaU + w_C C + w_G G + w_H H + w_R R)
# / (J + Cost + eps)
# - w_K K - w_B B
w_utility = 1.0
w_acs_coupling = 1.0
w_generativity = 0.5
w_transfer = 0.7
w_resilience = 0.7
w_cost = 0.2
w_lockin = 0.6

[ecan_pools]
warm_target = 10000
warm_hard_cap = 30000
hot_rules_per_context = 512
cold_to_warm_per_signature = 64
warm_to_hot_per_signature = 32
hot_quota_utility = 0.70
hot_quota_novelty = 0.15
hot_quota_rare_tail = 0.10
hot_quota_sb_guided = 0.05

[ecan_acs]
acs_dividend_interval_ticks = 25
min_acs_coupling = 0.05
min_acs_activity = 3
near_acs_defect_max = 2
rare_tail_percentile = 0.20
wage_acs_member = 0.02
wage_acs_catalyst = 0.04
wage_acs_macro = 0.06

[ecan_info_geom]
enabled = false

```

```

interval_ticks = 200
subgraph_cap = 2048
kl_weight = 0.1
transport_weight = 0.05
overuse_weight = 0.1
solver_iterations = 10

[optimal_transport]
enabled = false
ot_every_ticks = 50
ot_active_rule_cap = 1024
ot_neighbors_per_rule = 16
ot_solver_iterations = 20

```

```

[worker_backend]
initial_backend = "petta"
target_backend = "metta-il"
backend_rendering_tests = true
allow_shared_hot_pool_mutation = false

```

```

[petta_wrapper]
enabled = true
max_par_group_size = 64
context_chamber_workers = 32
moses_deme_workers = 8
acs_scan_workers = 4
replay_batch_workers = 4
bridge_fit_workers = 1
ecan_epoch_workers = 1
default_chamber_budget_ms = 50
reduce_period_ticks = 1
use_thread_pool = true
message_queue_max_size = 10000

```

```

[worker_ecan]
enabled = false_initially
worker_sti_budget = 1.0
worker_lti_budget = 1.0
worker_wage_sti = 0.02
worker_wage_lti = 0.002
worker_base_rent = 0.001
worker_cpu_rent_scale = 0.0001
worker_memory_rent_scale = 0.0001

```

U.2 Default serious config

```

[server]
target_cores = 96
target_ram_gb = 768
reserve_ram_fraction = 0.30

[soup]

```

```

persistent_rule_target = 300000
persistent_rule_hard_cap = 1000000
active_rule_target = 30000
active_rule_hard_cap = 100000

[context]
working_memory_fact_cap = 16384
scratch_atom_cap = 25000
active_signature_cap = 24
rules_per_signature_cap = 64

[inner_loop]
tick_budget_ms = 50
hot_rules_per_context = 2048
candidate_cap = 4096
fire_cap = 24
prediction_rollout_cap = 128

[traces]
trace_ttl_ticks = 8192
recent_trace_cap = 1000000
compress_old_traces = true

[learning]
logit_update_every_ticks = 1
credit_assignment_every_ticks = 10
structural_epoch_ticks = 300
mutation_fraction = 0.01
new_rule_insert_cap_per_epoch = 10000
retire_fraction_per_epoch = 0.02

[moses]
demes = 64
population_per_deme = 256
elite_per_deme = 32
eda_variables_cap = 256
factor_arity_cap = 4
nary_factor_cap = 512
cenf_every_generations = 10

[ecan]
sti_budget = 1.0
lti_budget = 1.0
wage_sti = 0.02
wage_lti = 0.002
base_hot_rent = 0.001
base_warm_rent = 0.0001
cost_rent_scale = 0.0005
overuse_rent_scale = 0.002      # legacy broad-overuse rent (mode = broad)
overuse_threshold = 5.0        # legacy
overuse_decay = 0.98           # legacy
max_spread_fraction = 0.10
max_inv_spread_fraction = 0.05
diffusion_depth = 2

```

```

max_hebb_edges = 64
max_inv_hebb_edges = 32
hebb_decay = 0.995
hebb_learn_rate = 0.01
forget_lti_floor = 0.001

[anti_precedence]
mode = family_protected
enabled = true
family_trace_decay = 0.98
value_beta = 4.0
overuse_threshold = 0.05
overuse_power = 2.0
anti_scale = 0.25          # xi
release_scale = 0.10       # rho
functional_protection_scale = 0.20 # eta_func
neighbor_release_temperature = 0.25
min_family_flux_for_overuse = 25
value_smoothing = 0.97
high_value_threshold = 0.30
high_overuse_threshold = 0.08

[anti_precedence_value]
w_utility      = 1.0
w_acs_coupling = 1.2
w_generativity = 0.6
w_transfer     = 0.8
w_resilience  = 0.7
w_cost         = 0.2
w_lockin       = 0.7

[ecan_pools]
warm_target = 30000
warm_hard_cap = 100000
hot_rules_per_context = 2048
cold_to_warm_per_signature = 128
warm_to_hot_per_signature = 64
hot_quota_utility = 0.70
hot_quota_novelty = 0.15
hot_quota_rare_tail = 0.10
hot_quota_sb_guided = 0.05

[ecan_acs]
acs_dividend_interval_ticks = 25
min_acs_coupling = 0.05
min_acs_activity = 3
near_acs_defect_max = 2
rare_tail_percentile = 0.20
wage_acs_member = 0.02
wage_acs_catalyst = 0.04
wage_acs_macro = 0.06

[ecan_info_geom]
enabled = false_initially

```

```

interval_ticks = 200
subgraph_cap = 4096
kl_weight = 0.1
transport_weight = 0.05
overuse_weight = 0.1
solver_iterations = 10

[ecan_actpc_coupling]
enabled = false_initially
kl_p_to_a = 0.05
kl_a_to_p = 0.02

[optimal_transport]
enabled = false_initially
ot_every_ticks = 100
ot_active_rule_cap = 2048
ot_neighbors_per_rule = 16
ot_solver_iterations = 20

```

```

[worker_backend]
initial_backend = "petta"
target_backend = "metta-il"
backend_rendering_tests = true
allow_shared_hot_pool_mutation = false

```

```

[petta_wrapper]
enabled = true
max_par_group_size = 192
context_chamber_workers = 64
moses_deme_workers = 32
acs_scan_workers = 16
replay_batch_workers = 16
bridge_fit_workers = 4
ecan_epoch_workers = 2
default_chamber_budget_ms = 50
reduce_period_ticks = 1
use_thread_pool = true
message_queue_max_size = 100000

```

```

[worker_ecan]
enabled = true
worker_sti_budget = 1.0
worker_lti_budget = 1.0
worker_wage_sti = 0.02
worker_wage_lti = 0.002
worker_base_rent = 0.001
worker_cpu_rent_scale = 0.0001
worker_memory_rent_scale = 0.0001
worker_utility_weight = 1.0
worker_novelty_weight = 0.2
worker_acs_yield_weight = 0.5

```

U.3 Minimal experiment configuration

```
experiment: bug_simple
seed: 12345
max_episodes: 1000
max_ticks_per_episode: 50

inner_loop:
  candidate_cap_default: 64
  temperature_initial: 1.0
  temperature_final: 0.2
  eta_logit: 0.05
  lambda_cost: 0.001
  prior_pull: 0.0001
  gamma_trace: 0.95
  max_trace_age: 100

rewards:
  alpha_env: 1.0
  alpha_int: 0.2
  alpha_ep: 0.05
  eta_entropy: 0.01

structural_loop:
  enabled: true
  run_every_ticks: 100
  max_trial_rules_per_run: 20
  min_trial_uses: 30
  promote_credit_threshold: 0.10
  retire_credit_threshold: -0.05

anti_precedence:
  # mode: "broad" for legacy ablation, "family_protected" for recommended
  mode: family_protected
  enabled: true
  family_trace_decay: 0.98
  value_beta: 4.0
  overuse_threshold: 0.05
  overuse_power: 2.0
  anti_scale: 0.20
  release_scale: 0.08
  functional_protection_scale: 0.15
  neighbor_release_temperature: 0.25
  min_family_flux_for_overuse: 10
  value_smoothing: 0.95

anti_precedence_value:
  w_utility: 1.0
  w_acs_coupling: 1.0
  w_generativity: 0.5
  w_transfer: 0.7
  w_resilience: 0.7
  w_cost: 0.2
  w_lockin: 0.6
```



```

ot_step:
  enabled: false
  step_size: 0.02
  damping: 0.001
  max_iter: 50

```

```

worker_backend:
  initial_backend: petta
  target_backend: metta-il
  backend_rendering_tests: true
  allow_shared_hot_pool_mutation: false

```

```

petta_wrapper:
  enabled: true
  max_par_group_size: 1
  context_chamber_workers: 1
  use_thread_pool: false

```

V References and further reading

- Ben Goertzel, *ActPC-Chem: Discrete Active Predictive Coding for Goal-Guided Algorithmic Chemistry as a Potential Cognitive Kernel for Hyperon and PRIMUS-Based AGI*.
- Ben Goertzel, *Reverse-Engineering the Origin of Life: A Geoteleomic (GEOdesics + TELEOlogy) Path to Autocatalysis, Self-Reproduction, and Selection in Self-Organizing Chemical Networks & Algorithmic Chemistry*, technical report. The near-ACS defect score, rare-tail enrichment metric, and Schrödinger-Bridge / geoteleomic guidance machinery used in this document are adapted from this work.
- Ben Goertzel, *Precedence, Anti-Precedence, and Open-Ended Intelligence: A Systems-Theoretic Formalization of Individuation, Self-Transformation, and Value-Relative Release*, working draft, April 2026. The value-relative anti-precedence formulation used here (S_g , S_g^* , O_g , V_g , functional protection c_r , structured release R_g , and the family-level effective propensity π_r^{eff}) is adapted directly from this paper, in particular its v8 Lane-seep simulation results showing that protected release outperforms both broad biphasic memory and a conversion-bridge alone in shock and takeover protocols.
- Matthew Ikle, Joel Pitt, Ben Goertzel, and George Sellman. *Economic Attention Networks: Associative Memory and Resource Allocation for General Intelligence*. AGI 2009. The classical ECAN formulation: STI/LTI currencies, Hebbian and InverseHebbian links, wages, rent, tax, welfare, diffusion, and forgetting.
- Matthew Ikle and Ben Goertzel. *Nonlinear-Dynamical Attention Allocation via Information Geometry*. AGI 2011. The information-geometric ECAN formulation, including adaptive natural-gradient learning (ECAN+ANGL) over attentional probability distributions, on which the mirror-descent and Wasserstein ECAN updates in this document are based.
- Shun-ichi Amari, *Natural Gradient Works Efficiently in Learning*, Neural Computation, 1998. Foundation for the Fisher-Rao geometry used in information-geometric ECAN.

- Shun-ichi Amari and Hiroshi Nagaoka, *Methods of Information Geometry*, AMS, 2000.
- The MORK wiki pages on MM2 operational semantics, data representation, prefix-sensitive PathMap indexing, and sources/sinks.
- The MOSES-on-MORK design notes covering GCoDD, ENF/CENF normal forms, and factor-graph EDA.
- Origin-of-life literature on autocatalytic sets and RAFs (reflexively autocatalytic, food-generated sets) more broadly.
- Active predictive coding, Friston-style free-energy and learning-progress epistemic rewards.
- Discrete optimal-transport and Wasserstein natural-gradient methods on graphs.